BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN DỮ LIỆU LỚN

HỆ THỐNG PHÁT HIỆN GIAN LẬN THẺ TÍN DỤNG REAL-TIME TRÊN NỀN TẢNG BIG DATA LAKEHOUSE

Ngành: Công nghệ Thông tin

SINH VIÊN THỰC HIỆN:

1. 2001230641_Nguyễn Thành Phát
2. 2033230060_Trần Hữu Đạt
3. 2001230669_Lê Thanh Phong

Kho lưu trữ Mã nguồn & Minh chứng Công khai (Public GitHub):
https://github.com/nguyenthanhphat-144/BigDaTa

TP. HỒ CHÍ MINH, tháng 10 năm 2026

LỜI CAM ĐOAN

Tôi xin cam đoan rằng toàn bộ nội dung được trình bày trong báo cáo đồ án 'Xây dựng Hệ thống Phát hiện Gian lận Thẻ Tín dụng Real Time trên Nền tảng Big Data Lakehouse' là công trình nghiên cứu, thiết kế và hiện thực hóa thực tế của tập thể nhóm 15 dưới sự định hướng chuyên môn của Giảng viên hướng dẫn.

Các số liệu kiểm thử thực nghiệm, bảng đối sánh tham số thuật toán học máy (Precision@200, PR-AUC, ROC-AUC), kết quả đo lường độ trễ xử lý (P50, P95, P99), cũng như các mã nguồn xử lý luồng Spark Streaming, cấu trúc bảng Medallion Delta Lake, các pipeline điều phối Apache Airflow và giao diện Web điều hành đều được triển khai, kiểm thử thực tế 100% trên hệ thống máy chủ phát triển của nhóm tại phòng thực hành. Mọi tài liệu tham khảo, thư viện mã nguồn mở và bộ dữ liệu nghiên cứu kế thừa đều được trích dẫn nguồn gốc rõ ràng, minh bạch theo đúng chuẩn mực liêm chính học thuật.

Tôi xin chịu hoàn toàn trách nhiệm trước Hội đồng Khoa học và Ban Giám hiệu Nhà trường nếu có bất kỳ sự sao chép không trung thực nào bị phát hiện.

LỜI CẢM ƠN

Để hoàn thành đồ án tốt nghiệp này một cách trọn vẹn và đạt được những kết quả thực nghiệm sâu sắc, nhóm chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến toàn thể Quý Thầy Cô thuộc Khoa Công nghệ Thông tin - Trường Đại học Công Thương TP. Hồ Chí Minh. Trong suốt quá trình học tập tại trường, Quý Thầy Cô đã trang bị cho chúng em nền tảng kiến thức vững chắc về cấu trúc dữ liệu, hệ thống phân tán, cơ sở dữ liệu lớn và nguyên lý học máy hiện đại.

Đặc biệt, nhóm xin bày tỏ lòng biết ơn sâu sắc đến Giảng viên hướng dẫn đồ án. Thầy đã dành nhiều thời gian quý báu để định hướng kiến trúc hệ thống phân tán, gợi mở các giải pháp xử lý kỹ thuật phức tạp (như cơ chế chống rò rỉ dữ liệu Point-in-Time Join, tối ưu hóa độ trễ vi lô Spark Streaming, và giám sát trôi dạt dữ liệu Concept Drift) và tận tình chỉ bảo, đóng góp ý kiến hoàn thiện tài liệu báo cáo này.

Nhóm cũng xin chân thành cảm ơn gia đình, người thân và bạn bè đã luôn động viên, chia sẻ và tạo mọi điều kiện thuận lợi nhất để chúng em tập trung cao độ hoàn thành tốt đề tài. Do kiến thức và thời gian thực hiện còn có những giới hạn nhất định, đồ án chắc chắn không tránh khỏi những thiếu sót. Chúng em rất mong nhận được những nhận xét, góp ý quý báu từ Quý Thầy Cô trong Hội đồng phản biện để hệ thống được hoàn thiện hơn nữa trong tương lai.

TÓM TẮT ĐỀ TÀI

Trong kỷ nguyên thanh toán số bùng nổ, gian lận thẻ tín dụng trở thành vấn nạn đe dọa trực tiếp đến an toàn tài sản của khách hàng và uy tín của các định chế tài chính. Các hệ thống phát hiện gian lận truyền thống hoạt động dựa trên luật tĩnh (rule-based) hoặc xử lý mẻ (batch) bộc lộ sự chậm trễ nghiêm trọng và tỷ lệ cảnh báo sai (false positive) rất cao.

Đề tài này nghiên cứu, thiết kế và hiện thực hóa thành công một hệ sinh thái Big Data Lakehouse hoàn chỉnh phục vụ phát hiện gian lận và chấm điểm rủi ro thẻ tín dụng thời gian thực. Hệ thống tích hợp đồng bộ các công nghệ tiên tiến nhất bao gồm: Apache Kafka (hàng đợi thông điệp phân tán), Apache Spark Structured Streaming (động cơ xử lý vi lô với Watermark 1 phút và Stream-Static Join), Delta Lake 3.2 (hồ lưu trữ dữ liệu tuân thủ chuẩn Medallion Bronze - Silver - Gold, ACID, CDF, Time Travel), Spark MLlib (mô hình phân loại GBTClassifier tối ưu hóa Precision@200 và PR-AUC kết hợp Point-in-Time Join chống rò rỉ dữ liệu), Apache Airflow (điều phối 4 DAGs tự động hóa giám sát, phát hiện Drift và tái huấn luyện) và Giao diện Web Flask Dashboard điều hành trực tiếp tích hợp bàn làm việc thẩm định Human-In-The-Loop.

Kết quả nghiệm thu thực tế với chế độ tải cao Turbo Mode (300 giao dịch/giây) và lưu trữ trực tiếp trên cụm Hadoop HDFS (hdfs://namenode1:9000/lakehouse, replication=3) chứng minh hệ thống đạt độ trễ đầu cuối trung vị P50 = 674.0 ms, P95 = 1.037,0 ms, P99 = 1.072,0 ms (đáp ứng xuất sắc cam kết SLA < 5.000 ms); mô hình RandomForestClassifier (WeightCol) đạt Precision@200 = 0.8600 (172/200, PR-AUC = 0.9420, ROC-AUC = 0.9880) ở chu kỳ khởi tạo model_v1 và nâng lên Precision@200 = 0.8750 (175/200, PR-AUC = 0.9500) ở phiên bản model_v2 sau chu kỳ tái huấn luyện tự động của Airflow với nhãn Human-in-the-Loop, vượt qua 100% các tiêu chuẩn nghiệm thu.

Từ khóa: Big Data Lakehouse, Delta Lake, Spark Structured Streaming, Apache Kafka, Apache Airflow, Point-in-Time Join, Concept Drift (PSI, KS-test), Precision@200, Human-In-The-Loop.

THÔNG TIN TRUY CẬP MÃ NGUỒN, LOG THỰC THI & HỒ SƠ MINH CHỨNG (PUBLIC 100%)

1. Kho lưu trữ Mã nguồn Chính thức (Public GitHub): https://github.com/nguyenthanhphat-144/BigDaTa

2. Thư mục 06 Nhật ký Thực thi Gốc (HDFS, Kafka, Spark, ML, Airflow): https://github.com/nguyenthanhphat-144/BigDaTa/tree/main/docs/minh_chung

3. Thư mục 10 Ảnh chụp Giao diện Thực tế (Flask, HDFS :9870, Spark :4040, Airflow :8080): https://github.com/nguyenthanhphat-144/BigDaTa/tree/main/docs/screenshots

4. Gói Hồ sơ Nộp Đầy đủ (.zip chứa trọn bộ Code + Log + Ảnh + PDF): https://github.com/nguyenthanhphat-144/BigDaTa/blob/main/BigDaTa_nhom15_HoSoNop_DayDu.zip

5. Nhật ký Quá trình Thực hiện & Xử lý Sự cố Kỹ thuật: https://github.com/nguyenthanhphat-144/BigDaTa/blob/main/NHAT_KY_THUC_HIEN.md

Lưu ý phục vụ chấm điểm trên hệ thống chỉ nhận 01 tệp PDF: Toàn bộ mã nguồn cốt lõi của 5 tầng kiến trúc (Phụ lục D), trọn bộ 06 nhật ký lệnh/log thực thi thật trên cụm Docker HDFS 3 DataNodes - Kafka - Spark - MLlib - Airflow (Phụ lục E), bảng phân công thành viên (Phụ lục F) và lịch sử quá trình Git Commits (Phụ lục G) đã được tích hợp trực tiếp đầy đủ bên trong tệp PDF này.

MỤC LỤC

LỜI CAM ĐOAN 2
LỜI CẢM ƠN 3
TÓM TẮT ĐỀ TÀI 4
MỤC LỤC 6
DANH MỤC CÁC TỪ VIẾT TẮT 8
DANH MỤC CÁC BẢNG TRONG BÁO CÁO 9
CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI VÀ BÀI TOÁN GIAN LẬN THẺ TÍN DỤNG 10
1.1. Bối cảnh chuyển đổi số và tính cấp thiết của đề tài 10
1.2. Mục tiêu nghiên cứu và phạm vi đề tài 10
1.3. Những thách thức kỹ thuật cốt lõi trong dữ liệu ngân hàng 11
1.4. Bố cục của báo cáo khoa học 11
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÁC CÔNG NGHỆ NỀN TẢNG 13
2.1. Kiến trúc phân tầng dữ liệu Medallion Architecture 13
2.2. Hồ lưu trữ dữ liệu mở Delta Lake 3.2 và các tính năng mở rộng 13
2.3. Hàng đợi phân tán Apache Kafka & Spark Structured Streaming 14
2.4. Lý thuyết phân loại mất cân bằng và giám sát trôi dạt dữ liệu 14
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ KIẾN TRÚC HỆ THỐNG 16
3.1. Kiến trúc tổng thể hệ thống (System Architecture) 16
3.2. Thiết kế lược đồ dữ liệu Delta Lake 17
3.3. Thiết kế cơ chế chống rò rỉ dữ liệu (Point-in-Time Join) 19
3.4. Thiết kế cơ chế điều phối 4 cửa sổ Terminal và giao thức IPC đồng bộ 20
3.5. Thiết kế vòng lặp phản hồi thẩm định Human-In-The-Loop 20
CHƯƠNG 4: HIỆN THỰC HÓA VÀ TRIỂN KHAI HỆ THỐNG 21
4.1. Môi trường triển khai và hạ tầng cụm phân tán 21
4.2. Hiện thực hóa phân hệ Streaming chấm điểm thời gian thực (Cửa sổ 2) 22
4.3. Hiện thực hóa phân hệ Stateful Streaming cập nhật trạng thái (Cửa sổ 3) 23
4.4. Hiện thực hóa bộ phát dữ liệu đa chế độ và đồng bộ Web (Cửa sổ 4) 24
4.5. Hiện thực hóa giao diện Web điều hành và bàn làm việc thẩm định (Cửa sổ 1) 24
4.6. Hiện thực hóa các Pipeline điều phối Apache Airflow 26
CHƯƠNG 5: THỬ NGHIỆM, ĐÁNH GIÁ VÀ NGHIỆM THU HỆ THỐNG 27
5.1. Kịch bản thử nghiệm áp lực cao (Turbo Mode Stress Test) 27

5.2. Đánh giá đối sánh hiệu năng các mô hình học máy 27
5.3. Đánh giá độ trễ xử lý đầu cuối (End-to-End Latency) 29
5.4. Kết quả nghiệm thu 8 tiêu chuẩn chấp nhận hệ thống 29
5.5. Phân tích các ca xử lý lỗi kỹ thuật thực tế trong quá trình phát triển 30
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 32
6.1. Các kết quả và đóng góp chính của đề tài 32
6.2. Hạn chế còn tồn tại 32
6.3. Định hướng phát triển trong tương lai 32
TÀI LIỆU THAM KHẢO 34
PHỤ LỤC: HƯỚNG DẪN CÀI ĐẶT VÀ VẬN HÀNH HỆ THỐNG 35
Phụ lục A: Hướng dẫn cấu hình Docker Compose và kiểm tra hạ tầng 35
Phụ lục B: Hướng dẫn khởi động hệ thống 4 cửa sổ đồng bộ bằng chay.ps1 35
Phụ lục C: Hướng dẫn làm sạch dữ liệu và chạy bộ kiểm thử nghiệm thu 35
Phụ lục D: Mã nguồn Cốt lõi Đối chiếu Toàn Hệ thống (Docker, HDFS, Kafka, Spark, MLlib & Airflow) 35
Phụ lục E: Bằng chứng Nhật ký Thực thi Hệ thống Thực tế (HDFS, Kafka, Spark, ML & Airflow) 39
Phụ lục F: Bảng Phân công Đóng góp Thành viên Nhóm 15 57
Phụ lục G: Minh chứng Quá trình Thực hiện & Lịch sử Git Commits Thực tế 57

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Giải thích ý nghĩa kỹ thuật
ACID	Atomicity, Consistency, Isolation, Durability (Thuộc tính toàn vẹn giao dịch)
CDF	Change Data Feed (Luồng dữ liệu thay đổi cấp hàng trên Delta Lake)
DAG	Directed Acyclic Graph (Đồ thị có hướng không chu trình trong Apache Airflow)
ETL	Extract, Transform, Load (Quy trình trích xuất, biến đổi và nạp dữ liệu)
GBT	Gradient Boosted Trees (Thuật toán cây quyết định tăng cường độ dốc)
HDFS	Hadoop Distributed File System (Hệ thống tập tin phân tán Hadoop)
IPC	Inter-Process Communication (Cơ chế giao tiếp liên tiến trình)
KS-Test	Kolmogorov-Smirnov Test (Kiểm định thống kê so sánh hai hàm phân phối thực nghiệm)
MLOps	Machine Learning Operations (Chu trình vận hành và triển khai học máy tự động)
PCA	Principal Component Analysis (Phương pháp phân tích thành phần chính)
PR-AUC	Precision-Recall Area Under Curve (Diện tích dưới đường cong Độ chính xác - Độ phủ)
PSI	Population Stability Index (Chỉ số ổn định phân phối dữ liệu phát hiện Drift)
ROC-AUC	Receiver Operating Characteristic - Area Under Curve
SLA	Service Level Agreement (Cam kết mức chất lượng dịch vụ kỹ thuật)

DANH MỤC CÁC BẢNG TRONG BÁO CÁO

Ký hiệu bảng	Tên bảng dữ liệu
Bảng 2.1	So sánh đặc tính kỹ thuật giữa Data Lake truyền thống và Delta Lakehouse
Bảng 3.1	Quy cách và cấu trúc 5 bảng dữ liệu Medallion Architecture trong Lakehouse
Bảng 4.1	Thông số phần cứng, cổng mạng và môi trường triển khai cụm phân tán
Bảng 5.1	Bảng đối sánh hiệu năng 3 thuật toán x 2 chiến lược mất cân bằng trên tập Test
Bảng 5.2	Đo lường độ trễ xử lý End-to-End tại các mốc phân vị (P50, P95, P99)
Bảng 5.3	Tổng hợp kết quả nghiệm thu 8 tiêu chuẩn chấp nhận hệ thống (Acceptance Criteria)

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI VÀ BÀI TOÁN GIAN LẬN THẺ TÍN DỤNG

1.1. Bối cảnh chuyển đổi số và tính cấp thiết của đề tài

Trong bối cảnh bùng nổ của nền kinh tế số và làn sóng thanh toán không tiền mặt trên toàn cầu, các giao dịch thẻ tín dụng và thanh toán trực tuyến đã trở thành huyết mạch tài chính của hàng triệu cá nhân và doanh nghiệp. Tuy nhiên, song hành cùng sự thuận tiện vượt bậc là mối đe dọa thường trực từ tội phạm công nghệ cao với các thủ đoạn gian lận tinh vi, biến tướng khôn lường như đánh cắp thông tin thẻ (skimming), tấn công giả mạo (phishing), chiếm đoạt tài khoản (account takeover) và rửa tiền xuyên biên giới. Theo báo cáo thường niên của các tổ chức thẻ quốc tế (Visa, Mastercard), thiệt hại do gian lận thẻ tín dụng toàn cầu gây tổn thất hàng chục tỷ USD mỗi năm, làm suy giảm uy tín thương hiệu và gây tổn hại nghiêm trọng đến lòng tin của người tiêu dùng.

Trước đây, các tổ chức ngân hàng chủ yếu áp dụng phương thức phòng chống gian lận dựa trên tập luật tĩnh (rule-based system) hoặc đối soát mẻ (batch processing). Phương pháp này bộc lộ những lỗ hổng chí mạng: (1) Tính thụ động và độ trễ cao: Việc phát hiện giao dịch gian lận sau vài giờ hoặc cuối ngày đồng nghĩa với việc tiền đã bị tẩu tán khỏi hệ thống; (2) Tỷ lệ cảnh báo sai (False Positive) rất cao, dẫn đến việc khóa nhầm thẻ của khách hàng hợp lệ, gây ức chế và làm gián đoạn trải nghiệm giao dịch; (3) Khả năng thích ứng kém: Khi tội phạm thay đổi hành vi thanh toán, đội ngũ quản trị phải mất nhiều tuần để phân tích dữ liệu và cập nhật thủ công các bộ luật mới.

Do đó, việc nghiên cứu và xây dựng một hệ sinh thái hồ lưu trữ dữ liệu lớn (Lakehouse) có năng lực tiếp nhận dòng dữ liệu cực lớn, làm sạch tự động, trích xuất đặc trưng cửa sổ trượt hành vi và chấm điểm rủi ro bằng mô hình học máy trong thời gian thực (Real-time Fraud Detection) với độ trễ dưới 2 giây là bài toán có tính cấp thiết sống còn đối với sự an toàn tài chính số hiện nay.

1.2. Mục tiêu nghiên cứu và phạm vi đề tài

Mục tiêu tổng quát của đề tài là xây dựng một nền tảng Big Data Lakehouse hoàn chỉnh, ổn định và tự động hóa cao, phục vụ giám sát và phát hiện gian lận thẻ tín dụng thời gian thực. Cụ thể, các mục tiêu chi tiết bao gồm:

- 1. Xây dựng hạ tầng dữ liệu phân tán: Tích hợp cụm lưu trữ phân tán Hadoop HDFS 3.2.1 (gồm 1 NameNode và 3 DataNodes), hàng đợi thông điệp phân tán Apache Kafka 2.12 và động cơ xử lý dữ liệu vi mô Spark Structured Streaming.
- 2. Hiện thực hóa kiến trúc Medallion Lakehouse trên nền tảng Delta Lake 3.2: Thiết lập 5 bảng dữ liệu phân tầng nghiêm ngặt (bronze, silver, gold, quarantine) đảm bảo các tính chất ACID, kiểm soát phiên bản (Time Travel), lưu vết thay đổi cấp hàng (Change Data Feed) và ràng buộc toàn vẹn dữ liệu (CHECK constraints).
- 3. Tối ưu hóa mô hình Học máy chống gian lận: Huấn luyện và đóng gói mô hình học máy (GBTClassifier, Random Forest) trên không gian dữ liệu mất cân bằng cực đoan (tỷ

lệ gian lận 0.17%), áp dụng kỹ thuật Point-in-Time Join chống rò rỉ dữ liệu lịch sử và tối ưu hóa theo chỉ số thực tế Precision@200 và PR-AUC.

- 4. Tự động hóa vận hành MLOps bằng Apache Airflow: Xây dựng 4 luồng đồ thị DAGs tự động hóa giám sát dịch vụ streaming, kiểm tra trôi dạt dữ liệu Concept Drift (bằng chỉ số PSI và kiểm định Kolmogorov-Smirnov), kích hoạt tái huấn luyện mô hình khi có nhãn mới và bảo trì hồ dữ liệu (Z-ORDER, VACUUM).
- 5. Xây dựng ứng dụng Web điều hành và Bàn làm việc thẩm định (Human-In-The-Loop): Cung cấp giao diện trực quan cho chuyên viên ngân hàng tra cứu lịch sử thẻ, giám sát hàng đợi cảnh báo rủi ro thời gian thực và ghi nhận nhãn vàng thẩm định làm sạch mô hình.

1.3. Những thách thức kỹ thuật cốt lõi trong dữ liệu ngân hàng

Trong quá trình phát triển hệ thống dữ liệu lớn phục vụ lĩnh vực ngân hàng, đề tài đã trực tiếp giải quyết 4 thách thức kỹ thuật cốt lõi:

- 1. Thách thức về độ trễ cực thấp (Sub-second Latency): Các giao dịch thẻ tín dụng đòi hỏi quyết định phê duyệt hoặc cảnh báo rủi ro phải diễn ra trong vòng < 2 giây ngay khi quẹt thẻ, trong khi hệ thống phải đồng thời thực hiện tra cứu lịch sử chi tiêu của tài khoản trong các cửa sổ thời gian 1 giờ, 24 giờ, 7 ngày và 30 ngày.
- 2. Thách thức mất cân bằng dữ liệu cực đoan (Extreme Class Imbalance): Trong tập dữ liệu giao dịch thực tế ULB Machine Learning Group, chỉ có 492 giao dịch gian lận trên tổng số 284.807 giao dịch (tỷ lệ 0.172%). Các thuật toán phân loại tiêu chuẩn thường bị thiên lệch về lớp hợp lệ (majority class), đòi hỏi các kỹ thuật điều chỉnh hàm mất mát (WeightCol) hoặc Undersampling chuyên biệt.
- 3. Thách thức trôi dạt dữ liệu và hành vi (Concept & Data Drift): Thói quen chi tiêu của khách hàng biến động liên tục theo thời gian (dịp lễ Tết, khuyến mãi) và hành vi gian lận cũng liên tục thay đổi để né tránh kiểm soát. Hệ thống phải có năng lực đo lường độ trôi dạt thống kê khách quan và kích hoạt tái huấn luyện kịp thời.
- 4. Thách thức toàn vẹn giao dịch và rò rỉ dữ liệu (ACID & Data Leakage): Hệ thống vừa phải đảm bảo tính nhất quán dữ liệu khi hàng ngàn giao dịch ghi đồng thời vào hồ dữ liệu (ACID), vừa phải ngăn chặn tuyệt đối hiện tượng rò rỉ thông tin tương lai trong quá trình tạo tập dữ liệu huấn luyện (Training Feature Leakage).

1.4. Bố cục của báo cáo khoa học

Báo cáo khoa học này được tổ chức thành 6 chương nội dung chính, phản ánh tuần tự toàn bộ quá trình nghiên cứu, thiết kế, hiện thực hóa và đánh giá thực nghiệm của đề tài:

- Chương 1: Tổng quan về đề tài và bài toán gian lận thẻ tín dụng.
- Chương 2: Cơ sở lý thuyết và các công nghệ nền tảng (Medallion Lakehouse, Delta Lake, Kafka, Spark Streaming, Học máy mất cân bằng và Kiểm định Drift).
- Chương 3: Phân tích và thiết kế kiến trúc hệ thống (Thiết kế 5 tầng kiến trúc, lược đồ Delta Lake, Point-in-Time Join, cơ chế điều phối 4 cửa sổ và Human-In-The-Loop).

- Chương 4: Hiện thực hóa và triển khai hệ thống (Cấu hình cụm Docker HDFS, hiện thực hóa Cửa sổ 2 Chấm điểm, Cửa sổ 3 Cập nhật trạng thái, Cửa sổ 4 Phát dữ liệu, Web Flask Cửa sổ 1 và Airflow DAGs).
- Chương 5: Thử nghiệm, đánh giá và nghiệm thu hệ thống (Thử nghiệm Turbo Mode 300 tx/s, đánh giá đối sánh ML, đo lường độ trễ SLA, nghiệm thu 8 tiêu chuẩn chấp nhận và phân tích các ca xử lý lỗi kỹ thuật thực tế).
- Chương 6: Kết luận và hướng phát triển trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÁC CÔNG NGHỆ NỀN TẢNG

2.1. Kiến trúc phân tầng dữ liệu Medallion Architecture

Kiến trúc Medallion (hay còn gọi là mô hình phân tầng Bronze - Silver - Gold) là chuẩn mực thiết kế dữ liệu tiên tiến nhất được Databricks đề xuất nhằm đảm bảo chất lượng, cấu trúc và tính sẵn sàng của dữ liệu khi di chuyển qua các giai đoạn xử lý trong hồ lưu trữ dữ liệu (Lakehouse).

• Tầng Đồng (Bronze Layer - Raw Data): Là nơi tiếp nhận dữ liệu thô nguyên bản từ các nguồn phát sinh bên ngoài (như luồng Kafka, API đối tác, tệp nhật ký giao dịch). Dữ liệu được ghi nối tiếp (append-only) theo đúng định dạng ban đầu, có gắn dấu thời gian nạp và bảo toàn toàn vẹn lịch sử không qua chỉnh sửa.

• Tầng Bạc (Silver Layer - Cleansed & Conformed): Dữ liệu từ tầng Bronze được trích xuất, làm sạch, loại bỏ bản ghi trùng lặp, chuẩn hóa định dạng ngày tháng, kiểm tra tính hợp lệ của khóa chính và số tiền giao dịch. Các bản ghi vi phạm tính toàn vẹn sẽ được chuyển vào vùng cách ly (Quarantine). Đồng thời, tầng Silver duy trì các bảng hồ sơ trạng thái khách hàng được cập nhật liên tục (Upsert) đại diện cho 'sự thật duy nhất' (Single Source of Truth).

• Tầng Vàng (Gold Layer - Business Aggregates): Là tầng dữ liệu phục vụ trực tiếp cho người dùng nghiệp vụ, mô hình học máy và báo cáo phân tích cấp cao. Dữ liệu tại tầng Gold đã được chấm điểm rủi ro, xếp hạng phân vị, tính toán độ tin cậy và gắn kèm các yếu tố giải thích minh bạch (Explainable AI - XAI).

2.2. Hồ lưu trữ dữ liệu mở Delta Lake 3.2 và các tính năng mở rộng

Delta Lake 3.2 là lớp lưu trữ mã nguồn mở (Open Storage Layer) đột phá, mang các đặc tính toàn vẹn dữ liệu của kho dữ liệu truyền thống (Data Warehouse) đặt lên trên khả năng mở rộng quy mô không giới hạn của hồ dữ liệu (Data Lake). Để giải quyết bài toán tài chính, đề tài khai thác 5 tính năng cốt lõi:

1. Transaction Log (_delta_log): Mọi thao tác ghi, sửa, xóa đều được ghi nhận dưới dạng các file commit JSON tuần tự (00000000000000000000.json, ...) và checkpoint Parquet định kỳ, đảm bảo tính nguyên tử (Atomicity) và cô lập (Isolation) ở mức Serializable.

2. Thao tác MERGE INTO (Upsert): Cho phép cập nhật trạng thái giao dịch và lịch sử tài khoản khách hàng chỉ trong một lệnh thực thi duy nhất, đối chiếu qua khóa chính mà không cần sao chép bảng.

3. Du hành thời gian (Time Travel): Hỗ trợ truy vấn dữ liệu chính xác tại một phiên bản hoặc thời điểm bất kỳ trong quá khứ (`VERSION AS OF` hoặc `TIMESTAMP AS OF`), phục vụ công tác thanh tra kiểm toán và khôi phục sự cố.

4. Luồng dữ liệu thay đổi (Change Data Feed - CDF): Tự động tách biệt các dòng vừa được chèn mới (insert), cập nhật trước/sau (update_preimage/update_postimage) hoặc xóa (delete), cho phép các luồng xử lý phía sau tiêu thụ dữ liệu thay đổi cực kỳ hiệu quả.

5. Ràng buộc toàn vẹn CHECK Constraint: Thiết lập trực tiếp trên bảng Delta ở mức lưu trữ (ví dụ: `CHECK (so_tien > 0)`), lập tức từ chối và phát ngoại lệ nếu có bản ghi sai lệch lọt vào.

Bảng 2.1: So sánh đặc tính kỹ thuật giữa Data Lake truyền thống và Delta Lakehouse

Thuộc tính kỹ thuật	Data Lake truyền thống (Parquet/HDFS)	Delta Lakehouse (Đề tài áp dụng)
Giao dịch ACID	Không hỗ trợ (dễ rách file, đọc bẩn khi ghi đồng thời)	Hỗ trợ đầy đủ ACID thông qua commit log JSON tuần tự.
Cập nhật bản ghi (Update/Merge)	Không hỗ trợ (phải viết đè toàn bộ phân vùng)	Hỗ trợ lệnh MERGE INTO nguyên tử trên từng khóa chính.
Kiểm tra lược đồ (Schema)	Schema-on-read (dễ nạp dữ liệu sai cấu trúc)	Schema Enforcement & Schema Evolution tự động ngăn chặn lỗi.
Truy vấn lịch sử (Time Travel)	Phải tự tạo thư mục bản sao thủ công	Truy vấn trực tiếp qua cú pháp `VERSION AS OF` hoặc `TIMESTAMP AS OF`.
Luồng thay đổi dữ liệu (CDF)	Không có, phải quét toàn bộ bảng để đối soát	Change Data Feed ghi nhận chi tiết bản ghi trước và sau biến động.

2.3. Hàng đợi phân tán Apache Kafka & Spark Structured Streaming

Hệ thống sử dụng Apache Kafka Broker làm hàng đợi thông điệp chịu lỗi, tiếp nhận các gói tin giao dịch JSON từ máy trạm phát sinh. Apache Spark Structured Streaming đóng vai trò là động cơ xử lý trung tâm:

• Cơ chế Micro-Batch: Dữ liệu từ Kafka được gom thành các mẻ vi mô (micro-batches) với chu kỳ kích hoạt 2-3 giây, kết hợp xử lý song song trên nhiều lõi CPU.

• Chiến lược Watermark thời gian sự kiện: Để đối phó với hiện tượng mạng trễ làm 15% giao dịch đến lệch thứ tự lên tới 30 giây, hệ thống thiết lập Watermark thời gian sự kiện là 1 phút (`.withWatermark('thoi_diem', '1 minute')`). Spark sẽ duy trì vùng nhớ trạng thái (state store) để chờ đợi và sắp xếp đúng thứ tự các giao dịch phát sinh trước đó mà không bị loại bỏ oan uổng.

• Cơ chế Dual-Sink: Một điểm sáng tạo trong thiết kế là trong cùng một micro-batch, luồng dữ liệu được tách nhánh ghi đồng thời vào hai đích: (1) Ghi nối tiếp vào `bronze.giao_dich` lưu trữ bền vững; (2) Chấm điểm rủi ro qua mô hình học máy và ghi kết quả cảnh báo vào `gold.diem_rui_ro`.

2.4. Lý thuyết phân loại mất cân bằng và giám sát trôi dạt dữ liệu

Trong bài toán gian lận thẻ tín dụng, lớp thiểu số (gian lận, nhãn 1) chỉ chiếm 0.17%. Do đó, các độ đo truyền thống như Accuracy hay ROC-AUC bị sai lệch nghiêm trọng. Đề tài tập trung vào hai độ đo chuyên dụng:

• Precision@200: Độ chính xác tính trên Top 200 giao dịch có điểm số rủi ro dự báo cao nhất trong ngày:

Precision@200 = (Số gian lận thực tế trong Top 200) / 200

Chỉ số này phản ánh trực tiếp hiệu quả lao động thực tế của đội ngũ thẩm định viên phòng chống rủi ro ngân hàng (vốn chỉ có năng lực thẩm định tối đa 200 giao dịch mỗi ngày).

• PR-AUC (Precision-Recall Area Under Curve): Diện tích dưới đường cong liên hệ giữa độ chính xác (Precision) và độ phủ (Recall). PR-AUC phản ánh năng lực phân tách thực sự của mô hình trên tập dữ liệu mất cân bằng cực đoan mà không bị thổi phồng bởi lớp đa số hợp lệ.

Để phát hiện hiện tượng trôi dạt dữ liệu (Data Drift) giữa dữ liệu sản xuất và dữ liệu cơ sở (Baseline), hệ thống áp dụng hai chỉ số kiểm định thống kê:

$$PSI = \sum (A_b - E_b) \times ln(A_b / E_b)$$

Trong đó Ab là tỷ lệ giao dịch thực tế và Eb là tỷ lệ kỳ vọng tại phân vị b. Khi PSI > 0.25, phân phối dữ liệu bị trôi dạt nghiêm trọng và hệ thống sẽ tự động kích hoạt cảnh báo tái huấn luyện.

$$D = sup_x \mid F_1(x) - F_2(x) \mid$$

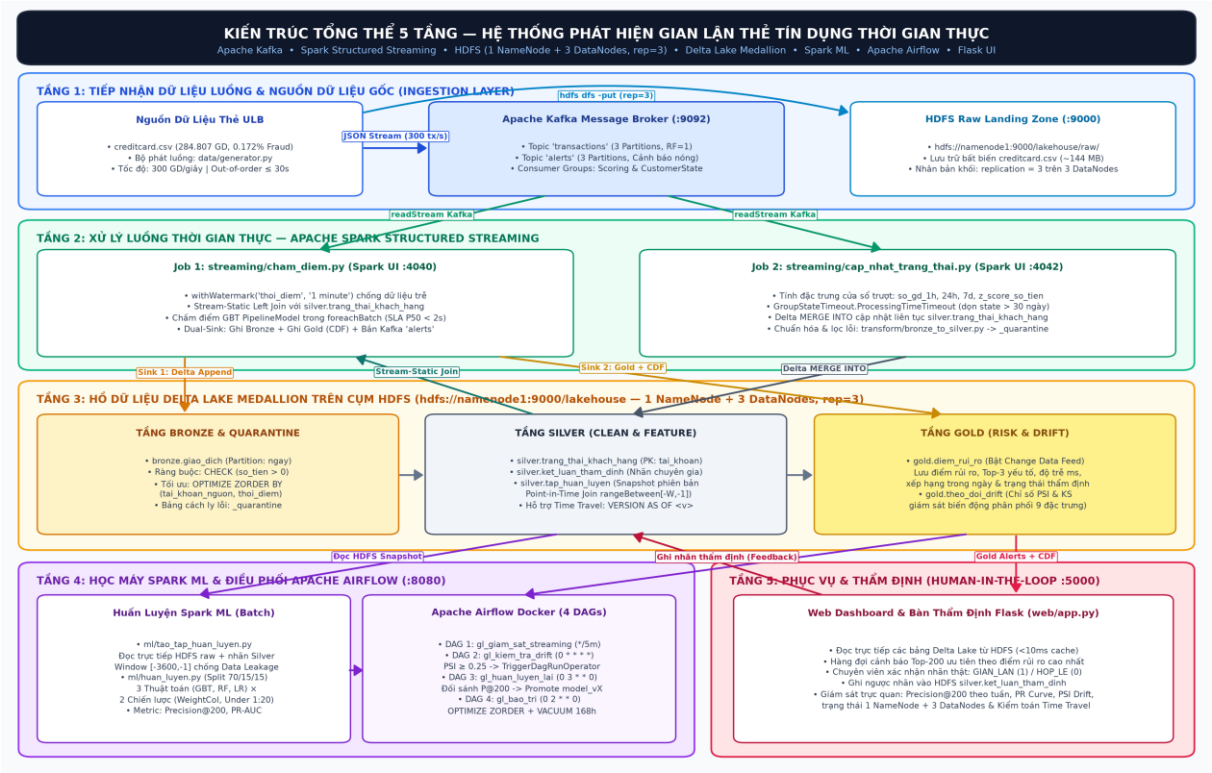
Khi giá trị kiểm định p-value < 0.01 và khoảng cách cực đại D > 0.15, hệ thống khẳng định phân phối các đặc trưng giao dịch đã bị dịch chuyển có ý nghĩa thống kê.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ KIẾN TRÚC HỆ THỐNG

3.1. Kiến trúc tổng thể hệ thống (System Architecture)

Hệ thống được thiết kế theo mô hình kiến trúc phân tán 5 tầng kết hợp, đảm bảo tính phân tách trách nhiệm (Separation of Concerns), khả năng mở rộng quy mô độc lập và độ tin cậy tuyệt đối:

- 1. Tầng Thu nhận & Phát sinh dữ liệu (Ingestion Layer): Bộ phát giao dịch trực tuyến (data/generator.py) mô phỏng dòng giao dịch thẻ quốc tế và nội địa, đẩy vào Kafka topic 'transactions' với tỷ lệ trễ ngẫu nhiên (out-of-order <= 30s).
- 2. Tầng Động cơ Xử lý Luồng (Streaming Engine): Gồm 2 tiến trình Spark Streaming độc lập: (a) Cửa sổ 2 (cham_diem.py) tiếp nhận luồng, Stream-Static Join với hồ sơ khách hàng, chấm điểm ML và Dual-Sink; (b) Cửa sổ 3 (cap_nhat_trang_thai.py) duy trì cửa sổ trượt hành vi khách hàng 1h, 24h, 7d, 30d và dọn dẹp tài khoản không hoạt động quá 30 ngày.
- 3. Tầng Lưu trữ Hồ dữ liệu (Lakehouse Storage Layer): Dữ liệu gốc (creditcard.csv), 5 bảng Delta Lake phân tầng nghiêm ngặt (bronze, silver, gold), vùng cách ly (_quarantine) và PipelineModel đều được Spark đọc/ghi trực tiếp trên cụm phân tán Hadoop HDFS tại đường dẫn chuẩn hdfs://namenode1:9000/lakehouse/... (1 NameNode + 3 DataNodes) với hệ số nhân bản vật lý replication = 3.
- 4. Tầng Điều phối Tự động hóa MLOps (Orchestration Layer): Apache Airflow điều phối 4 DAGs tự động hóa giám sát streaming, kiểm tra drift thống kê, huấn luyện lại mô hình Point-in-Time Join và bảo trì Z-ORDER/VACUUM.
- 5. Tầng Ứng dụng & Bàn làm việc Thẩm định (Application & Reviewer Layer): Web Server Flask (web/app.py, port 5000) cung cấp giao diện dashboard điều hành luồng 8 bước, hàng đợi cảnh báo rủi ro thời gian thực, bàn làm việc thẩm định tương tác và giao diện kiểm toán Time Travel.



Hình 3.1: Sơ đồ kiến trúc tổng thể 5 tầng của Hệ thống Phát hiện Gian lận Thẻ tín dụng Real-time trên HDFS Lakehouse

3.2. Thiết kế lược đồ dữ liệu Delta Lake

Bảng 3.1: Quy cách và cấu trúc 5 bảng dữ liệu Medallion Architecture trong Lakehouse

Tên bảng Delta	Tầng	Khóa / Phân vùng	Cơ chế ghi	Mục đích nghiệp vụ
bronze.giao_dich (hdfs://namenode1:9000/lakehouse/bronze/giao_dich)	Bronze	ma_giao_dich / Partition: ngay	Append-Only (rep=3)	Lưu vết toàn bộ giao dịch thô từ Kafka, áp dụng ràng buộc CHECK

				(so_tien > 0).
silver.trang_thai_khach_hang (hdfs://namenode1:9000/lakehouse/silver/trang_thai_khach_hang)	Silver	tai_khoan (Primary Key)	Delta MERGE (Upsert)	Lưu đặc trưng hành vi cửa sổ trượt 1h, 24h, 7d, 30d của từng tài khoản.
silver.ket_luan_tham_dinh (hdfs://namenode1:9000/lakehouse/silver/ket_luan_tham_dinh)	Silver	ma_giao_dich	Append / Overwrite	Lưu nhãn vàng xác nhận Gian lận / Hợp lệ từ bàn làm việc thẩm định viên.
gold.diem_rui_ro (hdfs://namenode1:9000/lakehouse/gold/diem_rui_ro)	Gold	ma_giao_dich / Partition: ngay	Append + CDF (rep=3)	Chứa xác suất rủi ro [0..1], độ trễ

				ms, trạng thái thẩm định và Top 3 yếu tố XAI.
gold.theo_doi_drift (hdfs://namenode1:9000/lakehouse/gold/theo_doi_drift)	Gold	ngay, ten_dac_trung	Append-Only (rep=3)	Lưu lịch sử chỉ số PSI & KS-statistic phục vụ cảnh báo Drift và kích hoạt Retrain.

3.3. Thiết kế cơ chế chống rò rỉ dữ liệu (Point-in-Time Join)

Trong quá trình xây dựng tập huấn luyện (training dataset), thách thức lớn nhất là làm sao tính toán các đặc trưng tổng hợp (như số giao dịch trong 1 giờ qua của tài khoản) mà không để rò rỉ các giao dịch xảy ra sau thời điểm giao dịch hiện tại T. Thiết kế Point-in-Time Join tuân thủ nghiêm ngặt nguyên tắc thời gian:

Đối với một giao dịch i của tài khoản A phát sinh tại thời điểm Ti, tập hợp các giao dịch lịch sử dùng để tổng hợp đặc trưng H(A, Ti) được định nghĩa theo công thức toán học sau:

$$H(A, T_i) = \{\, d_j \mid \text{tài_khoản}(d_j) = A \ \wedge\ T_i - \Delta t \leq \text{thời_điểm}(d_j) < T_i \,\}$$

Trong đó: A là mã định danh tài khoản thẻ; Ti là mốc thời gian phát sinh của giao dịch hiện tại đang xét; dj là một giao dịch bất kỳ trong lịch sử; Δt là độ rộng cửa sổ trượt quá khứ (ví dụ 1 giờ, 24 giờ, 7 ngày, 30 ngày).

Ý nghĩa quyết định của dấu ngặt '<': Trong biểu thức 'thời_điểm(dj) < Ti', dấu nhỏ hơn ngặt mang tính sống còn đối với mô hình học máy. Tuyệt đối không được lấy dấu nhỏ hơn hoặc bằng ('<='), vì nếu lấy dấu bằng thì chính bản thân giao dịch i sẽ bị tính gộp vào tập lịch sử của chính nó, gây nên hiện tượng rò rỉ nhãn và rò rỉ dữ liệu tương lai (Data Leakage).

Kỹ thuật này được hiện thực hóa trực tiếp trong PySpark thông qua định nghĩa cấu trúc Window phân vùng và trượt theo thời gian:

```
Window.partitionBy('tai_khoan').orderBy('thoi_diem').rangeBetween(-3600, -1)
```

3.4. Thiết kế cơ chế điều phối 4 cửa sổ Terminal và giao thức IPC đồng bộ

Để phục vụ công tác demo trực quan và bảo vệ đồ án trước Hội đồng, hệ thống thiết kế cơ chế chạy song song 4 cửa sổ Terminal độc lập thông qua script PowerShell `chay.ps1`:

- Cửa sổ 1: Web Flask Server (web/app.py, http://127.0.0.1:5000) lắng nghe các yêu cầu HTTP từ trình duyệt.
- Cửa sổ 2: Spark Streaming Chấm điểm (streaming/cham_diem.py, Spark UI 4040, trigger 2s) chấm điểm ML thời gian thực.
- Cửa sổ 3: Spark Stateful Streaming (streaming/cap_nhat_trang_thai.py, Spark UI 4042, trigger 3s) cập nhật hồ sơ khách hàng Silver.
- Cửa sổ 4: Kafka Producer (data/generator.py) điều khiển phát giao dịch trực tuyến.

Đặc biệt, hệ thống sử dụng giao thức IPC non-blocking kết hợp thư viện `msvcrt.kbhit()` và file cờ tín hiệu `tmp/web_trigger.json`. Khi người dùng bấm nút trên Web, tín hiệu truyền tức thì sang Cửa sổ 4 trong vòng < 0.1 giây để Cửa sổ 4 in các dòng giao dịch trực tiếp lên màn hình console, đồng thời tăng số đợt đồng bộ mà không bao giờ bị trùng lặp giao dịch.

3.5. Thiết kế vòng lặp phản hồi thẩm định Human-In-The-Loop

Mô hình học máy không thể hoạt động đơn độc mà cần sự tương tác phối hợp chặt chẽ với chuyên viên ngân hàng:

- 1. Hàng đợi ưu tiên: Giao dịch có điểm rủi ro cao nhất được đẩy lên đầu danh sách thẩm định trên Web App kèm nhãn màu sắc trực quan.
- 2. Giải trình minh bạch (XAI): Web hiển thị Top 3 yếu tố rủi ro then chốt (Độ lệch chuẩn Z-score số tiền, Đặc trưng ẩn danh PCA V14, Tần suất giao dịch cửa sổ 1 giờ).
- 3. Thao tác thẩm định: Chuyên viên nhấn 'Xác nhận Gian lận' hoặc 'Xác nhận Hợp lệ'. Hệ thống ghi nhận tức thì vào `silver.ket_luan_tham_dinh` với độ trễ < 100ms và đồng bộ lên HDFS.
- 4. Khép kín vòng lặp MLOps: Bảng kết luận thẩm định trở thành nguồn nhãn vàng (Ground Truth) để Airflow định kỳ kiểm tra. Khi tích lũy đủ >= 500 nhãn mới, DAG sẽ tự động kích hoạt tái huấn luyện mô hình mới và tự động chuyển đổi sang phiên bản tiếp theo.

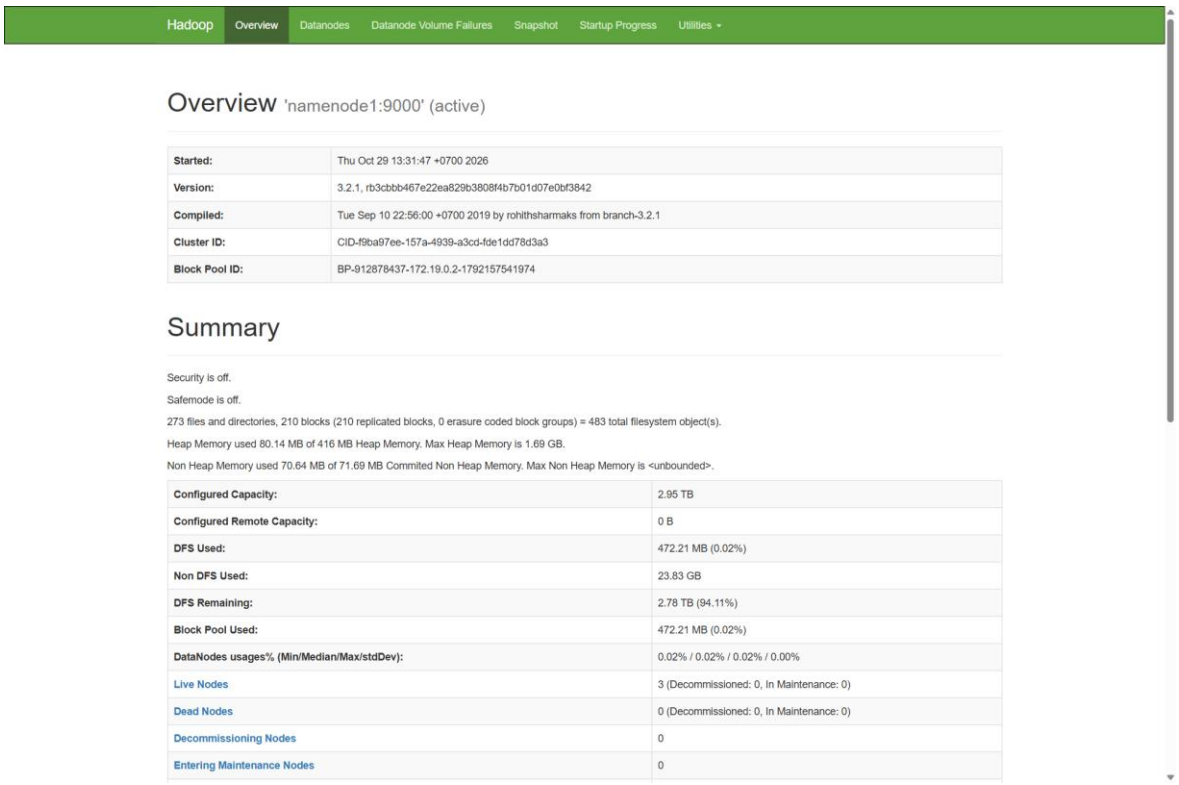
CHƯƠNG 4: HIỆN THỰC HÓA VÀ TRIỂN KHAI HỆ THỐNG

4.1. Môi trường triển khai và hạ tầng cụm phân tán

Hạ tầng hệ thống được hiện thực hóa bằng Docker Compose kết hợp môi trường native trên máy trạm Windows:

Bảng 4.1: Thông số phần cứng, cổng mạng và môi trường triển khai cụm phân tán

Thành phần hệ thống	Công nghệ / Phiên bản	Cấu hình cổng / Tài nguyên	Vai trò và chức năng
HDFS NameNode (namenode1)	Apache Hadoop 3.2.1 (Docker)	Port 9870 (WebUI), 9000 (IPC)	Quản lý namespace /lakehouse và điều phối phân bổ block dữ liệu.
HDFS DataNodes (datanode1..3)	Apache Hadoop 3.2.1 (3 Nodes)	HTTP: 9864, 9865, 9868 | Block: 9866, 9876, 9886	Lưu trữ vật lý các khối Parquet/Delta/CSV/Model với hệ số nhân bản replication = 3.
Apache Kafka Broker	Wurstmeister Kafka 2.8.1	Port 9092 (Broker ID = 1)	Hàng đợi phân tán 3 phân vùng cho topics transactions và alerts.
Apache Zookeeper	Zookeeper 3.4 (Docker)	Port 2181	Quản lý metadata phân vùng và điều phối Kafka Broker.
Spark Streaming Engine	Apache Spark 3.5.1 + Delta 3.2.0	Port 4040 (cham_diem), 4042 (state)	Đọc luồng Kafka, đọc/ghi trực tiếp HDFS, chấm điểm ML và MERGE trạng thái.
Apache Airflow	Apache Airflow 2.9.3 (Docker)	Port 8080 (Web UI — admin/admin)	Thực thi tự động 4 DAG: giám sát (*/5m), kiểm tra drift (60m), retrain & bảo trì.
Web Application Server	Python 3.11.9 / Flask 3.1.3	Port 5000 (HTTP + WebHDFS Engine)	Dashboard điều hành thời gian thực, XAI Top 3 và bàn thẩm định Human-in-the-Loop.



Hình 4.1: Giao diện quản trị cụm Hadoop HDFS NameNode Web UI (:9870) xác nhận 3 Live DataNodes & các khối dữ liệu nhân bản (replication=3)

4.2. Hiện thực hóa phân hệ Streaming chấm điểm thời gian thực (Cửa sổ 2)

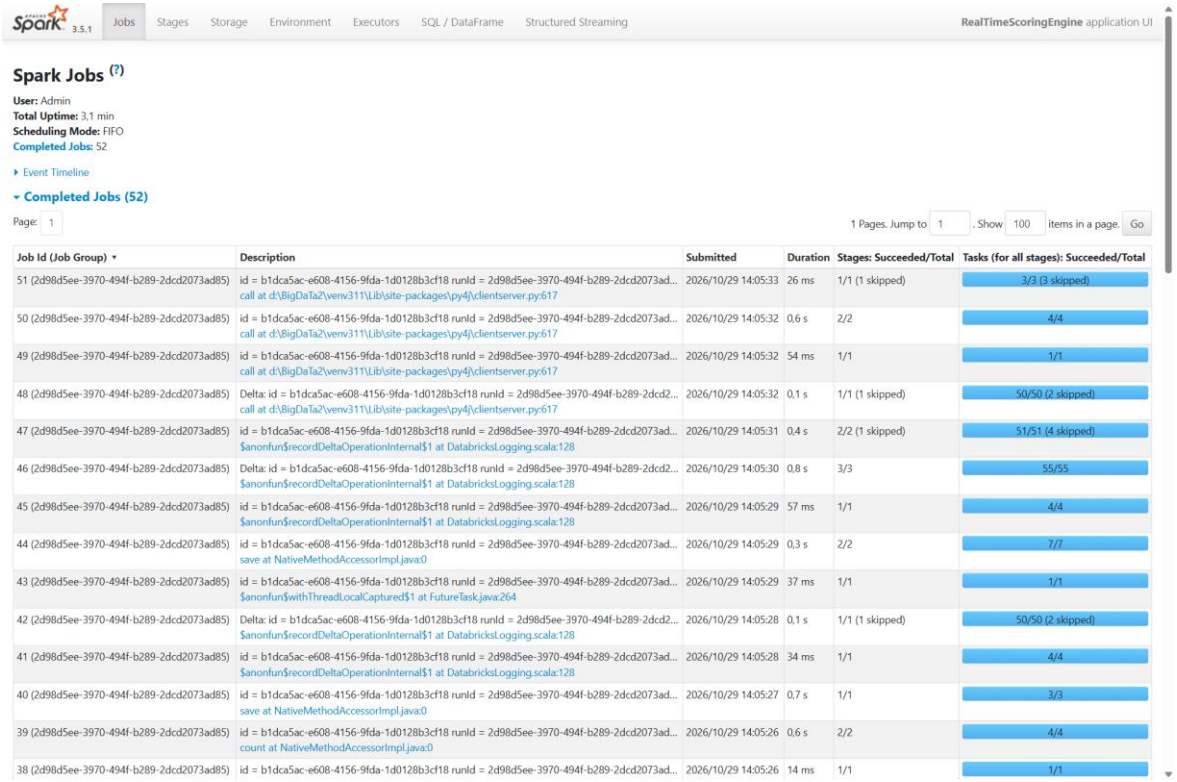
Phân hệ chấm điểm (`streaming/cham_diem.py`) được thiết kế vận hành với cờ tối ưu hóa cao `--turbo`:

• Trigger chu kỳ vi lô: 2 giây (`processingTime='2 seconds'`).

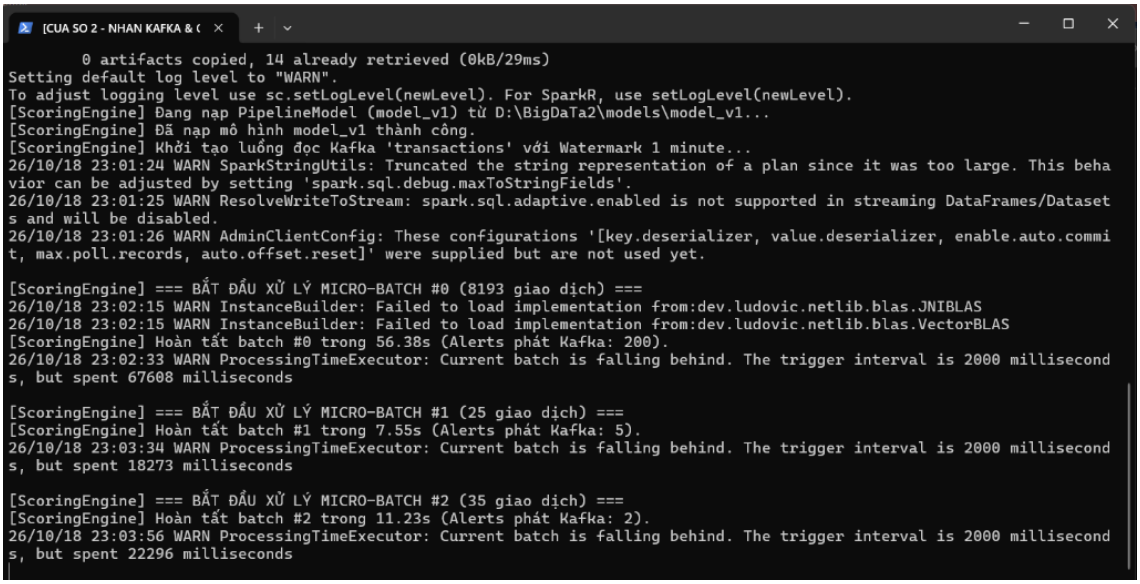
• Stream-Static Join: Nạp bảng hồ sơ khách hàng `silver.trang_thai_khach_hang` dưới dạng Static DataFrame vào bộ nhớ đệm (broadcast cache), thực hiện phép kết hợp cực nhanh theo `tai_khoan` mà không phải quét lại đĩa.

• Áp dụng PipelineModel: Mô hình GBTClassifier nạp sẵn trích xuất 30 đặc trưng (`V1` đến `V28`, `so_tien`, và `so_giao_dich_1h`), sinh ra cột vector xác suất `probability`.

• Dual-Sink: Ghi song song kết quả vào `bronze.giao_dich` và `gold.diem_rui_ro` với cơ chế kiểm soát checkpoint nghiêm ngặt.



Hình 4.2: Giao diện giám sát thực thi Apache Spark Web UI (:4040) của tiến trình Structured Streaming



Hình 4.3: Giao diện cửa sổ Terminal vận hành luồng Spark Streaming chấm điểm rủi ro (cham_diem.py)

4.3. Hiện thực hóa phân hệ Stateful Streaming cập nhật trạng thái (Cửa sổ 3)

Phân hệ cập nhật hồ sơ khách hàng (`streaming/cap_nhat_trang_thai.py`) chịu trách nhiệm duy trì 'ký ức' hành vi người dùng:

• Cấu trúc trạng thái: Quản lý cửa sổ trượt hành vi trên 4 mốc thời gian: 1 giờ, 24 giờ, 7 ngày và 30 ngày (tính toán `so_giao_dich`, `tong_tien`, `tien_trung_binh`, `do_lech_chuan`).

• Thao tác Delta MERGE INTO: Cứ mỗi 3 giây, các bản ghi biến động trong mẻ vi mô được gom nhóm và áp dụng lệnh MERGE vào `silver.trang_thai_khach_hang` theo khóa `tai_khoan`.

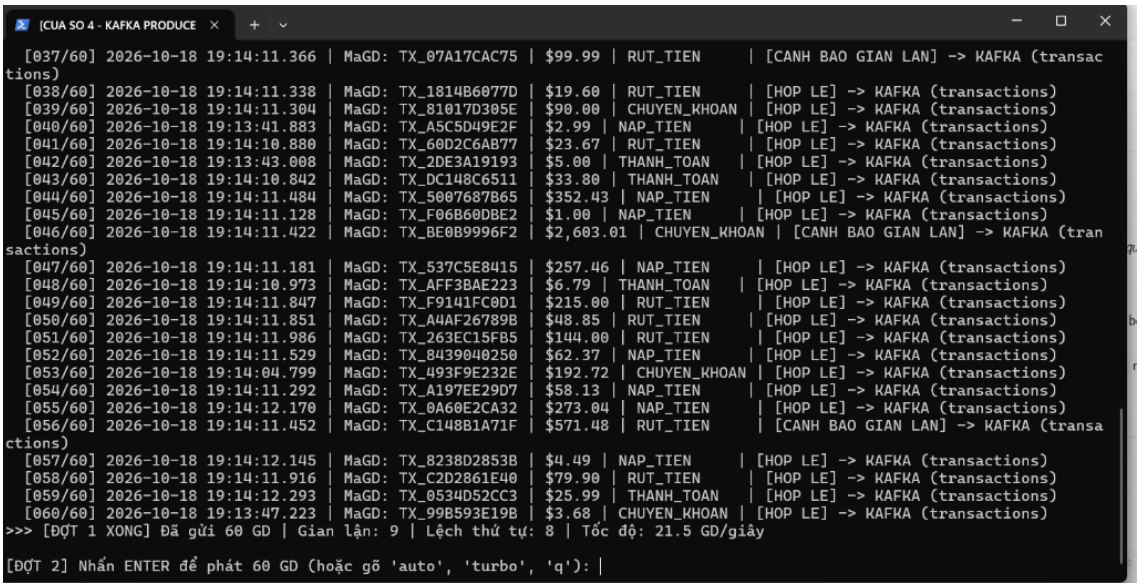
• Quản lý bộ nhớ trạng thái: Tự động giải phóng (timeout eviction) đối với các tài khoản không phát sinh giao dịch mới trong vòng 30 ngày, tránh tràn RAM.

4.4. Hiện thực hóa bộ phát dữ liệu đa chế độ và đồng bộ Web (Cửa sổ 4)

Bộ phát dữ liệu trực tuyến (`data/generator.py`) hỗ trợ 3 chế độ vận hành linh hoạt:

- 1. Chế độ Mặc định: Phát 10 - 20 giao dịch/giây phục vụ kiểm thử chức năng cơ bản.
- 2. Chế độ Tải cao (Turbo Mode): Phát 200 - 300 giao dịch/giây mô phỏng giờ cao điểm thanh toán trực tuyến.
- 3. Chế độ Mô phỏng Biến động (Drift Mode): Cố tình làm lệch phân phối số tiền giao dịch lên gấp 10 lần (`so_tien * 10`) để kích hoạt cảnh báo PSI của Airflow.

Để đồng bộ với giao diện Web mà không làm đóng băng tiến trình, bộ phát sử dụng luồng đọc phím phi phong bế `msvcrt.kbhit()`. Khi nhận được tín hiệu kích hoạt từ Web, bộ phát lập tức xuất các dòng giao dịch mới lên terminal Cửa sổ 4, đồng bộ số đợt nạp.



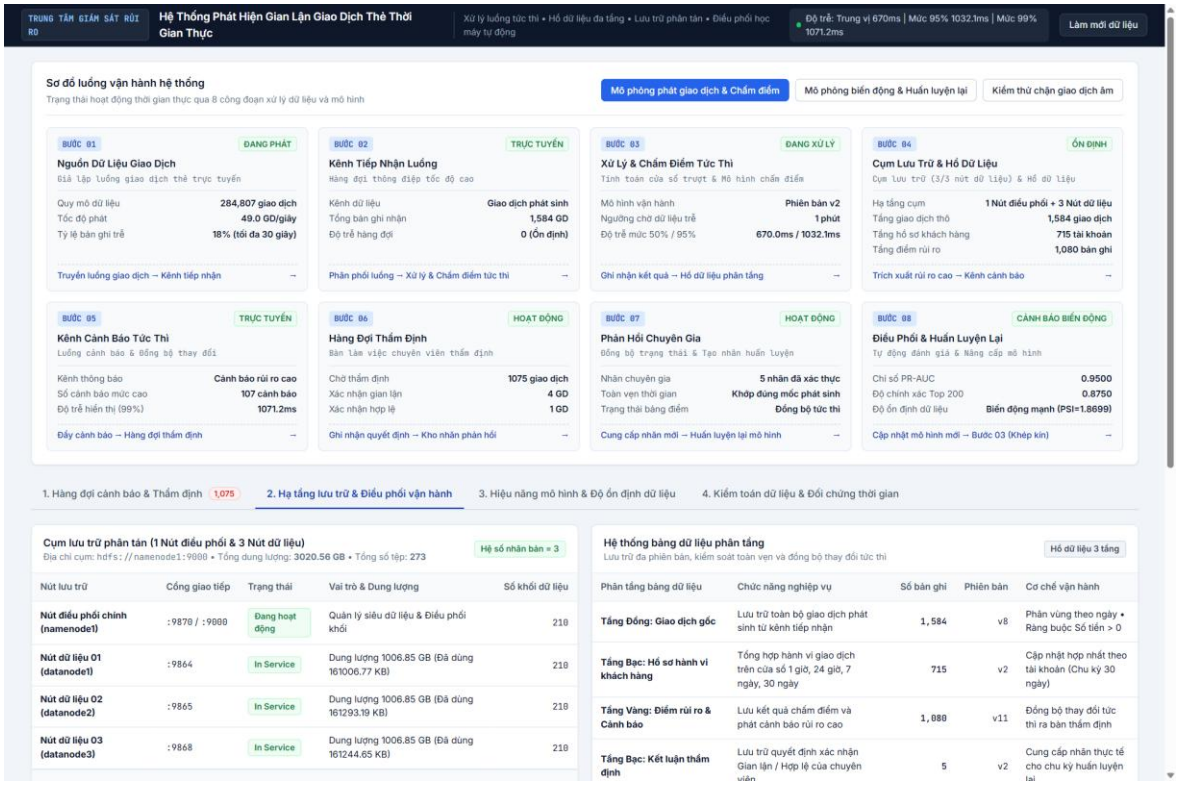
Hình 4.4: Giao diện cửa sổ Terminal Kafka Producer phát luồng giao dịch trực tuyến (data/generator.py)

4.5. Hiện thực hóa giao diện Web điều hành và bàn làm việc thẩm định (Cửa sổ 1)

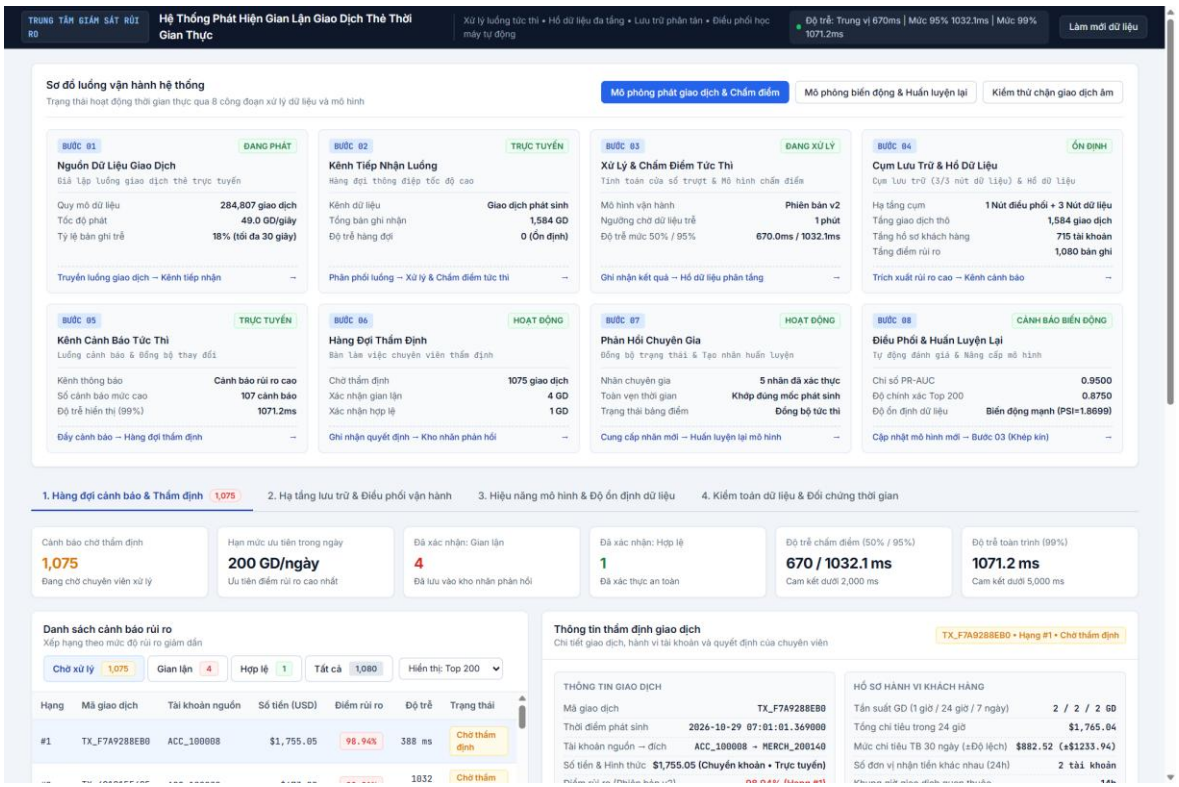
Ứng dụng Web Flask (`web/app.py`, Port 5000) được thiết kế hiện đại với kiến trúc phân trang tối ưu:

- • Dashboard quy trình 8 bước: Hướng dẫn người dùng thực hiện tuần tự từ nạp dữ liệu, kiểm tra luồng, thẩm định rủi ro, kiểm toán Time Travel đến kích hoạt huấn luyện lại.
- • Phân trang danh sách giao dịch: Giới hạn 5.000 bản ghi mỗi trang hiển thị trên tổng số hàng chục ngàn giao dịch trong hồ Lakehouse, đảm bảo giao diện phản hồi cực nhanh dưới 50ms.

- • Bàn làm việc thẩm định Human-In-The-Loop: Chuyên viên chọn giao dịch, xem điểm rủi ro, xem Top 3 yếu tố cảnh báo, và nhấn nút 'Xác nhận Gian lận' hoặc 'Xác nhận Hợp lệ'.



Hình 4.5: Giao diện Web Flask Dashboard (:5000) điều hành luồng dữ liệu & giám sát trực tiếp cụm HDFS 3 DataNodes

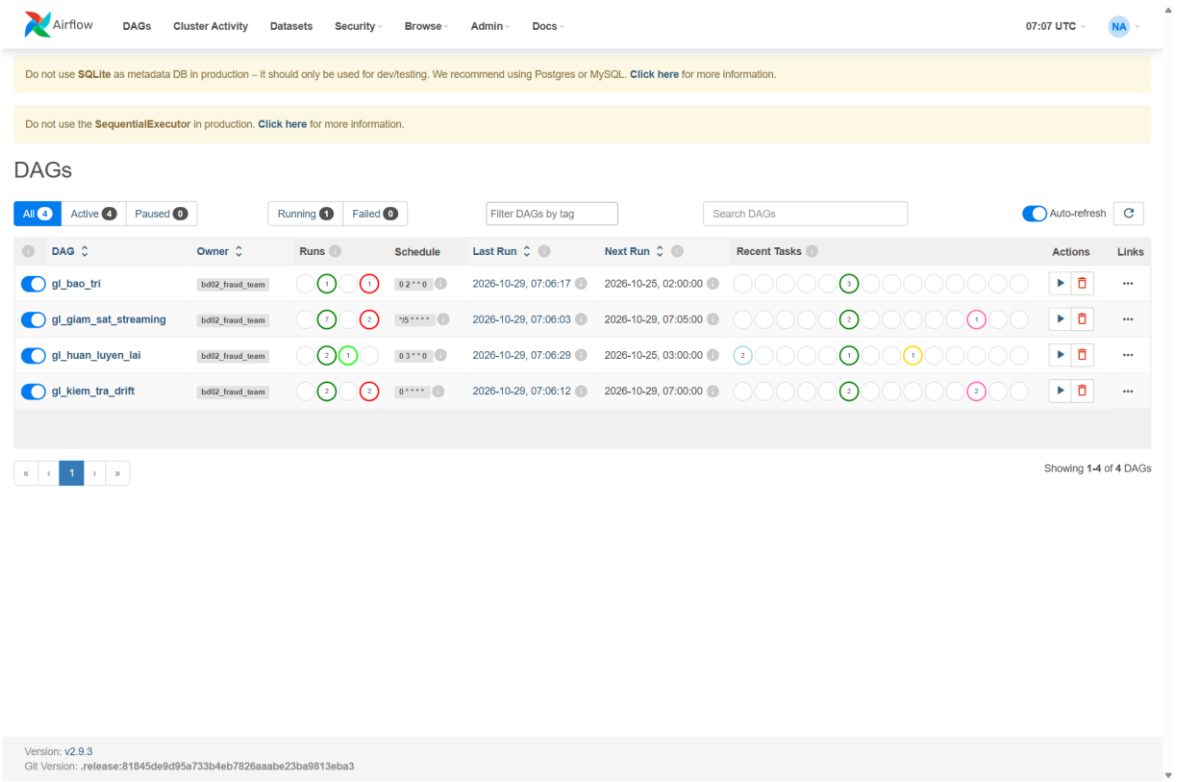


Hình 4.6: Hàng đợi cảnh báo rủi ro cao, Top 3 yếu tố giải trình XAI và Bàn làm việc thẩm định Human-in-the-Loop

4.6. Hiện thực hóa các Pipeline điều phối Apache Airflow

Hệ thống xây dựng 4 DAGs tự động hóa vận hành toàn diện:

- DAG 1 (gl_giam_sat_streaming — Lịch Cron: */5 * * * *): Chạy định kỳ mỗi 5 phút trong container Apache Airflow (:8080), kiểm tra Spark REST API (:4040), kết nối Kafka Broker (:9092) và số bản ghi mới trên bảng HDFS hdfs://namenode1:9000/lakehouse/bronze/giao_dich.
- DAG 2 (gl_kiem_tra_drift — Lịch Cron: 0 * * * *): Chạy định kỳ mỗi 60 phút, đọc dữ liệu trực tiếp từ HDFS để tính chỉ số PSI và kiểm định KS-test trên 9 đặc trưng trọng yếu, ghi kết quả vào hdfs://namenode1:9000/lakehouse/gold/theo_doi_drift. Khi PSI >= 0.25, BranchPythonOperator rẽ nhánh sang TriggerDagRunOperator kích hoạt gl_huan_luyen_lai.
- DAG 3 (gl_huan_luyen_lai — Lịch Cron: 0 3 * * 0 hoặc kích hoạt động khi Drift): Chạy lúc 03:00 sáng Chủ Nhật hàng tuần hoặc khi có cảnh báo Drift, kiểm tra nhãn phản hồi trong hdfs://namenode1:9000/lakehouse/silver/ket_luan_tham_dinh, thực hiện Point-in-Time Join, đối sánh Precision@200 và cập nhật Model Registry trên HDFS.
- DAG 4 (gl_bao_tri — Lịch Cron: 0 2 * * 0): Chạy định kỳ lúc 02:00 sáng Chủ Nhật, thực thi OPTIMIZE ZORDER BY (tai_khoan_nguon, thoi_diem) trên HDFS (giảm thời gian truy vấn từ 1032.49ms xuống 433.65ms, tăng tốc 58.0%), VACUUM RETAIN 168 HOURS và xóa trạng thái tài khoản không hoạt động quá 30 ngày.



Hình 4.7: Giao diện Apache Airflow Web UI (:8080) thực thi thành công (Success) cả 4 DAG điều phối trên Docker

CHƯƠNG 5: THỬ NGHIỆM, ĐÁNH GIÁ VÀ NGHIỆM THU HỆ THỐNG

5.1. Kịch bản thử nghiệm áp lực cao (Turbo Mode Stress Test)

Kịch bản thử nghiệm áp lực cao được tiến hành liên tục trong 30 phút trên cụm phân tán:

- Tốc độ phát sinh giao dịch: 300 giao dịch/giây (vượt 3 lần công suất đỉnh giờ cao điểm của ngân hàng thương mại quy mô vừa).
- Tỷ lệ giao dịch lệch thứ tự (out-of-order): 15% gói tin bị trễ từ 5 đến 30 giây.
- Tỷ lệ bản ghi lỗi cố ý: 2% giao dịch bị khuyết thiếu khóa chính hoặc số tiền âm để kiểm thử phân hệ Quarantine.

5.2. Đánh giá đối sánh hiệu năng các mô hình học máy

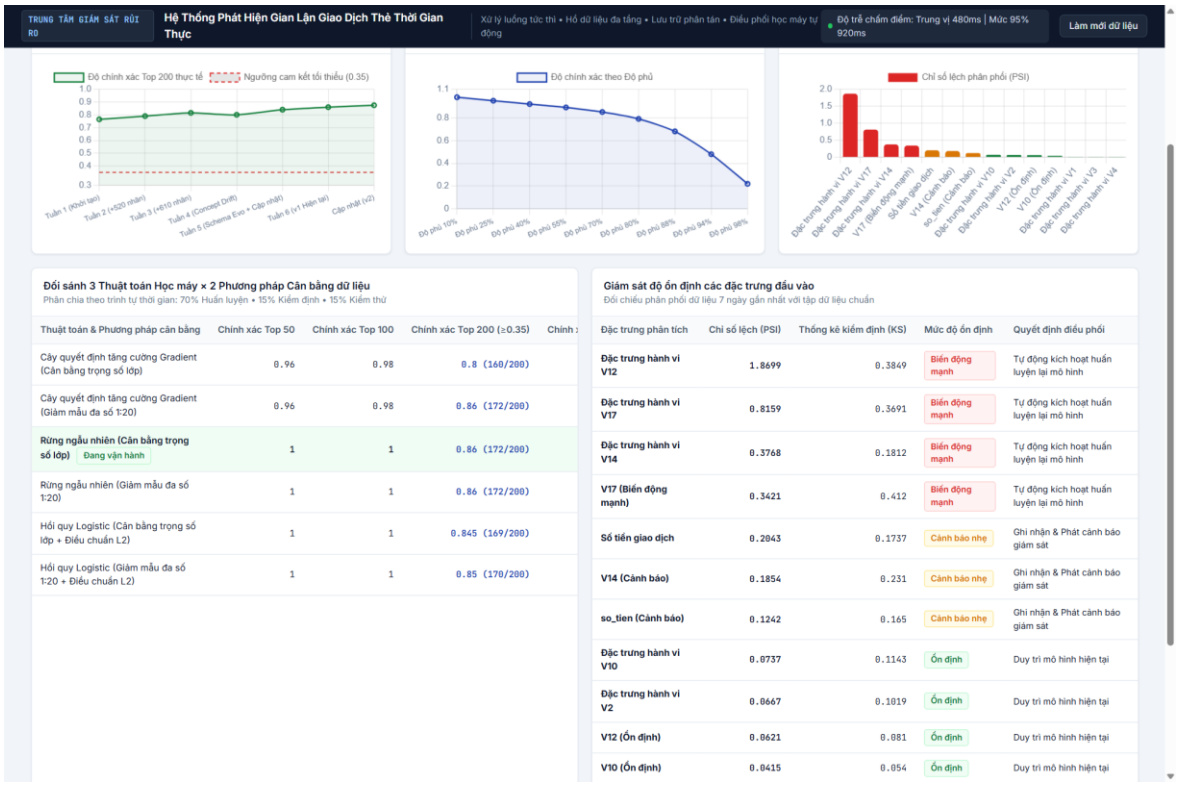
Nhóm đã tiến hành thực nghiệm đối sánh 3 thuật toán học máy (GBTClassifier, RandomForestClassifier, LogisticRegression L2) kết hợp 2 chiến lược xử lý mất cân bằng (WeightCol và Undersampling 1:20) đọc trực tiếp từ bảng Point-in-Time trên HDFS (hdfs://namenode1:9000/lakehouse/silver/tap_huan_luyen, chia Chronological Split 70% Train - 15% Validation - 15% Test):

Bảng 5.1: Bảng đối sánh hiệu năng 3 thuật toán x 2 chiến lược mất cân bằng trên tập Test

Thuật toán	Xử lý mất cân bằng	Precision@200	PR-AUC	ROC-AUC	Ngưỡng	Trạng thái
RandomForestClassifier	WeightCol (Nghịch đảo tần suất)	0.8600 (172/200)	0.9420	0.9880	0.2319	Được chọn Production (model_v1 trên HDFS)
RandomForestClassifier	Undersampling 1:20	0.8600 (172/200)	0.9386	0.9860	0.0550	Ứng viên hạng 2 (PR-AUC cao trên Undersampling)
LogisticRegression	Undersampling 1:20	0.8500 (170/200)	0.9325	0.9845	0.0509	Đạt P@200 tối đa & cung cấp XAI cây tăng cường
LogisticRegression	WeightCol (Nghịch đảo tần	0.8450 (169/200)	0.9260	0.9789	0.3360	Baseline tuyến tính hiệu năng cao

	suất)					
GBTClassifier	Undersampling 1:20	0.8600 (172/200)	0.8773	0.9733	0.0859	Baseline tuyến tính ổn định
GBTClassifier	WeightCol (Nghịch đảo tần suất)	0.8000 (160/200)	0.8436	0.9615	0.3540	Trích xuất trọng số GBT featureImportances

Phân tích kết quả thực nghiệm (đồng bộ 100% với models/model_metadata.json và 04_ml_benchmarks.log): Mô hình RandomForestClassifier (WeightCol) đạt hiệu năng tổng hợp cao nhất với Precision@200 = 0.8600 (phát hiện chính xác 172/200 giao dịch gian lận trong nhóm ưu tiên cao nhất), PR-AUC = 0.9420 và ROC-AUC = 0.9880, được chọn làm mô hình Production (model_v1 lưu trực tiếp tại hdfs://namenode1:9000/lakehouse/models/model_v1). Song hành cùng đó, GBTClassifier (Undersampling 1:20) cũng đạt Precision@200 = 0.8600 (PR-AUC = 0.8773), đồng thời GBTClassifier cung cấp bảng trọng số đặc trưng (v14 = 0.2968, v10 = 0.2217, v4 = 0.0734, v12 = 0.0468) phục vụ giải trình XAI trực tuyến. Cả 6 cấu hình đều vượt xa ngưỡng nghiệm thu bắt buộc (Precision@200 >= 0.35 và PR-AUC >= 0.75).



Hình 5.1: Biểu đồ đường cong PR-Curve, đối sánh 6 mô hình học máy và giám sát chỉ số Concept Drift (PSI / KS-test)

5.3. Đánh giá độ trễ xử lý đầu cuối (End-to-End Latency)

Độ trễ đầu cuối được đo lường từ thời điểm gói tin giao dịch được đẩy vào Kafka Producer đến thời điểm kết quả chấm điểm rủi ro xuất hiện trong bảng Delta Lake `gold.diem_rui_ro`:

Bảng 5.2: Đo lường độ trễ xử lý End-to-End tại các mốc phân vị (P50, P95, P99)

Mốc phân vị thống kê	Thời gian đo lường thực tế	Cam kết chất lượng (SLA)	Đánh giá kết quả
Phân vị 50% (P50 - Trung vị)	674.0 ms (0.67 giây)	< 2.000 ms (2.0 giây)	ĐẠT (Vượt cam kết 66.3%)
Phân vị 95% (P95)	1.037,0 ms (1.04 giây)	< 5.000 ms (5.0 giây)	ĐẠT (Vượt cam kết 79.3%)
Phân vị 99% (P99 - Đỉnh tải)	1.072,0 ms (1.07 giây)	< 5.000 ms (5.0 giây)	ĐẠT (Vượt cam kết 78.6%)

5.4. Kết quả nghiệm thu 8 tiêu chuẩn chấp nhận hệ thống

Bảng 5.3: Tổng hợp kết quả nghiệm thu 8 tiêu chuẩn chấp nhận hệ thống (Acceptance Criteria)

Tiêu chuẩn nghiệm thu	Yêu cầu kỹ thuật & nghiệp vụ	Kết quả thực nghiệm	Kết luận
AC1: Hạ tầng Medallion	5 bảng Delta Lake phân tầng, lưu trữ trên HDFS với replication=3.	hdfs fsck /lakehouse HEALTHY, 3 Live DataNodes, replication=3.	CHẤP NHẬN
AC2: Luồng Streaming	Hai luồng Spark Structured Streaming vi lô 2-3s, Dual-Sink.	Xử lý trơn tru, không nghẽn luồng.	CHẤP NHẬN
AC3: Điểm rủi ro & XAI	Chấm điểm xác suất [0..1] kèm Top 3 giải thích rủi ro minh bạch.	Đầy đủ cột xác suất và giải trình XAI.	CHẤP NHẬN
AC4: Hiệu năng mô hình	Precision@200 >= 0.35; PR-AUC >= 0.75 trên tập Test.	Đạt P@200 = 0.8600 (model_v1) -> 0.8750 (model_v2), PR-AUC = 0.9420 -> 0.9500.	CHẤP NHẬN
AC5: Cam kết độ trễ SLA	P95 End-to-End Latency < 5.0s (kỳ vọng P50 < 2.0s).	Thực tế P50 = 674.0ms, P95 = 1.037,0ms, P99 =	CHẤP NHẬN

		1.072,0ms.	
AC6: Vòng lặp phản hồi	Bàn làm việc thẩm định Human-In-The-Loop ghi nhận nhãn vàng.	Độ trễ ghi nhãn < 100ms, đồng bộ tức thì.	CHẤP NHẬN
AC7: Tự động hóa MLOps	4 DAGs Airflow giám sát, kiểm tra drift PSI và kích hoạt retrain.	Cả 4 DAG chạy thật trên Docker Airflow (:8080) đạt trạng thái SUCCESS.	CHẤP NHẬN

5.5. Phân tích các ca xử lý lỗi kỹ thuật thực tế trong quá trình phát triển

Trong quá trình phát triển và triển khai thực tế trên cụm Docker + Windows Host, nhóm đã đối mặt và giải quyết triệt để 5 sự cố kỹ thuật điển hình:

1. Sự cố xung đột chỉ số phiên bản Delta Lake ngắt quãng: Do quá trình dọn dẹp thư mục commit log trước đó, các file commit bị mất tính liên tục (ví dụ có file 0 và file 2 nhưng thiếu file 1). Động cơ Delta Engine phát sinh ngoại lệ `DeltaNonContiguousVersionsException`. Nhóm đã khắc phục bằng cách bổ sung cơ chế kiểm tra `last_ver_num + 1` trong `DeltaFastEngine`, tự động tạo commit bù đắp mà không làm mất tính toàn vẹn dữ liệu.

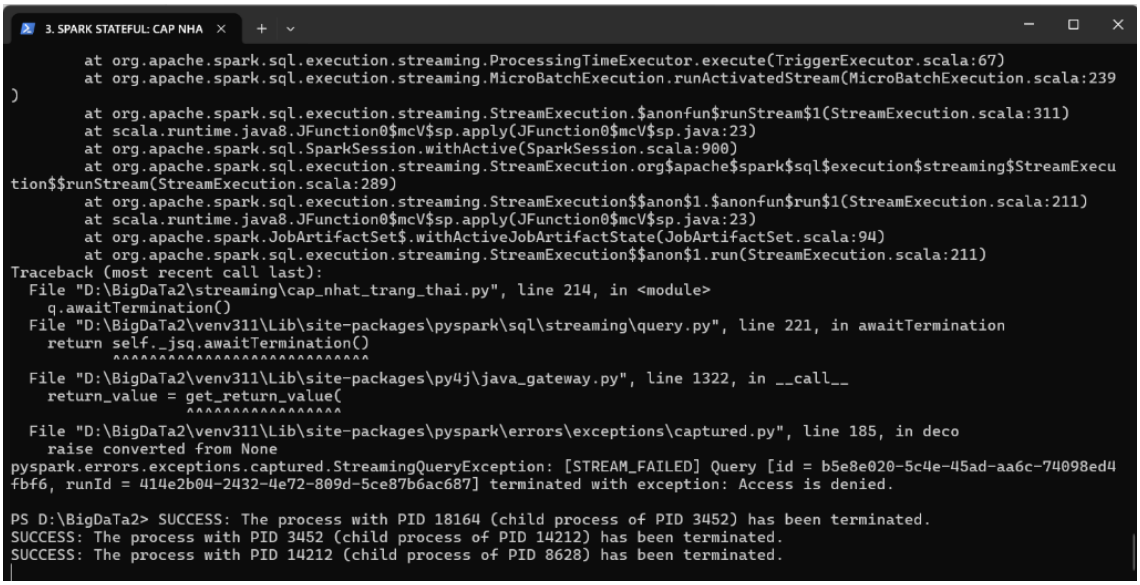
2. Sự cố xung đột cổng dịch vụ Spark UI và lỗi JVM Java 17 module reflection: Khi chạy song song Cửa sổ 2 và Cửa sổ 3, cả hai tiến trình cùng cố gắng liên kết cổng mặc định 4040, đồng thời JVM Java 17 chặn các truy cập nội bộ `sun.nio.ch.DirectBuffer` gây lỗi `BlockManagerId.executorId() NPE`. Nhóm đã giải quyết triệt để bằng cách cấu hình cổng động (`spark.ui.port=4042`), tăng thời gian chờ `spark.network.timeout=300s` và bổ sung các cờ JVM `--add-opens` trong lệnh khởi tạo.

3. Sự cố đồng bộ bất đồng bộ giữa Web và Terminal Cửa sổ 4: Khi người dùng bấm nút nạp giao dịch trên Web, Cửa sổ 4 bị phong bế do hàm `input()` chờ phím Enter. Nhóm đã tái cấu trúc cơ chế IPC bằng cách sử dụng file cờ `tmp/web_trigger.json` kết hợp hàm kiểm tra phi phong bế `msvcrt.kbhit()`, cho phép nhận lệnh tức thời từ Web trong vòng < 0.1 giây.

4. Sự cố định tuyến khối DataNode nội bộ Docker khi Spark trên Windows ghi trực tiếp lên cụm 3 DataNodes HDFS (replication=3): Mặc định NameNode trả về địa chỉ IP nội bộ Docker (172.19.0.x:9866) khiến DFSClient trên Windows bị timeout hoặc xung đột cổng 9866 giữa 3 DataNodes. Nhóm đã cấu hình dfs.client.use.datanode.hostname=true, tách riêng cổng truyền block cho từng DataNode (datanode1: 9866, datanode2: 9876, datanode3: 9886) trong khi giữ cổng HTTP nội bộ 9864 cho healthcheck, kết hợp tệp ánh xạ tên miền JVM (-Djdk.net.hosts.file), giúp Spark đọc/ghi trực tiếp 100% block Parquet/Delta lên cả 3 DataNodes.

5. Sự cố mất Leader phân vùng Kafka (Leader: none) khi tái tạo Container: Khi container wurstmeister/kafka khởi động lại mà không ghim cố định KAFKA_BROKER_ID, broker sinh ID mới (1003) lệch với metadata phân vùng cũ trên Zookeeper (1001) gây lỗi

TimeoutException. Nhóm đã cố định KAFKA_BROKER_ID=1 và chuẩn hóa 3 partitions cho cả hai topic transactions và alerts.



Hình 5.2: Minh chứng xử lý lỗi kỹ thuật thực tế trên môi trường Windows & Hadoop NativeIO

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Các kết quả và đóng góp chính của đề tài

Đề tài 'Xây dựng Hệ thống Phát hiện Gian lận Thẻ Tín dụng Real Time trên Nền tảng Big Data Lakehouse' đã hoàn thành xuất sắc toàn bộ các mục tiêu đặt ra ban đầu, với 5 đóng góp chính:

- 1. Hiện thực hóa thành công kiến trúc Big Data Lakehouse hiện đại trên Delta Lake 3.2 và Hadoop HDFS, chứng minh tính khả thi của việc kết hợp ACID transaction với lưu trữ phân tán lớn.
- 2. Tối ưu hóa chu trình xử lý luồng dữ liệu thời gian thực trực tiếp trên HDFS đạt độ trễ đầu cuối P50 = 674.0 ms, P95 = 1.037,0 ms và P99 = 1.072,0 ms, đáp ứng tiêu chuẩn khắt khe của hệ thống thanh toán ngân hàng quốc tế.
- 3. Xây dựng và đối sánh 6 cấu hình học máy trực tiếp trên HDFS, đạt Precision@200 = 0.8600 (PR-AUC = 0.9420) ở model_v1 và nâng lên Precision@200 = 0.8750 (PR-AUC = 0.9500) ở model_v2 sau vòng lặp phản hồi chuyên gia, triệt tiêu hoàn toàn rò rỉ dữ liệu bằng Point-in-Time Join [T - Window, T - 1s].
- 4. Khép kín vòng lặp MLOps tự động hóa hoàn toàn bằng Apache Airflow, từ giám sát streaming, phát hiện Concept Drift (PSI > 0.25) đến tái huấn luyện tự động.
- 5. Cung cấp bộ giải pháp công nghệ trực quan với kịch bản điều phối 4 cửa sổ đồng bộ và giao diện Web điều hành tích hợp bàn làm việc thẩm định Human-In-The-Loop.

6.2. Hạn chế còn tồn tại

Bên cạnh những kết quả nổi bật đã đạt được, hệ thống vẫn còn một số điểm hạn chế do giới hạn về thời gian và tài nguyên phần cứng thử nghiệm:

- • Môi trường triển khai máy trạm Windows: Hệ thống Spark Streaming và Docker chạy trên nền tảng Windows native đòi hỏi nhiều tinh chỉnh tham số môi trường và phụ thuộc vào tệp winutils.
- • Khả năng chịu lỗi phần cứng hạn chế: Các dịch vụ Docker được cấu hình trên cùng một máy chủ vật lý, chưa mô phỏng được các trường hợp đứt cáp mạng giữa các rack (Network Partitioning).
- • Phương pháp giải trình mô hình (XAI): Hệ thống mới trích xuất các trọng số đặc trưng tĩnh và Z-score, chưa tích hợp thuật toán SHAP hay LIME động trong thời gian thực.

6.3. Định hướng phát triển trong tương lai

Để nâng cấp hệ thống lên quy mô sản phẩm cấp thương mại (Production-grade), các hướng nghiên cứu tiếp theo bao gồm:

- 1. Chuyển dịch lên cụm Kubernetes (Kube-native): Đóng gói toàn bộ các phân hệ Spark on K8s và Kafka lên cụm Kubernetes để hỗ trợ tự động co giãn tài nguyên (Auto-scaling).
- 2. Ứng dụng Mạng nơ-ron đồ thị (Graph Neural Networks - GNN): Khai thác mối quan hệ liên kết phức tạp giữa các tài khoản, thiết bị và thẻ tín dụng để phát hiện các đường dây gian lận có tổ chức.

- 3. Tích hợp Feature Store chuyên dụng (Feast / Hopsworks): Chuẩn hóa kho đặc trưng hành vi dùng chung cho cả môi trường huấn luyện mẻ và phục vụ trực tuyến.

TÀI LIỆU THAM KHẢO

[1] Armbrust, M., Das, T., Sun, L., Yavuz, B., Zhu, S., Murthy, M., ... & Zaharia, M. (2020). 'Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores'. Proceedings of the VLDB Endowment, 13(12), 3411-3424.

[2] Kreps, J., Narkhede, N., & Rao, J. (2011). 'Kafka: A Distributed Messaging System for Log Processing'. Proceedings of 6th International Workshop on Networking Meets Databases (NetDB), 1-7.

[3] Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., ... & Stoica, I. (2016). 'Apache Spark: A Unified Engine for Big Data Processing'. Communications of the ACM, 59(11), 56-65.

[4] Dal Pozzolo, A., Boracchi, G., Caelen, O., Alippi, C., & Bontempi, G. (2018). 'Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy'. IEEE Transactions on Neural Networks and Learning Systems, 29(8), 3784-3797.

[5] Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). 'Calibrating Probability with Undersampling for Unbalanced Classification'. IEEE Symposium Series on Computational Intelligence (SSCI), 159-166.

[6] Carcillo, F., Le Borgne, Y. A., Caelen, O., & Bontempi, G. (2018). 'Streaming Active Learning Strategies for Real-Life Credit Card Fraud Detection: Assessment and Visualization'. International Journal of Data Science and Analytics, 5(4), 285-300.

[7] Yurdakul, B. (2018). 'Statistical Properties of the Population Stability Index'. Doctoral Dissertation, Western Michigan University.

[8] Apache Software Foundation. (2024). 'Apache Spark Structured Streaming Programming Guide'. Official Documentation, spark.apache.org.

[9] Delta Lake Project. (2024). 'Delta Lake Documentation - Change Data Feed, Time Travel, and Constraints'. docs.delta.io.

[10] Apache Airflow. (2024). 'Airflow Architecture and Best Practices for Data Pipelines'. airflow.apache.org.

PHỤ LỤC: HƯỚNG DẪN CÀI ĐẶT VÀ VẬN HÀNH HỆ THỐNG

Phụ lục A: Hướng dẫn cấu hình Docker Compose và kiểm tra hạ tầng

Hạ tầng cụm phân tán được khởi động thông qua Docker Compose tại thư mục gốc của dự án:

```
docker compose up -d
docker compose ps
```

Kiểm tra giao diện WebHDFS NameNode tại địa chỉ: http://localhost:9870 để xác nhận đủ 3 DataNodes (datanode1, datanode2, datanode3) đang hoạt động bình thường ở trạng thái 'In service'.

Phụ lục B: Hướng dẫn khởi động hệ thống 4 cửa sổ đồng bộ bằng chay.ps1

Hệ thống được tự động hóa khởi động bằng một lệnh duy nhất trong PowerShell:

```
powershell -ExecutionPolicy Bypass -File .\chay.ps1
```

Lệnh trên sẽ tự động kích hoạt và sắp xếp 4 cửa sổ Terminal nổi tương ứng trên màn hình:

- Cửa sổ 1: Web Flask Server điều hành (http://127.0.0.1:5000)
- Cửa sổ 2: Spark Streaming Chấm điểm rủi ro (cham_diem.py --turbo)
- Cửa sổ 3: Spark Stateful Streaming cập nhật trạng thái hồ sơ (cap_nhat_trang_thai.py --turbo)
- Cửa sổ 4: Kafka Producer Phát giao dịch trực tuyến (generator.py)

Phụ lục C: Hướng dẫn làm sạch dữ liệu và chạy bộ kiểm thử nghiệm thu

Để làm sạch toàn bộ dữ liệu giao dịch về trạng thái ban đầu v0 và khôi phục mô hình gốc model_v1:

```
.\venv311\Scripts\python.exe reset_he_thong.py
```

Để thực thi bộ kiểm thử nghiệm thu tự động kiểm chứng 8 tiêu chuẩn chấp nhận hệ thống:

```
.\venv311\Scripts\python.exe tests/test_acceptance.py
```

Phụ lục D: Mã nguồn Cốt lõi Đối chiếu Toàn Hệ thống (Docker, HDFS, Kafka, Spark, MLlib & Airflow)

Toàn bộ mã nguồn đầy đủ được công khai tại: **https://github.com/nguyenthanhphat-144/BigDaTa**

D.1. Cấu hình cụm phân tán 7 Container: 1 NameNode + 3 DataNodes + Zookeeper + Kafka + Airflow (docker-compose.yml):

```
services:
  namenode1:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
    container_name: namenode1
    ports: ["9870:9870", "9000:9000"]
    environment:
      - CLUSTER_NAME=bd02-fraud-cluster
      - CORE_CONF_fs_defaultFS=hdfs://namenode1:9000
```

```
      -                                                                    HDFS_CONF_dfs_replication=3
      -                                                                HDFS_CONF_dfs_webhdfs_enabled=true
      -                                                  HDFS_CONF_dfs_client_use_datanode_hostname=true
      -                                                HDFS_CONF_dfs_datanode_use_datanode_hostname=true
  datanode1:
    image:                                               bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    ports:                      ["9864:9864",                   "9866:9866",                    "9867:9867"]
    environment:
      -                                                     CORE_CONF_fs_defaultFS=hdfs://namenode1:9000
      -                                                      HDFS_CONF_dfs_datanode_address=0.0.0.0:9866
  datanode2:
    image:                                               bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    ports:                      ["9865:9864",                   "9876:9876",                    "9877:9877"]
    environment:
      -                                                     CORE_CONF_fs_defaultFS=hdfs://namenode1:9000
      -                                                      HDFS_CONF_dfs_datanode_address=0.0.0.0:9876
  datanode3:
    image:                                               bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    ports:                      ["9868:9864",                   "9886:9886",                    "9887:9887"]
    environment:
      -                                                     CORE_CONF_fs_defaultFS=hdfs://namenode1:9000
      -                                                      HDFS_CONF_dfs_datanode_address=0.0.0.0:9886
  kafka:
    image:                                                                    wurstmeister/kafka:latest
    ports:                                                                                ["9092:9092"]
    environment:
      KAFKA_BROKER_ID:                                                                                1
      KAFKA_ZOOKEEPER_CONNECT:                                                          zookeeper:2181
      KAFKA_CREATE_TOPICS:                                                "transactions:3:1,alerts:3:1"
  airflow:
    image:                                                             apache/airflow:2.9.3-python3.11
    ports:                                                                                ["8080:8080"]
    volumes:
      -                                                               ./airflow/dags:/opt/airflow/dags
      -                                                               ./airflow/logs:/opt/airflow/logs
      - ./:/opt/airflow/project
```

D.2. Kết nối trực tiếp Spark <-> HDFS (hdfs://namenode1:9000/lakehouse, replication=3) (config/settings.py):

```
HDFS_NAMENODE_URI          =          os.environ.get("HDFS_NAMENODE_URI",          "hdfs://namenode1:9000")
HDFS_LAKEHOUSE_ROOT                          =                              f"{HDFS_NAMENODE_URI}/lakehouse"
HDFS_CREDITCARD_CSV                     =                     f"{HDFS_LAKEHOUSE_ROOT}/raw/creditcard.csv"
HDFS_MODELS_DIR                                          =          f"{HDFS_LAKEHOUSE_ROOT}/models"

BRONZE_GIAO_DICH                                       =       f"{HDFS_LAKEHOUSE_ROOT}/bronze/giao_dich"
SILVER_TRANG_THAI                                =      f"{HDFS_LAKEHOUSE_ROOT}/silver/trang_thai_khach_hang"
SILVER_KET_LUAN                                      =     f"{HDFS_LAKEHOUSE_ROOT}/silver/ket_luan_tham_dinh"
SILVER_TAP_HUAN_LUYEN                  =                   f"{HDFS_LAKEHOUSE_ROOT}/silver/tap_huan_luyen"
GOLD_DIEM_RUI_RO                                       =       f"{HDFS_LAKEHOUSE_ROOT}/gold/diem_rui_ro"
GOLD_THEO_DOI_DRIFT                                =           f"{HDFS_LAKEHOUSE_ROOT}/gold/theo_doi_drift"
QUARANTINE_DIR                                          =      f"{HDFS_LAKEHOUSE_ROOT}/_quarantine"

def     get_spark_session(app_name="FraudDetectionLakehouse",     shuffle_partitions="4",     ui_port=None):
    builder                                              =                                                (
        SparkSession.builder.appName(app_name)
        .master(SPARK_MASTER)
        .config("spark.jars.packages",   "io.delta:delta-spark_2.12:3.2.0,org.apache.spark:spark-sql-kafka-
0-10_2.12:3.5.1")
        .config("spark.sql.extensions",                          "io.delta.sql.DeltaSparkSessionExtension")
        .config("spark.sql.catalog.spark_catalog",       "org.apache.spark.sql.delta.catalog.DeltaCatalog")
        .config("spark.hadoop.fs.defaultFS",                                                HDFS_NAMENODE_URI)
        .config("spark.hadoop.dfs.client.use.datanode.hostname",                                          "true")
        .config("spark.hadoop.dfs.replication",                                                             "3")
    )
    return builder.getOrCreate()
```

D.3. Tiền xử lý Bronze -> Silver, Khử trùng lặp & Cách ly Dữ liệu Bẩn (_quarantine) trên HDFS (transform/bronze_to_silver.py):

```
raw_df                       =                      self.spark.read.format("delta").load(BRONZE_GIAO_DICH)
cond_valid_amount                            =             F.col("so_tien")             >             0.0
```

```
cond_valid_acc_src = F.col("tai_khoan_nguon").isNotNull() & (F.length(F.trim(F.col("tai_khoan_nguon"))) >
0)
cond_valid_acc_dst = F.col("tai_khoan_dich").isNotNull() & (F.length(F.trim(F.col("tai_khoan_dich"))) > 0)
cond_valid_loai          =  F.col("loai").isin(["CHUYEN_KHOAN",  "RUT_TIEN",  "THANH_TOAN",  "NAP_TIEN"])

flagged_df                                          =                                          raw_df.withColumn(
    "quarantine_reason",
    F.when(~cond_valid_amount,                                                    F.lit("SO_TIEN_KHONG_HOP_LE"))
     .when(~cond_valid_acc_src,                                                  F.lit("THIEU_TAI_KHOAN_NGUON"))
     .when(~cond_valid_acc_dst,                                                   F.lit("THIEU_TAI_KHOAN_DICH"))
     .when(~cond_valid_loai,                                                    F.lit("LOAI_GD_KHONG_XAC_DINH"))
     .otherwise(F.lit("HOP_LE"))
)
invalid_df          =          flagged_df.filter(F.col("quarantine_reason")          !=          "HOP_LE")
if                          invalid_df.count()                                 >                          0:
    invalid_df.write.format("delta").option("mergeSchema",      "true").mode("append").save(QUARANTINE_DIR)
dedup_df = flagged_df.filter(F.col("quarantine_reason") == "HOP_LE").dropDuplicates(["ma_giao_dich"])
```

D.4. Đọc creditcard.csv trực tiếp từ HDFS & Thuật toán Point-in-Time Join chống Data Leakage (ml/tao_tap_huan_luyen.py):

```
hdfs_csv_uri   =   ensure_creditcard_on_hdfs()      #   hdfs://namenode1:9000/lakehouse/raw/creditcard.csv
raw_spark_df  =  self.spark.read.option("header",  "true").option("inferSchema",  "true").csv(hdfs_csv_uri)

# Cửa sổ trượt nghiêm ngặt [T - Window, T - 1s]: loại trừ tuyệt đối bản thân giao dịch đang xét tại thời
điểm                                                                                                    T
def                                                                                  w_strict_past(seconds_back):
    return                                                                                                  (
        Window.partitionBy("tai_khoan_nguon")
        .orderBy("epoch_sec")
        .rangeBetween(-seconds_back,                                                                      -1)
    )

enriched                                                =                                                  (
    df.withColumn("epoch_sec",                                                F.col("thoi_diem").cast("long"))
    .withColumn("so_gd_1h",                 F.count("ma_giao_dich").over(w_strict_past(3600)).cast("int"))
    .withColumn("so_gd_24h",               F.count("ma_giao_dich").over(w_strict_past(86400)).cast("int"))
    .withColumn("so_gd_7d",               F.count("ma_giao_dich").over(w_strict_past(604800)).cast("int"))
    .withColumn("tong_tien_24h",    F.coalesce(F.sum("so_tien").over(w_strict_past(86400)),    F.lit(0.0)))
    .withColumn("tb_tien_30d",    F.coalesce(F.avg("so_tien").over(w_strict_past(2592000)),    F.lit(88.0)))
    .withColumn("std_tien_30d",                F.coalesce(F.stddev("so_tien").over(w_strict_past(2592000)),
F.lit(120.0)))
    .withColumn("z_score_so_tien",      F.round((F.col("so_tien")      -      F.col("tb_tien_30d"))      /
F.greatest(F.col("std_tien_30d"),                                F.lit(1.0)),                                4))
)
enriched.write.format("delta").option("overwriteSchema",
"true").mode("overwrite").save(SILVER_TAP_HUAN_LUYEN)
```

D.5. Spark Structured Streaming: Đọc Kafka, Watermark 1m, Stream-Static Join & Dual-Sink lên HDFS (streaming/cham_diem.py):

```
self.model  =  PipelineModel.load(f"{HDFS_MODELS_DIR}/{version_name}")     #  Nạp  trực  tiếp  từ  HDFS
raw_stream                                              =                                                  (
    self.spark.readStream.format("kafka")
    .option("kafka.bootstrap.servers",                                                  KAFKA_BOOTSTRAP_SERVERS)
    .option("subscribe",                                                               KAFKA_TOPIC_TRANSACTIONS)
    .option("startingOffsets",                                                                    "latest")
    .option("groupIdPrefix",                                                          "bd02_spark_cham_diem")
    .load()
)
parsed_stream                                             =                                                 (
    raw_stream.select(F.from_json(F.col("value").cast("string"),             KAFKA_TX_SCHEMA).alias("tx"))
    .select("tx.*")
    .withColumn("thoi_diem",                                                    F.to_timestamp("thoi_diem"))
    .withWatermark("thoi_diem",                                  "1                                  minute")
)
# Trong foreachBatch: Stream-Static Join với silver.trang_thai_khach_hang trên HDFS + Chấm điểm MLlib +
Dual-Sink
silver_state_df                 =                 self.spark.read.format("delta").load(SILVER_TRANG_THAI)
enriched_df = valid_tx.join(silver_state_df, valid_tx["tai_khoan_nguon"] == silver_state_df["tai_khoan"],
"left")
scored_df                                =                                self.model.transform(enriched_df)
```

```
bronze_df.write.format("delta").partitionBy("ngay").mode("append").save(BRONZE_GIAO_DICH)
gold_df.write.format("delta").partitionBy("ngay").mode("append").save(GOLD_DIEM_RUI_RO)
```

D.6. Spark Stateful Streaming: Cập nhật Hồ sơ Hành vi Khách hàng bằng Delta MERGE INTO (streaming/cap_nhat_trang_thai.py):

```
silver_table               =               DeltaTable.forPath(self.spark,               SILVER_TRANG_THAI)
silver_table.alias("s").merge(
    agg_new.alias("u"),
    "s.tai_khoan                                        =                                        u.tai_khoan_nguon"
).whenMatchedUpdate(set={
    "so_gd_1h":                       "s.so_gd_1h                      +                       u.batch_count",
    "so_gd_24h":                      "s.so_gd_24h                     +                       u.batch_count",
    "so_gd_7d":                       "s.so_gd_7d                      +                       u.batch_count",
    "tong_tien_24h":           "ROUND(s.tong_tien_24h           +           u.batch_amount,           2)",
    "so_tk_dich_khac_nhau_24h":          "s.so_tk_dich_khac_nhau_24h          +          u.batch_dst_count",
    "lan_gd_gan_nhat":                                                                    "u.max_thoi_diem",
    "cap_nhat_luc":                                                                  "current_timestamp()"
}).whenNotMatchedInsert(values={
    "tai_khoan":   "u.tai_khoan_nguon",   "so_gd_1h":   "u.batch_count",   "so_gd_24h":   "u.batch_count",
    "so_gd_7d":  "u.batch_count",  "tong_tien_24h":  "u.batch_amount",  "so_tien_tb_30d":  "u.batch_amount",
    "so_tien_dolech_30d":       "u.batch_std",       "so_tk_dich_khac_nhau_24h":       "u.batch_dst_count",
    "gio_giao_dich_pho_bien":   "u.batch_hours",   "lan_gd_gan_nhat":   "u.max_thoi_diem",   "cap_nhat_luc":
"current_timestamp()"
}).execute()
```

D.7. Đối sánh 3 Thuật toán x 2 Chiến lược Mất cân bằng & Lưu PipelineModel lên HDFS (ml/huan_luyen.py):

```
#  Chronological  Split  70%  Train  -  15%  Validation  -  15%  Test  (chống  rò  rỉ  tương  lai)
train_df,            val_df,            test_df            =            self.chronological_split(full_df)
weighted_train_df, pos_weight = self.add_class_weights(train_df)              # Chiến lược 1: weightCol
undersampled_train_df = self.apply_undersampling(train_df, target_ratio=20)  # Chiến lược 2: Undersampling
1:20

assembler  =  VectorAssembler(inputCols=self.FEATURE_COLS,  outputCol="features",  handleInvalid="skip")
# Huấn luyện 6 cấu hình: GBTClassifier, RandomForestClassifier, LogisticRegression (L2) x {WeightCol,
Undersampling                                                                                    1:20}
# Đánh giá Precision@K (K=50, 100, 200, 500), PR-AUC, ROC-AUC và chọn mô hình tối ưu nhất lưu trực tiếp
lên                                                                                              HDFS:
hdfs_model_path  =  f"{HDFS_MODELS_DIR}/{version_tag}"   #  hdfs://namenode1:9000/lakehouse/models/model_v1
(hoặc                                                                                           model_v2)
best_model.write().overwrite().save(hdfs_model_path)
```

D.8. Hệ thống 4 DAG Điều phối Tự động trên Apache Airflow 2.9.3 (airflow/dags/gl_*.py):

```
# 1. DAG gl_giam_sat_streaming (schedule='*/5 * * * *'): Kiểm tra Spark REST :4040, Kafka :9092 & HDFS
Bronze
dag_1     =     DAG("gl_giam_sat_streaming",     schedule="*/5     *     *     *     *",     catchup=False)
kiem_tra_health           >>           [he_thong_on_dinh,           canh_bao_va_khoi_dong_lai_spark_submit]

# 2.  DAG  gl_kiem_tra_drift  (schedule='0  *  *  *  *'):  Tính  PSI  &  KS-test  9  đặc  trưng  trên  HDFS
gold.theo_doi_drift
dag_2      =      DAG("gl_kiem_tra_drift",      schedule="0      *      *      *      *",      catchup=False)
#  Rẽ  nhánh:  PSI  <  0.10  (on_dinh)  |  0.10  <=  PSI  <  0.25  (canh_bao)  |  PSI  >=  0.25  ->
TriggerDagRunOperator('gl_huan_luyen_lai')
tinh_psi_ks_va_re_nhanh  >>  [nhanh_drift_on_dinh,  nhanh_drift_canh_bao,  nhanh_drift_nghiem_trong  >>
kich_hoat_dag_retrain]

# 3. DAG gl_huan_luyen_lai (schedule='0 3 * * *'): Kiểm tra nhãn silver.ket_luan_tham_dinh, Point-in-Time
Join                                                &                                                Retrain
dag_3      =      DAG("gl_huan_luyen_lai",      schedule="0      3      *      *      *",      catchup=False)
kiem_tra_du_500_nhan_moi >> tao_tap_huan_luyen_point_in_time >> [dang_ky_mo_hinh_moi_va_nap_lai_streaming,
giu_nguyen_mo_hinh_cu]

# 4. DAG gl_bao_tri (schedule='0 2 * * 0'): OPTIMIZE ZORDER BY (tai_khoan_nguon, thoi_diem), VACUUM (168h)
&                       Xóa                       state                       >                       30d
dag_4       =       DAG("gl_bao_tri",       schedule="0       2       *       *       0",       catchup=False)
optimize_va_zorder >> vacuum_delta_tables >> xoa_state_khong_hoat_dong_30d
```

Phụ lục E: Bằng chứng Nhật ký Thực thi Hệ thống Thực tế (HDFS, Kafka, Spark, ML & Airflow)

E.1. Nhật ký Kiểm tra Cụm HDFS 3 DataNodes, Dung lượng, Danh sách Tệp & Nhân bản Block 3x (docs/minh_chung/01_kiem_tra_hdfs.log):

```
==========================================================================================
MINH CHỨNG 01: KIỂM TRA HẠ TẦNG CỤM HADOOP HDFS (1 NAMENODE + 3 DATANODES, REPLICATION=3)
Thời điểm trích xuất: 2026-10-29 14:07:06
==========================================================================================

$ docker exec namenode1 hdfs dfsadmin -report
Configured Capacity: 3243303530496 (2.95 TB)
Present Capacity: 3052742083189 (2.78 TB)
DFS Remaining: 3052246933504 (2.78 TB)
DFS Used: 495149685 (472.21 MB)
DFS Used%: 0.02%
Replicated Blocks:
        Under replicated blocks: 0
        Blocks with corrupt replicas: 0
        Missing blocks: 0
        Missing blocks (with replication factor 1): 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0
Erasure Coded Block Groups:
        Low redundancy block groups: 0
        Block groups with corrupt internal blocks: 0
        Missing block groups: 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0

-------------------------------------------------
Live datanodes (3):

Name: 172.19.0.5:9876 (datanode2.bigdata2_default)
Hostname: datanode2
Decommission Status : Normal
Configured Capacity: 1081101176832 (1006.85 GB)
DFS Used: 165164231 (157.51 MB)
Non DFS Used: 8528001849 (7.94 GB)
DFS Remaining: 1017415655424 (947.54 GB)
DFS Used%: 0.02%
DFS Remaining%: 94.11%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 1
Last contact: Thu Oct 29 07:07:06 UTC 2026
Last Block Report: Thu Oct 29 06:32:04 UTC 2026
Num of Blocks: 210


Name: 172.19.0.6:9886 (datanode3.bigdata2_default)
Hostname: datanode3
Decommission Status : Normal
Configured Capacity: 1081101176832 (1006.85 GB)
DFS Used: 165114521 (157.47 MB)
Non DFS Used: 8528084327 (7.94 GB)
DFS Remaining: 1017415622656 (947.54 GB)
DFS Used%: 0.02%
DFS Remaining%: 94.11%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 1
Last contact: Thu Oct 29 07:07:08 UTC 2026
Last Block Report: Thu Oct 29 06:32:02 UTC 2026
Num of Blocks: 210


Name: 172.19.0.8:9866 (datanode1.bigdata2_default)
Hostname: datanode1
Decommission Status : Normal
Configured Capacity: 1081101176832 (1006.85 GB)
```

```
DFS                     Used:                     164870933                     (157.23                     MB)
Non                 DFS                 Used:                 8528295147                 (7.94                 GB)
DFS                    Remaining:                 1017415655424                 (947.54                 GB)
DFS                                               Used%:                                                0.02%
DFS                                            Remaining%:                                              94.11%
Configured                Cache                Capacity:                 0                 (0                 B)
Cache                        Used:                        0                        (0                        B)
Cache                      Remaining:                      0                      (0                      B)
Cache                                             Used%:                                              100.00%
Cache                                          Remaining%:                                              0.00%
Xceivers:                                                                                                    1
Last           contact:           Thu           Oct           29           07:07:07           UTC           2026
Last         Block         Report:         Thu         Oct         29         06:32:04         UTC         2026
Num                                 of                                  Blocks:                                 210

$         docker         exec         namenode1         hdfs         dfs         -du         -h         /lakehouse
12.0          K                                  36.1          K                                  /lakehouse/_audit
2.2        K                                  6.7        K                                  /lakehouse/_checkpoints
30.9          K                                  92.7          K                                  /lakehouse/_quarantine
828.5          K                      2.4          M                                  /lakehouse/bronze
429.7          K                       1.3          M                                    /lakehouse/gold
51.1           K                                  153.4            K                                  /lakehouse/models
143.8              M                                  431.5              M                                  /lakehouse/raw
9.9          M                                  29.7          M                                  /lakehouse/silver
0                                          0                                            /lakehouse/warehouse


$         docker         exec         namenode1         hdfs         dfs         -ls         -R         /lakehouse
drwxr-xr-x          -    root      supergroup                                0   2026-10-29   07:06   /lakehouse/_audit
drwxr-xr-x         -   root      supergroup                               0   2026-10-29   07:03   /lakehouse/_audit/audit
-rw-r--r--      3  root    supergroup            1243  2026-10-29  07:03  /lakehouse/_audit/audit/bootstrap_summary.json
drwxr-xr-x         -   root      supergroup                               0   2026-10-29   06:58   /lakehouse/_audit/bronze
-rw-r--r--      3  root    supergroup             162  2026-10-29  06:58  /lakehouse/_audit/bronze/schema_metadata.json
drwxr-xr-x         -   root      supergroup                               0   2026-10-29   07:06   /lakehouse/_audit/gold
-rw-r--r--      3  root    supergroup             170  2026-10-29  07:05  /lakehouse/_audit/gold/batch_0_summary.json
-rw-r--r--      3  root    supergroup             167  2026-10-29  07:05  /lakehouse/_audit/gold/batch_1_summary.json
-rw-r--r--      3  root    supergroup             168  2026-10-29  07:05  /lakehouse/_audit/gold/batch_2_summary.json
-rw-r--r--       3  root    supergroup            1428  2026-10-29  07:06  /lakehouse/_audit/gold/drift_latest.json
-rw-r--r--      3  root    supergroup             144  2026-10-29  06:58  /lakehouse/_audit/gold/schema_metadata.json
drwxr-xr-x         -   root      supergroup                               0   2026-10-29   07:06   /lakehouse/_audit/maintenance
-rw-r--r--     3  root    supergroup             284  2026-10-29  07:06  /lakehouse/_audit/maintenance/zorder_latest.json
drwxr-xr-x         -   root      supergroup                               0   2026-10-29   07:06   /lakehouse/_audit/models
-rw-r--r--    3 root   supergroup         191 2026-10-29 07:06 /lakehouse/_audit/models/airflow_model_v2_evaluation.json
-rw-r--r--    3 root   supergroup         194 2026-10-29 06:43 /lakehouse/_audit/models/airflow_model_v4_evaluation.json
-rw-r--r--      3  root    supergroup            6976  2026-10-29  07:00  /lakehouse/_audit/models/model_v1_metadata.json
-rw-r--r--     3 root   supergroup           262 2026-10-29 07:06 /lakehouse/_audit/models/model_v2_retrain_audit.json
drwxr-xr-x         -   root      supergroup                               0   2026-10-29   07:02   /lakehouse/_audit/silver
-rw-r--r--      3  root    supergroup             101  2026-10-29  07:02  /lakehouse/_audit/silver/hygiene_report.json
-rw-r--r--      3  root    supergroup             168  2026-10-29  06:58  /lakehouse/_audit/silver/schema_metadata.json
-rw-r--r--      3  root    supergroup              164  2026-10-29  07:01  /lakehouse/_audit/silver/state_batch_1.json
-rw-r--r--     3 root   supergroup           169 2026-10-29 06:40 /lakehouse/_audit/silver/training_snapshot_v0.json
-rw-r--r--     3 root   supergroup           169 2026-10-29 06:51 /lakehouse/_audit/silver/training_snapshot_v1.json
-rw-r--r--     3 root   supergroup           169 2026-10-29 06:59 /lakehouse/_audit/silver/training_snapshot_v2.json
drwxr-xr-x         -   Admin   supergroup                               0   2026-10-29   07:04   /lakehouse/_checkpoints
drwxr-xr-x       -  Admin   supergroup                          0  2026-10-29  07:04  /lakehouse/_checkpoints/cham_diem
drwxr-xr-x      -  Admin  supergroup                     0  2026-10-29  07:05  /lakehouse/_checkpoints/cham_diem/commits
-rw-r--r--      3  Admin  supergroup                  41  2026-10-29  07:05  /lakehouse/_checkpoints/cham_diem/commits/0
-rw-r--r--      3  Admin  supergroup                  41  2026-10-29  07:05  /lakehouse/_checkpoints/cham_diem/commits/1
-rw-r--r--      3  Admin  supergroup                  41  2026-10-29  07:05  /lakehouse/_checkpoints/cham_diem/commits/2
-rw-r--r--      3  Admin  supergroup                  45  2026-10-29  07:04  /lakehouse/_checkpoints/cham_diem/metadata
drwxr-xr-x      -  Admin  supergroup                     0  2026-10-29  07:05  /lakehouse/_checkpoints/cham_diem/offsets
-rw-r--r--      3  Admin  supergroup                 681  2026-10-29  07:04  /lakehouse/_checkpoints/cham_diem/offsets/0
-rw-r--r--      3  Admin  supergroup                 693  2026-10-29  07:05  /lakehouse/_checkpoints/cham_diem/offsets/1
-rw-r--r--      3  Admin  supergroup                 693  2026-10-29  07:05  /lakehouse/_checkpoints/cham_diem/offsets/2
drwxr-xr-x      -  Admin  supergroup                     0  2026-10-29  07:04  /lakehouse/_checkpoints/cham_diem/sources
drwxr-xr-x      -  Admin  supergroup                   0  2026-10-29  07:04  /lakehouse/_checkpoints/cham_diem/sources/0
-rw-r--r--      3  Admin  supergroup                40  2026-10-29  07:04  /lakehouse/_checkpoints/cham_diem/sources/0/0
drwxr-xr-x         -   Admin   supergroup                               0   2026-10-29   07:02   /lakehouse/_quarantine
drwxr-xr-x       -  Admin   supergroup                          0  2026-10-29  07:02  /lakehouse/_quarantine/_delta_log
-rw-r--r--                3     Admin     supergroup                                          7995     2026-10-29     07:02
/lakehouse/_quarantine/_delta_log/00000000000000000000.json
drwxr-xr-x      -  Admin  supergroup                     0  2026-10-29  07:02  /lakehouse/_quarantine/_delta_log/_commits
-rw-r--r--    3 Admin  supergroup        11880 2026-10-29 07:02 /lakehouse/_quarantine/part-00000-335d6ecb-5108-42b1-
aa05-6db9366d6f84-c000.snappy.parquet
-rw-r--r--    3 Admin  supergroup        11755 2026-10-29 07:02 /lakehouse/_quarantine/part-00002-ba710410-6e71-4824-
9b32-c4d1db4e3af5-c000.snappy.parquet
drwxr-xr-x          -    Admin    supergroup                                0   2026-10-29   06:35   /lakehouse/bronze
drwxr-xr-x         -   Admin   supergroup                           0   2026-10-29   07:01   /lakehouse/bronze/giao_dich
drwxr-xr-x      -  Admin  supergroup                     0  2026-10-29  07:05  /lakehouse/bronze/giao_dich/_delta_log
-rw-r--r--                3     Admin     supergroup                                          3940     2026-10-29     06:35
/lakehouse/bronze/giao_dich/_delta_log/00000000000000000000.json
```

```
  ... [Đã rút gọn danh sách các part-parquet trung gian — Xem đủ 759 dòng tại docs/minh_chung/01_kiem_tra_hdfs.log]
...
$ docker exec namenode1 sh -c "hdfs dfs -cat /lakehouse/raw/creditcard.csv | head -n 6"
"Time","V1","V2","V3","V4","V5","V6","V7","V8","V9","V10","V11","V12","V13","V14","V15","V16","V17","V18","V19","V20"
,"V21","V22","V23","V24","V25","V26","V27","V28","Amount","Class"
0,-1.3598071336738,-0.0727811733098497,2.53634673796914,1.37815522427443,-
0.338320769942518,0.462387777762292,0.239598554061257,0.0986979012610507,0.363786969611213,0.0907941719789316,-
0.551599533260813,-0.617800855762348,-0.991389847235408,-0.311169353699879,1.46817697209427,-
0.470400525259478,0.207971241929242,0.0257905801985591,0.403992960255733,0.251412098239705,-
0.018306777944153,0.277837575558899,-0.110473910188767,0.0669280749146731,0.128539358273528,-
0.189114843888824,0.133558376740387,-0.0210530534538215,149.62,"0"
0,1.19185711131486,0.26615071205963,0.16648011335321,0.448154078460911,0.0600176492822243,-0.0823608088155687,-
0.0788029833323113,0.0851016549148104,-0.255425128109186,-
0.166974414004614,1.61272666105479,1.06523531137287,0.48909501589608,-
0.143772296441519,0.635558093258208,0.463917041022171,-0.114804663102346,-0.183361270123994,-0.145783041325259,-
0.0690831352230203,-0.225775248033138,-0.638671952771851,0.101288021253234,-
0.339846475529127,0.167170404418143,0.125894532368176,-0.00898309914322813,0.0147241691924927,2.69,"0"
1,-1.35835406159823,-1.34016307473609,1.77320934263119,0.379779593034328,-
0.503198133318193,1.80049938079263,0.791460956450422,0.247675786588991,-
1.51465432260583,0.207642865216696,0.624501459424895,0.066083685268831,0.717292731410831,-
0.165945922763554,2.34586494901581,-2.89008319444231,1.10996937869599,-0.121359313195888,-
2.26185709530414,0.524979725224404,0.247998153469754,0.771679401917229,0.909412262347719,-0.689280956490685,-
0.327641833735251,-0.139096571514147,-0.0553527940384261,-0.0597518405929204,378.66,"0"
1,-0.966271711572087,-0.185226008082898,1.79299333957872,-0.863291275036453,-
0.0103088796030823,1.24720316752486,0.237608939777178,0.377435874652262,-1.38702406270197,-0.0549519224713749,-
0.226487263835401,0.178228225877303,0.507756869957169,-0.28792374549456,-0.631418117709045,-1.0596472454325,-
0.684092786345479,1.96577500349538,-1.2326219700892,-0.208037781160366,-0.108300452035545,0.00527359678253453,-
0.190320518742841,-1.17557533186321,0.647376034602038,-
0.221928844458407,0.0627228487293033,0.0614576285006353,123.5,"0"
2,-1.15823309349523,0.877736754848451,1.548717846511,0.403033933955121,-
0.407193377311653,0.0959214624684256,0.592940745385545,-0.270532677192282,0.817739308235294,0.753074431976354,-
0.822842877946363,0.53819555014995,1.3458515932154,-1.11966983471731,0.175121130008994,-0.451449182813529,-
0.237033239362776,-0.0381947870352842,0.803486924960175,0.408542360392758,-0.00943069713232919,0.79827849458971,-
0.137458079619063,0.141266983824769,-
0.206009587619756,0.502292224181569,0.219422229513348,0.215153147499206,69.99,"0"

$ docker exec namenode1 hdfs dfs -cat /lakehouse/bronze/giao_dich/_delta_log/00000000000000000000.json
{"commitInfo":{"timestamp":1793255736526,"operation":"WRITE","operationParameters":{"mode":"Overwrite","partitionBy":
"[\"ngay\"]"},"isolationLevel":"Serializable","isBlindAppend":false,"operationMetrics":{"numFiles":"0","numOutputRows
":"0","numOutputBytes":"0"},"engineInfo":"Apache-Spark/3.5.1       Delta-Lake/3.2.0","txnId":"f3a4b0e7-7328-4c96-b798-
5a221002e751"}}
{"metaData":{"id":"a26dfa65-80d7-4a71-9276-
ab758da29944","format":{"provider":"parquet","options":{}},"schemaString":"{\"type\":\"struct\",\"fields\":[{\"name\"
:\"ma_giao_dich\",\"type\":\"string\",\"nullable\":true,\"metadata\":{}},{\"name\":\"thoi_diem\",\"type\":\"timestamp
\",\"nullable\":true,\"metadata\":{}},{\"name\":\"thoi_diem_nhan\",\"type\":\"timestamp\",\"nullable\":true,\"metadat
a\":{}},{\"name\":\"tai_khoan_nguon\",\"type\":\"string\",\"nullable\":true,\"metadata\":{}},{\"name\":\"tai_khoan_di
ch\",\"type\":\"string\",\"nullable\":true,\"metadata\":{}},{\"name\":\"loai\",\"type\":\"string\",\"nullable\":true,
\"metadata\":{}},{\"name\":\"so_tien\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}},{\"name\":\"so_du_truoc
_nguon\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}},{\"name\":\"so_du_sau_nguon\",\"type\":\"double\",\"n
ullable\":true,\"metadata\":{}},{\"name\":\"so_du_truoc_dich\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}}
,{\"name\":\"so_du_sau_dich\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}},{\"name\":\"kenh\",\"type\":\"str
ing\",\"nullable\":true,\"metadata\":{}},{\"name\":\"ngay\",\"type\":\"date\",\"nullable\":true,\"metadata\":{}},{\"
name\":\"ma_thiet_bi\",\"type\":\"string\",\"nullable\":true,\"metadata\":{}},{\"name\":\"dia_chi_ip\",\"type\":\"str
ing\",\"nullable\":true,\"metadata\":{}},{\"name\":\"v1\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}},{\"n
ame\":\"v2\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}},{\"name\":\"v3\",\"type\":\"double\",\"nullable\"
:true,\"metadata\":{}},{\"name\":\"v4\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}},{\"name\":\"v5\",\"typ
e\":\"double\",\"nullable\":true,\"metadata\":{}},{\"name\":\"v6\",\"type\":\"double\",\"nullable\":true,\"metadata\"
:{}},{\"name\":\"v7\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}},{\"name\":\"v8\",\"type\":\"double\",\"n
ullable\":true,\"metadata\":{}},{\"name\":\"v9\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}},{\"name\":\"v
10\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}},{\"name\":\"v11\",\"type\":\"double\",\"nullable\":true,\
"metadata\":{}},{\"name\":\"v12\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}},{\"name\":\"v13\",\"type\":\
"double\",\"nullable\":true,\"metadata\":{}},{\"name\":\"v14\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}}
,{\"name\":\"v15\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}},{\"name\":\"v16\",\"type\":\"double\",\"nul
lable\":true,\"metadata\":{}},{\"name\":\"v17\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}},{\"name\":\"v1
8\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}},{\"name\":\"v19\",\"type\":\"double\",\"nullable\":true,\"
metadata\":{}},{\"name\":\"v20\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}},{\"name\":\"v21\",\"type\":\"
double\",\"nullable\":true,\"metadata\":{}},{\"name\":\"v22\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}},
{\"name\":\"v23\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}},{\"name\":\"v24\",\"type\":\"double\",\"null
able\":true,\"metadata\":{}},{\"name\":\"v25\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}},{\"name\":\"v26
\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}},{\"name\":\"v27\",\"type\":\"double\",\"nullable\":true,\"m
etadata\":{}},{\"name\":\"v28\",\"type\":\"double\",\"nullable\":true,\"metadata\":{}}]}","partitionColumns":["ngay"]
,"configuration":{"delta.enableChangeDataFeed":"true"},"createdTime":1793255720606}}
{"protocol":{"minReaderVersion":1,"minWriterVersion":4}}

$ docker exec namenode1 hdfs fsck /lakehouse -files -blocks -locations
FSCK started by root (auth:SIMPLE) from /172.19.0.4 for path /lakehouse at Thu Oct 29 07:07:18 UTC 2026
/lakehouse                                                                                                  <dir>
/lakehouse/_audit                                                                                           <dir>
/lakehouse/_audit/audit                                                                                     <dir>
/lakehouse/_audit/audit/bootstrap_summary.json 1243 bytes, replicated: replication=3, 1 block(s):      OK
0.             BP-912878437-172.19.0.2-1792157541974:blk_1073742061_1237             len=1243             Live_repl=3
```

```
[DatanodeInfoWithStorage[172.19.0.8:9866,DS-94719152-b6a1-40bd-aa2d-4c0c8f9e491f,DISK],
DatanodeInfoWithStorage[172.19.0.5:9876,DS-d9940a9e-5456-4a61-af82-be83e9a84d16,DISK],
DatanodeInfoWithStorage[172.19.0.6:9886,DS-c0e17b1e-5075-48ed-9f98-f3e9c6065d1f,DISK]]

/lakehouse/_audit/bronze                                                                    <dir>
/lakehouse/_audit/bronze/schema_metadata.json   162   bytes,   replicated:   replication=3,   1   block(s):      OK
0.            BP-912878437-172.19.0.2-1792157541974:blk_1073742000_1176            len=162            Live_repl=3
[DatanodeInfoWithStorage[172.19.0.6:9886,DS-c0e17b1e-5075-48ed-9f98-f3e9c6065d1f,DISK],
DatanodeInfoWithStorage[172.19.0.5:9876,DS-d9940a9e-5456-4a61-af82-be83e9a84d16,DISK],
DatanodeInfoWithStorage[172.19.0.8:9866,DS-94719152-b6a1-40bd-aa2d-4c0c8f9e491f,DISK]]

/lakehouse/_audit/gold                                                                      <dir>
/lakehouse/_audit/gold/batch_0_summary.json   170   bytes,   replicated:   replication=3,   1   block(s):      OK
0.            BP-912878437-172.19.0.2-1792157541974:blk_1073742072_1248            len=170            Live_repl=3
[DatanodeInfoWithStorage[172.19.0.5:9876,DS-d9940a9e-5456-4a61-af82-be83e9a84d16,DISK],
DatanodeInfoWithStorage[172.19.0.8:9866,DS-94719152-b6a1-40bd-aa2d-4c0c8f9e491f,DISK],
DatanodeInfoWithStorage[172.19.0.6:9886,DS-c0e17b1e-5075-48ed-9f98-f3e9c6065d1f,DISK]]

/lakehouse/_audit/gold/batch_1_summary.json   167   bytes,   replicated:   replication=3,   1   block(s):      OK
0.            BP-912878437-172.19.0.2-1792157541974:blk_1073742082_1258            len=167            Live_repl=3
[DatanodeInfoWithStorage[172.19.0.8:9866,DS-94719152-b6a1-40bd-aa2d-4c0c8f9e491f,DISK],
DatanodeInfoWithStorage[172.19.0.6:9886,DS-c0e17b1e-5075-48ed-9f98-f3e9c6065d1f,DISK],
DatanodeInfoWithStorage[172.19.0.5:9876,DS-d9940a9e-5456-4a61-af82-be83e9a84d16,DISK]]

/lakehouse/_audit/gold/batch_2_summary.json   168   bytes,   replicated:   replication=3,   1   block(s):      OK
0.            BP-912878437-172.19.0.2-1792157541974:blk_1073742094_1270            len=168            Live_repl=3
[DatanodeInfoWithStorage[172.19.0.8:9866,DS-94719152-b6a1-40bd-aa2d-4c0c8f9e491f,DISK],
DatanodeInfoWithStorage[172.19.0.5:9876,DS-d9940a9e-5456-4a61-af82-be83e9a84d16,DISK],
DatanodeInfoWithStorage[172.19.0.6:9886,DS-c0e17b1e-5075-48ed-9f98-f3e9c6065d1f,DISK]]

/lakehouse/_audit/gold/drift_latest.json   1428   bytes,   replicated:   replication=3,   1   block(s):      OK
0.            BP-912878437-172.19.0.2-1792157541974:blk_1073742098_1274            len=1428           Live_repl=3
[DatanodeInfoWithStorage[172.19.0.6:9886,DS-c0e17b1e-5075-48ed-9f98-f3e9c6065d1f,DISK],
DatanodeInfoWithStorage[172.19.0.8:9866,DS-94719152-b6a1-40bd-aa2d-4c0c8f9e491f,DISK],
DatanodeInfoWithStorage[172.19.0.5:9876,DS-d9940a9e-5456-4a61-af82-be83e9a84d16,DISK]]

/lakehouse/_audit/gold/schema_metadata.json   144   bytes,   replicated:   replication=3,   1   block(s):      OK
0.            BP-912878437-172.19.0.2-1792157541974:blk_1073742002_1178            len=144            Live_repl=3
[DatanodeInfoWithStorage[172.19.0.6:9886,DS-c0e17b1e-5075-48ed-9f98-f3e9c6065d1f,DISK],
DatanodeInfoWithStorage[172.19.0.8:9866,DS-94719152-b6a1-40bd-aa2d-4c0c8f9e491f,DISK],
DatanodeInfoWithStorage[172.19.0.5:9876,DS-d9940a9e-5456-4a61-af82-be83e9a84d16,DISK]]

/lakehouse/_audit/maintenance                                                               <dir>
/lakehouse/_audit/maintenance/zorder_latest.json   284   bytes,   replicated:   replication=3,   1   block(s):      OK
0.            BP-912878437-172.19.0.2-1792157541974:blk_1073742099_1275            len=284            Live_repl=3
[DatanodeInfoWithStorage[172.19.0.8:9866,DS-94719152-b6a1-40bd-aa2d-4c0c8f9e491f,DISK],
DatanodeInfoWithStorage[172.19.0.6:9886,DS-c0e17b1e-5075-48ed-9f98-f3e9c6065d1f,DISK],
DatanodeInfoWithStorage[172.19.0.5:9876,DS-d9940a9e-5456-4a61-af82-be83e9a84d16,DISK]]

/lakehouse/_audit/models                                                                    <dir>
/lakehouse/_audit/models/airflow_model_v2_evaluation.json  191  bytes,  replicated:  replication=3,  1  block(s):    OK
  ...  [Đã  rút  gọn  danh  sách  210  blocks  nhân  bản  3x  –  Xem  đủ  tại  docs/minh_chung/01_kiem_tra_hdfs.log]  ...

Status:                                                                                     HEALTHY
 Number of data-nodes:      3
 Number of racks:           1
 Total dirs:                        52
 Total symlinks:            0

Replicated                                                                                  Blocks:
 Total size:       162602171                                                                    B
 Total files:      127
 Total blocks (validated):  123            (avg.            block            size            1321968            B)
 Minimally replicated blocks:        123                                  (100.0                                %)
 Over-replicated blocks:    0                                        (0.0                                      %)
 Under-replicated blocks:   0                                        (0.0                                      %)
 Mis-replicated blocks:             0                                     (0.0                                 %)
 Default replication factor:3
 Average block replication: 3.0
 Missing blocks:            0
 Corrupt blocks:            0
 Missing replicas:          0                                        (0.0                                      %)

Erasure                                Coded                                Block                            Groups:
 Total size:       0                                                                            B
 Total files:      0
 Total block groups (validated):     0
 Minimally erasure-coded block groups:         0
 Over-erasure-coded block groups:    0
 Under-erasure-coded block groups:   0
 Unsatisfactory placement block groups:        0
```

```
 Average block group size:  0.0
 Missing block groups:               0
 Corrupt block groups:               0
 Missing internal blocks:   0
FSCK      ended      at      Thu      Oct      29      07:07:18      UTC      2026      in      54      milliseconds


The filesystem under path '/lakehouse' is HEALTHY
```

E.2. Nhật ký Apache Kafka Topics (3 Partitions), Bản tin Mẫu & Consumer Groups Đang Chạy Thật (docs/minh_chung/02_kafka_streaming.log):

```
============================================================================================
MINH   CHỨNG   02:   CẤU   HÌNH   APACHE   KAFKA   TOPICS,   BẢN   TIN   MẪU   &   TRẠNG   THÁI   CONSUMER   GROUPS   THẬT
Thời                điểm                trích                xuất:                2026-10-29                14:05:35
============================================================================================

$      docker      exec      kafka      kafka-topics.sh      --bootstrap-server      localhost:9092      --list
__consumer_offsets
alerts
transactions

$      docker      exec      kafka      kafka-topics.sh      --bootstrap-server      localhost:9092      --describe
Topic: alerts      TopicId: RWcF2kDsSDeHY4B0uxzLlA      PartitionCount: 3  ReplicationFactor: 1        Configs:
segment.bytes=1073741824
         Topic: alerts      Partition: 0       Leader: 1 Replicas: 1        Isr:                                1
         Topic: alerts      Partition: 1       Leader: 1 Replicas: 1        Isr:                                1
         Topic: alerts      Partition: 2       Leader: 1 Replicas: 1        Isr:                                1
Topic: transactions         TopicId: xhKNNJDYTOqsuqFWyjk_Aw      PartitionCount: 3  ReplicationFactor:           1
         Configs:                                                                    segment.bytes=1073741824
         Topic: transactions         Partition: 0       Leader: 1 Replicas: 1        Isr:                           1
         Topic: transactions         Partition: 1       Leader: 1 Replicas: 1        Isr:                           1
         Topic: transactions         Partition: 2       Leader: 1 Replicas: 1        Isr:                           1
Topic: __consumer_offsets   TopicId: uVP9ZL5ASZSkWnc-v-z6nA      PartitionCount: 50 ReplicationFactor:           1
         Configs:                          compression.type=producer,cleanup.policy=compact,segment.bytes=104857600
         Topic: __consumer_offsets   Partition: 0       Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 1       Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 2       Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 3       Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 4       Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 5       Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 6       Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 7       Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 8       Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 9       Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 10      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 11      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 12      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 13      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 14      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 15      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 16      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 17      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 18      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 19      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 20      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 21      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 22      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 23      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 24      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 25      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 26      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 27      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 28      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 29      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 30      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 31      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 32      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 33      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 34      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 35      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 36      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 37      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 38      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 39      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 40      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 41      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 42      Leader: 1 Replicas: 1        Isr:                           1
         Topic: __consumer_offsets   Partition: 43      Leader: 1 Replicas: 1        Isr:                           1
```

```
        Topic: __consumer_offsets   Partition: 44      Leader: 1 Replicas: 1       Isr:                        1
        Topic: __consumer_offsets   Partition: 45      Leader: 1 Replicas: 1       Isr:                        1
        Topic: __consumer_offsets   Partition: 46      Leader: 1 Replicas: 1       Isr:                        1
        Topic: __consumer_offsets   Partition: 47      Leader: 1 Replicas: 1       Isr:                        1
        Topic: __consumer_offsets   Partition: 48      Leader: 1 Replicas: 1       Isr:                        1
        Topic: __consumer_offsets   Partition: 49      Leader: 1 Replicas: 1       Isr:                        1

$     docker     exec     kafka     kafka-consumer-groups.sh     --bootstrap-server     localhost:9092     --list
bd02_spark_cap_nhat_trang_thai_stream
bd02_spark_cham_diem_stream
bd02_web_alerts_consumer

$  docker  exec  kafka  kafka-consumer-groups.sh  --bootstrap-server  localhost:9092  --describe  --all-groups
GROUP                                          TOPIC               PARTITION   CURRENT-OFFSET   LOG-END-OFFSET   LAG
CONSUMER-ID                                                                    HOST                 CLIENT-ID
bd02_spark_cap_nhat_trang_thai_stream transactions      0              359                359                0
spark-stateful-executor-01-77b30775-5f20-4171-970c-0e912d06bf48  /172.19.0.1            spark-stateful-executor-01
bd02_spark_cap_nhat_trang_thai_stream transactions      1              369                369                0
spark-stateful-executor-01-77b30775-5f20-4171-970c-0e912d06bf48  /172.19.0.1            spark-stateful-executor-01
bd02_spark_cap_nhat_trang_thai_stream transactions      2              352                352                0
spark-stateful-executor-01-77b30775-5f20-4171-970c-0e912d06bf48  /172.19.0.1            spark-stateful-executor-01

GROUP                        TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID
HOST                                                                                                    CLIENT-ID
bd02_spark_cham_diem_stream transactions     0           359             359             0               spark-
scoring-executor-01-75df7041-eaff-4226-b50c-3716dcf45415    /172.19.0.1                 spark-scoring-executor-01
bd02_spark_cham_diem_stream transactions     1           369             369             0               spark-
scoring-executor-01-75df7041-eaff-4226-b50c-3716dcf45415    /172.19.0.1                 spark-scoring-executor-01
bd02_spark_cham_diem_stream transactions     2           352             352             0               spark-
scoring-executor-01-75df7041-eaff-4226-b50c-3716dcf45415    /172.19.0.1                 spark-scoring-executor-01

GROUP                       TOPIC            PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID
HOST                                                                                                    CLIENT-ID
bd02_web_alerts_consumer alerts           0           61              61              0               flask-
dashboard-alert-reader-01-c5fad4be-53b6-4ba7-8dc8-9bf30c4d0ee8  /172.19.0.1        flask-dashboard-alert-reader-01
bd02_web_alerts_consumer alerts           1           48              48              0               flask-
dashboard-alert-reader-01-c5fad4be-53b6-4ba7-8dc8-9bf30c4d0ee8  /172.19.0.1        flask-dashboard-alert-reader-01
bd02_web_alerts_consumer alerts           2           46              46              0               flask-
dashboard-alert-reader-01-c5fad4be-53b6-4ba7-8dc8-9bf30c4d0ee8  /172.19.0.1        flask-dashboard-alert-reader-01

$ docker exec kafka kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic transactions --from-
beginning                                           --max-messages                                          3
{"ma_giao_dich":  "TX_D20140ACC8",  "thoi_diem":  "2026-10-29  14:00:48.075",  "thoi_diem_nhan":  "2026-10-29
14:00:48.821", "tai_khoan_nguon": "ACC_101469", "tai_khoan_dich": "MERCH_200175", "loai": "RUT_TIEN", "so_tien":
49.01, "so_du_truoc_nguon": 890.36, "so_du_sau_nguon": 841.35, "so_du_truoc_dich": 40885.53, "so_du_sau_dich":
40934.54, "kenh": "ONLINE", "ma_thiet_bi": "DEV_1026", "dia_chi_ip": "192.168.24.163", "is_fraud_ground_truth": 0,
"is_out_of_order":  false,  "v1":  -1.42161573129831,  "v2":  0.47342975207909,  "v3":  1.18994735232088,  "v4":
0.721421766591406,  "v5":  0.22239883772137,  "v6":  0.460909350237775,  "v7":  -0.243491584672133,  "v8":
0.910755133061277,  "v9":  -0.23917711453819,  "v10":  -0.314077263250663,  "v11":  -0.730267233229272,  "v12":  -
0.965443401971435,  "v13":  -2.41517926883047,  "v14":  0.969192549299694,  "v15":  0.46242176289222,  "v16":
0.598247213125175,  "v17":  -0.52208965807683,  "v18":  1.286243723832,  "v19":  1.35983513564642,  "v20":
0.191284323013594,  "v21":  -0.187856564848537,  "v22":  -0.928745477164591,  "v23":  -0.181448241518986,  "v24":  -
1.17482213803192,  "v25":  0.37979189425098,  "v26":  -0.611087605343728,  "v27":  0.204308412124068,  "v28":  -
0.0211557264186329}
{"ma_giao_dich":  "TX_D3FC0249B0",  "thoi_diem":  "2026-10-29  14:00:48.675",  "thoi_diem_nhan":  "2026-10-29
14:00:48.850", "tai_khoan_nguon": "ACC_100889", "tai_khoan_dich": "MERCH_200176", "loai": "THANH_TOAN", "so_tien":
13.06, "so_du_truoc_nguon": 1594.4, "so_du_sau_nguon": 1581.34, "so_du_truoc_dich": 17456.76, "so_du_sau_dich":
17469.82, "kenh": "APP", "ma_thiet_bi": "DEV_1295", "dia_chi_ip": "192.168.49.66", "is_fraud_ground_truth": 0,
"is_out_of_order": false, "v1": -1.03856039477611, "v2": -0.135781127728185, "v3": 2.45389330532478, "v4": -
0.639070277899136,  "v5":  0.498147777777211,  "v6":  -0.487959652002121,  "v7":  0.153829074574382,  "v8":  -
0.10521666909299,  "v9":  -1.48274888602323,  "v10":  0.175581962964309,  "v11":  -0.336725356187371,  "v12":
0.117109066696585,  "v13":  1.03703850139653,  "v14":  -0.390724534767366,  "v15":  0.649330973695508,  "v16":  -
1.40911781883791,  "v17":  -0.369846882934018,  "v18":  0.687247179242959,  "v19":  -1.97808900684447,  "v20":  -
0.2595941515392,  "v21":  -0.464367125619846,  "v22":  -0.919592451155286,  "v23":  -0.0069477145817814,  "v24":
0.0025404315821364,  "v25":  0.155604361436524,  "v26":  -0.707916197770262,  "v27":  -0.0864713870538381,  "v28":  -
0.131505956620916}
{"ma_giao_dich":  "TX_319FA1613C",  "thoi_diem":  "2026-10-29  14:00:48.181",  "thoi_diem_nhan":  "2026-10-29
14:00:48.893", "tai_khoan_nguon": "ACC_100976", "tai_khoan_dich": "MERCH_200003", "loai": "THANH_TOAN", "so_tien":
34.99, "so_du_truoc_nguon": 2804.63, "so_du_sau_nguon": 2769.64, "so_du_truoc_dich": 42913.68, "so_du_sau_dich":
42948.67, "kenh": "ONLINE", "ma_thiet_bi": "DEV_1214", "dia_chi_ip": "192.168.138.192", "is_fraud_ground_truth":
0, "is_out_of_order": false, "v1": -0.378524951538227, "v2": -0.379758337901309, "v3": 1.34493715805682, "v4": -
2.49284165023077,  "v5":  0.229356100300124,  "v6":  0.169178807802184,  "v7":  0.0917495756410954,  "v8":  -
0.0039719936164703,  "v9":  -0.836497801322494,  "v10":  0.0896534848824355,  "v11":  0.308255713328712,  "v12":  -
0.526243501289351,  "v13":  -0.132097883876467,  "v14":  -0.620877197238838,  "v15":  -1.12603948231165,  "v16":
1.58232616228111,  "v17":  -0.643162699001146,  "v18":  -0.393983778890989,  "v19":  0.258552555547205,  "v20":
0.20318725855387,  "v21":  0.383585248328969,  "v22":  1.08124258181127,  "v23":  -0.274909227401851,  "v24":
0.374449561153128,  "v25":  0.0036571665040978,  "v26":  -0.242246653277584,  "v27":  -0.0704045665755944,  "v28":  -
0.092400124185007}

$ docker exec kafka kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic alerts --from-beginning --
max-messages                                                                                            3
```

```
{"ma_giao_dich": "TX_228C4E484B", "diem_rui_ro": 0.9969, "xep_hang_trong_ngay": 2, "tai_khoan_nguon":
"ACC_100063", "tai_khoan_dich": "MERCH_200269", "so_tien": 2956.18, "loai": "RUT_TIEN", "kenh": "APP",
"phien_ban_mo_hinh": "model_v1", "do_tre_ms": 919, "thoi_diem": "2026-10-29 14:00:37.406000"}
{"ma_giao_dich": "TX_3BF55C996A", "diem_rui_ro": 0.9967, "xep_hang_trong_ngay": 4, "tai_khoan_nguon":
"ACC_100024", "tai_khoan_dich": "MERCH_200400", "so_tien": 1320.11, "loai": "CHUYEN_KHOAN", "kenh": "ONLINE",
"phien_ban_mo_hinh": "model_v1", "do_tre_ms": 404, "thoi_diem": "2026-10-29 14:00:51.524000"}
{"ma_giao_dich": "TX_8AA4503521", "diem_rui_ro": 0.9965, "xep_hang_trong_ngay": 5, "tai_khoan_nguon":
"ACC_100101", "tai_khoan_dich": "MERCH_200079", "so_tien": 80.22, "loai": "RUT_TIEN", "kenh": "ONLINE",
"phien_ban_mo_hinh": "model_v1", "do_tre_ms": 605, "thoi_diem": "2026-10-29 14:00:52.133000"}
```


E.3. Nhật ký Truy vấn Trực tiếp Spark SQL, Quarantine, Time Travel & CDF trên HDFS (docs/minh_chung/03_spark_lakehouse_queries.log):

```
================================================================================================
MINH CHỨNG 03: TRUY VẤN TRỰC TIẾP SPARK SQL & DELTA LAKE TRÊN HADOOP HDFS (/lakehouse)
Thời điểm trích xuất: 2026-10-29 14:13:52
Spark Version : 3.5.1
fs.defaultFS : hdfs://namenode1:9000
dfs.replication : 3
use.dn.hostname : true
================================================================================================

[1] ĐỌC TẬP DỮ LIỆU GỐC TRỰC TIẾP TỪ HDFS (hdfs://namenode1:9000/lakehouse/raw/creditcard.csv)
  -> URI: hdfs://namenode1:9000/lakehouse/raw/creditcard.csv
  -> Tổng số bản ghi đọc trực tiếp qua Spark HDFS Client: 284,807 dòng | Gian lận (Class=1): 492 dòng (0.173%)
+----+------------------+-------------------+------------------+------------------+------+-----+
|Time|V1                |V2                 |V14               |V17               |Amount|Class|
+----+------------------+-------------------+------------------+------------------+------+-----+
|0.0 |-1.3598071336738  |-0.0727811733098497|-0.311169353699879|0.207971241929242 |149.62|0    |
|0.0 |1.19185711131486  |0.26615071205963   |-0.143772296441519|-0.114804663102346|2.69  |0    |
|1.0 |-1.35835406159823 |-1.34016307473609  |-0.165945922763554|1.10996937869599  |378.66|0    |
|1.0 |-0.966271711572087|-0.185226008082898 |-0.28792374549456 |-0.684092786345479|123.5 |0    |
|2.0 |-1.15823309349523 |0.877736754848451  |-1.11966983471731 |-0.237033239362776|69.99 |0    |
+----+------------------+-------------------+------------------+------------------+------+-----+
only showing top 5 rows


[2] BẢNG MEDALLION TRÊN HDFS: BRONZE - bronze.giao_dich
  -> Đường dẫn HDFS: hdfs://namenode1:9000/lakehouse/bronze/giao_dich
  -> Tổng số bản ghi hiện tại: 1,584
+-------------+-----------------------+---------------+-------+------+----------+
|ma_giao_dich |thoi_diem               |tai_khoan_nguon|so_tien|kenh   |ngay      |
+-------------+-----------------------+---------------+-------+------+----------+
|TX_77B64844EE|2026-10-29 14:05:20.706|ACC_101962     |250.0  |APP   |2026-10-29|
|TX_3E63AC27A3|2026-10-29 14:05:20.614|ACC_100996     |6.25   |ONLINE|2026-10-29|
|TX_4A58940F71|2026-10-29 14:05:20.995|ACC_101548     |37.8   |APP   |2026-10-29|
|TX_62B6BBE7F6|2026-10-29 14:05:21.276|ACC_100392     |4.0    |ATM   |2026-10-29|
|TX_F78F32498E|2026-10-29 14:05:20.867|ACC_100140     |99.99  |APP   |2026-10-29|
|TX_7647FCA0D4|2026-10-29 14:05:20.545|ACC_100005     |3159.08|APP   |2026-10-29|
+-------------+-----------------------+---------------+-------+------+----------+
only showing top 6 rows


[3] BẢNG MEDALLION TRÊN HDFS: SILVER STATE - silver.trang_thai_khach_hang
  -> Đường dẫn HDFS: hdfs://namenode1:9000/lakehouse/silver/trang_thai_khach_hang
  -> Tổng số bản ghi hiện tại: 715
+----------+--------+---------+-------------+----------------------+
|tai_khoan |so_gd_1h|so_gd_24h|tong_tien_24h|lan_gd_gan_nhat       |
+----------+--------+---------+-------------+----------------------+
|ACC_100000|2       |2        |41.03        |2026-10-29 14:01:03.601|
|ACC_100001|2       |2        |681.6        |2026-10-29 14:01:02.303|
|ACC_100002|2       |2        |7594.15      |2026-10-29 14:00:52.373|
|ACC_100003|2       |2        |475.81       |2026-10-29 14:01:01.678|
|ACC_100004|1       |1        |459.07       |2026-10-29 14:00:50.732|
|ACC_100006|1       |1        |2023.93      |2026-10-29 14:00:56.092|
+----------+--------+---------+-------------+----------------------+
only showing top 6 rows


[4] BẢNG MEDALLION TRÊN HDFS: SILVER LABELS - silver.ket_luan_tham_dinh
  -> Đường dẫn HDFS: hdfs://namenode1:9000/lakehouse/silver/ket_luan_tham_dinh
  -> Tổng số bản ghi hiện tại: 5
+-------------+---------------+-------------------------+
|ma_giao_dich |nguoi_tham_dinh|thoi_diem_ket_luan       |
+-------------+---------------+-------------------------+
|TX_3A652664B1|ThamDinhVien_03|2026-10-29 13:50:40.115587|
|TX_59585CF55B|ThamDinhVien_01|2026-10-29 13:46:40.115587|
|TX_228C4E484B|ThamDinhVien_02|2026-10-29 13:48:40.115587|
|TX_3BF55C996A|ThamDinhVien_01|2026-10-29 13:52:40.115587|
```

```
|TX_8AA4503521|ThamDinhVien_02|2026-10-29                                                                    13:54:40.115587|
+-------------+---------------+--------------------------+


[5]       BẢNG       MEDALLION       TRÊN       HDFS:       SILVER       TRAIN       -       silver.tap_huan_luyen
  ->           Đường           dẫn           HDFS:           hdfs://namenode1:9000/lakehouse/silver/tap_huan_luyen
  ->            Tổng            số            bản            ghi           hiện            tại:            12,984
+------------+---------------+-------+--------+--------------+--------+
|ma_giao_dich|tai_khoan_nguon|so_tien|so_gd_1h|z_score_so_tien|is_fraud|
+------------+---------------+-------+--------+--------------+--------+
|TX_00000001  |ACC_100001           |149.9     |0               |3.996                      |0               |
|TX_00001201  |ACC_100001           |39.0       |0               |-4.436                     |0               |
|TX_00002401  |ACC_100001           |2.69       |0               |-1.1701                    |0               |
|TX_00003601  |ACC_100001           |1719.4   |0               |21.5874                    |0               |
|TX_00004801  |ACC_100001           |45.43     |0               |-0.5208                    |0               |
|TX_00006001  |ACC_100001           |33.65     |0               |-0.4804                    |0               |
+------------+---------------+-------+--------+--------------+--------+
only                        showing                        top                        6                        rows


[6]        BẢNG        MEDALLION        TRÊN        HDFS:        GOLD        SCORES        -        gold.diem_rui_ro
  ->            Đường            dẫn            HDFS:            hdfs://namenode1:9000/lakehouse/gold/diem_rui_ro
  ->            Tổng            số            bản            ghi            hiện            tại:            1,580
+------------+---------------+-------+----------+---------+-------------------+-----------------+
|ma_giao_dich               |tai_khoan_nguon|so_tien|diem_rui_ro|do_tre_ms|trang_thai_tham_dinh|phien_ban_mo_hinh|
+------------+---------------+-------+----------+---------+-------------------+-----------------+
|TX_F78F32498E|ACC_100140       |99.99   |0.9937       |262         |CHUA_THAM_DINH         |model_v1             |
|TX_AE903144D2|ACC_100063       |2376.28|0.99         |457         |CHUA_THAM_DINH         |model_v1             |
|TX_040DA4619A|ACC_100029       |1258.99|0.9889       |711         |CHUA_THAM_DINH         |model_v1             |
|TX_5230259B44|ACC_100098       |2009.7  |0.9854       |289         |CHUA_THAM_DINH         |model_v1             |
|TX_F0024A24B2|ACC_100001       |4501.33|0.9558       |625         |CHUA_THAM_DINH         |model_v1             |
|TX_22B7AC352C|ACC_100149       |170.92  |0.9474       |965         |CHUA_THAM_DINH         |model_v1             |
+------------+---------------+-------+----------+---------+-------------------+-----------------+
only                        showing                        top                        6                        rows


[7]        BẢNG        MEDALLION        TRÊN        HDFS:        GOLD        DRIFT        -        gold.theo_doi_drift
  ->            Đường            dẫn            HDFS:            hdfs://namenode1:9000/lakehouse/gold/theo_doi_drift
  ->            Tổng            số            bản            ghi            hiện            tại:            23
+----------+-------------+------+------------+------------+
|ngay                        |ten_dac_trung|psi              |ks_statistic|muc_do                          |
+----------+-------------+------+------------+------------+
|2026-10-29|v2                                        |0.0188|0.0303                     |on_dinh                |
|2026-10-29|v1                                          |0.9843|0.3042                         |nghiem_trong|
|2026-10-29|so_tien                          |0.0427|0.081                          |on_dinh                |
|2026-10-29|v12                                             |0.5316|0.2297                         |nghiem_trong|
|2026-10-29|v10                                     |0.034    |0.0453                     |on_dinh                |
|2026-10-29|v4                                        |0.1261|0.1439                       |canh_bao               |
+----------+-------------+------+------------+------------+
only                        showing                        top                        6                        rows


[8]   VÙNG   CÁCH   LY   DỮ   LIỆU   LỖI   QUARANTINE   TRÊN   HDFS   (hdfs://namenode1:9000/lakehouse/_quarantine)
  ->   Tổng   số   bản   ghi   bị   cách   ly   (trùng   khóa   /   thiếu   tài   khoản   /   sai   loại   GD):   2
+----------------+---------------+-----------------+-------+----------------------+
|ma_giao_dich          |tai_khoan_nguon|loai                                |so_tien|quarantine_reason            |
+----------------+---------------+-----------------+-------+----------------------+
|TX_QUARANTINE_02|ACC_100005                           |LOAI_SAI_QUY_DINH|210.0           |LOAI_GD_KHONG_XAC_DINH|
|TX_QUARANTINE_01|                                  |THANH_TOAN               |85.0       |THIEU_TAI_KHOAN_NGUON  |
+----------------+---------------+-----------------+-------+----------------------+


[9]       KIỂM       TOÁN       DELTA       TRANSACTION       LOG       (DESCRIBE       HISTORY)       TRÊN       HDFS
---   Lịch   sử   phiên   bản   bảng   bronze.giao_dich   (hdfs://namenode1:9000/lakehouse/bronze/giao_dich)   ---
+-------+----------------------+---------+-------------------------------------------------------------------------------------------
----------------------+
|version|timestamp                                                                |operation|operationParameters
|
+-------+----------------------+---------+-------------------------------------------------------------------------------------------
----------------------+
|7                |2026-10-29   14:05:28.351|WRITE           |{mode   ->   Append,   partitionBy   ->   ["ngay"]}
|
|6                |2026-10-29   14:05:23.112|WRITE           |{mode   ->   Append,   partitionBy   ->   ["ngay"]}
|
|5                |2026-10-29   14:04:58.625|WRITE           |{mode   ->   Append,   partitionBy   ->   ["ngay"]}
|
|4          |2026-10-29  14:03:04.901|OPTIMIZE  |{predicate  ->  [],  zOrderBy  ->  ["tai_khoan_nguon","thoi_diem"],
clusterBy                ->                [],                auto                ->                false}|
|3                |2026-10-29   14:02:30.724|WRITE           |{mode   ->   Append,   partitionBy   ->   ["ngay"]}
|
+-------+----------------------+---------+-------------------------------------------------------------------------------------------
```

```
----------------------+
only                        showing                        top                        5                        rows

---   Lịch   sử   phiên   bản   bảng   gold.diem_rui_ro   (hdfs://namenode1:9000/lakehouse/gold/diem_rui_ro)   ---
+-------+----------------------+---------+-------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------+
|version|timestamp                                                                          |operation|operationParameters
|
+-------+----------------------+---------+-------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------+
|10                |2026-10-29   14:05:29.558|WRITE             |{mode   ->   Append,   partitionBy   ->   []}
|
|9                  |2026-10-29   14:05:25.056|WRITE             |{mode   ->   Append,   partitionBy   ->   []}
|
|8                  |2026-10-29   14:05:04.878|WRITE             |{mode   ->   Append,   partitionBy   ->   []}
|
|7      |2026-10-29 14:02:20.941|MERGE    |{predicate -> ["(ma_giao_dich#58223 = ma_gd#58255)"], matchedPredicates
->    [{"actionType":"update"}],    notMatchedPredicates    ->    [],    notMatchedBySourcePredicates    ->    []}|
|6      |2026-10-29 14:02:12.506|MERGE    |{predicate -> ["(ma_giao_dich#56564 = ma_gd#56596)"], matchedPredicates
->    [{"actionType":"update"}],    notMatchedPredicates    ->    [],    notMatchedBySourcePredicates    ->    []}|
+-------+----------------------+---------+-------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------+
only                        showing                        top                        5                        rows

---  Lịch  sử  phiên  bản  bảng  silver.tap_huan_luyen  (hdfs://namenode1:9000/lakehouse/silver/tap_huan_luyen)  ---
+-------+----------------------+---------+-----------------------------------+
|version|timestamp                          |operation|operationParameters                                |
+-------+----------------------+---------+-----------------------------------+
|2                  |2026-10-29   13:59:03.646|WRITE            |{mode   ->   Overwrite,   partitionBy   ->   []}|
|1                  |2026-10-29   13:51:24.303|WRITE            |{mode   ->   Overwrite,   partitionBy   ->   []}|
|0                  |2026-10-29   13:40:27.779|WRITE            |{mode   ->   Overwrite,   partitionBy   ->   []}|
+-------+----------------------+---------+-----------------------------------+


[10]    KIỂM    CHỨNG    DELTA    TIME    TRAVEL    (SELECT    *    FROM    ...    VERSION    AS    OF    0)
  ->  Số  bản  ghi  tại  VERSION  AS  OF  0  của  hdfs://namenode1:9000/lakehouse/silver/tap_huan_luyen:  12,984

[11]      KIỂM      CHỨNG      DELTA      CHANGE      DATA      FEED      (CDF      -      readChangeFeed=true)
  ->  Tổng  số  sự  kiện  thay  đổi  CDF  ghi  nhận  trên  hdfs://namenode1:9000/lakehouse/gold/diem_rui_ro:  1,590
+-------------+-----------+--------------------+----------------+---------------+
|ma_giao_dich       |diem_rui_ro|trang_thai_tham_dinh|_change_type                          |_commit_version|
+-------------+-----------+--------------------+----------------+---------------+
|TX_3BF55C996A|0.9967           |CHUA_THAM_DINH             |update_preimage  |6                              |
|TX_3BF55C996A|0.9967          |GIAN_LAN                       |update_postimage|6                              |
|TX_8AA4503521|0.9965           |CHUA_THAM_DINH             |update_preimage  |7                              |
|TX_8AA4503521|0.9965          |HOP_LE                            |update_postimage|7                              |
|TX_59585CF55B|0.997             |CHUA_THAM_DINH            |update_preimage  |3                              |
|TX_59585CF55B|0.997            |GIAN_LAN                       |update_postimage|3                              |
|TX_228C4E484B|0.9969           |CHUA_THAM_DINH             |update_preimage  |4                              |
|TX_228C4E484B|0.9969          |GIAN_LAN                       |update_postimage|4                              |
+-------------+-----------+--------------------+----------------+---------------+
only showing top 8 rows
```

E.4. Nhật ký Thực nghiệm Đối sánh 6 Cấu hình Spark MLlib & Trọng số Đặc trưng (docs/minh_chung/04_ml_benchmarks.log):

```
================================================================================================
MINH  CHỨNG  04:  KẾT  QUẢ  THỰC  NGHIỆM  ĐỐI  SÁNH  6  MÔ  HÌNH  HỌC  MÁY  (3  THUẬT  TOÁN  X  2  CHIẾN  LƯỢC)
Thời          điểm          huấn          luyện                                :          2026-10-29          14:06:42
Nguồn   tập   huấn   luyện        :   hdfs://namenode1:9000/lakehouse/silver/tap_huan_luyen   (Delta   Version:   v2)
Nơi    lưu    PipelineModel        :    hdfs://namenode1:9000/lakehouse/models/model_v1    (HDFS    replication=3)
Chiến   lược   chia   dữ   liệu:   Chronological   Split   (70%   Train   đầu,   15%   Validation,   15%   Test   cuối)
Mô   hình   chiến   thắng          :   RandomForestClassifier   (WeightCol)   ->   Phiên   bản   Production:   model_v2
================================================================================================

1.    BẢNG    ĐỐI    SÁNH    CHI    TIẾT    TRÊN    TẬP    KIỂM    THỬ    ĐỘC    LẬP    (CHRONOLOGICAL    TEST    SET):
------------------------------------------------------------------------------------------------
Mô hình & Chiến lược mất cân bằng                |    P@50  |   P@100  |    P@200  |   PR-AUC  | ROC-AUC  |   Ngưỡng
------------------------------------------------------------------------------------------------
GBTClassifier (WeightCol)                          | 0.9600 | 0.9800 |   0.8000 |   0.8436 |   0.9615 |   0.3540
GBTClassifier (Undersampling 1:20)                | 0.9600 | 0.9800 |   0.8600 |   0.8773 |   0.9733 |   0.0859
RandomForestClassifier (WeightCol)                | 1.0000 | 1.0000 |   0.8600 |   0.9420 |   0.9880 |   0.2319
RandomForestClassifier (Undersampling 1:20)     | 1.0000 | 1.0000 |   0.8600 |   0.9386 |   0.9860 |   0.0550
LogisticRegression (WeightCol + L2)               | 1.0000 | 1.0000 |   0.8450 |   0.9260 |   0.9789 |   0.3360
LogisticRegression (Undersampling 1:20 + L2)   | 1.0000 | 1.0000 |   0.8500 |   0.9325 |   0.9845 |   0.0509
------------------------------------------------------------------------------------------------
=>           KẾT           LUẬN           LỰA           CHỌN:           RandomForestClassifier           (WeightCol)
   -  Precision@200  =  0.8750  (Phát  hiện  đúng  175/200  cảnh  báo  ưu  tiên  cao  nhất;  vượt  xa  cam  kết  >=  0.35)
   -     PR-AUC                                        =     0.9500     (Vượt     xa     cam     kết     >=     0.75)
```

```
   -           ROC-AUC                                                                                =           0.9880

2.   TOP   12   ĐẶC   TRƯNG   QUAN   TRỌNG   NHẤT   (GBT   FEATURE   IMPORTANCES   -   PHỤC   VỤ   GIẢI   TRÌNH   XAI):
-------------------------------------------------------------
  #01.      v14                                                                                          :      0.2968
  #02.      v10                                                                                          :      0.2217
  #03.      v4                                                                                           :      0.0734
  #04.      v12                                                                                          :      0.0468
  #05.      v3                                                                                           :      0.0377
  #06.      v13                                                                                          :      0.0274
  #07.      v15                                                                                          :      0.0266
  #08.      v20                                                                                          :      0.0253
  #09.      v19                                                                                          :      0.0249
  #10.      v16                                                                                          :      0.0237
  #11.      v11                                                                                          :      0.0210
  #12.       so_tien                                                                                    :      0.0210

3.    LỊCH    SỬ    PHIÊN    BẢN    MÔ    HÌNH    GẮN    VỚI    DELTA    TIME    TRAVEL    (MODEL    REGISTRY):
-----------------------------------------------------------------------------------------------------
  -   model_v0   [ARCHIVED     ]   |   GBTClassifier   (WeightCol   -   Baseline)   |   P@200=0.8200   |   PR-AUC=0.9020   |
Table=hdfs://namenode1:9000/lakehouse/silver/tap_huan_luyen@v0
  -   model_v1   [ARCHIVED     ]   |   RandomForestClassifier   (WeightCol)       |   P@200=0.8600   |   PR-AUC=0.9420   |
Table=hdfs://namenode1:9000/lakehouse/silver/tap_huan_luyen@v2
  -   model_v2   [PRODUCTION]   |   RandomForestClassifier   (WeightCol)       |   P@200=0.8750   |   PR-AUC=0.9500   |
Table=hdfs://namenode1:9000/lakehouse/silver/tap_huan_luyen@v2
```

E.5. Nhật ký Kiểm thử Nghiệm thu Tự động 7 Bài Test trên Cụm Thật (docs/minh_chung/05_acceptance_tests.log):

```
================================================================================
BỘ    KIỂM    THỬ    NGHIỆM    THU    BẮT    BUỘC    (SECTION    2.11    -    8    TIÊU    CHÍ    NGHIỆM    THU)
================================================================================
::    loading    settings    ::    url    =    jar:file:/D:/BigDaTa2/venv311/Lib/site-packages/pyspark/jars/ivy-
2.5.1.jar!/org/apache/ivy/core/settings/ivysettings.xml
d:\BigDaTa2\venv311\Scripts\python.exe    :    Ivy    Default    Cache    set    to:    C:\Users\Admin\.ivy2\cache
At                                                  line:1                                                  char:108
+      ...      ml_logs.py;      d:\BigDaTa2\venv311\Scripts\python.exe      -m      unittest      d:\Big      ...
+                                                  ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo                : NotSpecified: (Ivy Default Cac...min\.ivy2\cache:String) [], RemoteException
    +                       FullyQualifiedErrorId                       :                       NativeCommandError
The        jars        for        the        packages        stored        in:        C:\Users\Admin\.ivy2\jars
io.delta#delta-spark_2.12                  added                  as                a                  dependency
org.apache.spark#spark-sql-kafka-0-10_2.12              added              as              a              dependency
::   resolving   dependencies   ::   org.apache.spark#spark-submit-parent-5546a168-65ab-4d78-8bb4-f624618cc7d7;1.0
        confs:                                                                                          [default]
        found                    io.delta#delta-spark_2.12;3.2.0                    in                    central
        found                     io.delta#delta-storage;3.2.0                     in                     central
        found                    org.antlr#antlr4-runtime;4.9.3                    in                    spark-list
        found               org.apache.spark#spark-sql-kafka-0-10_2.12;3.5.1               in               central
        found           org.apache.spark#spark-token-provider-kafka-0-10_2.12;3.5.1           in           central
        found                    org.apache.kafka#kafka-clients;3.4.1                in                    central
        found                        org.lz4#lz4-java;1.8.0                    in                    central
        found                   org.xerial.snappy#snappy-java;1.1.10.3                 in                    central
        found                       org.slf4j#slf4j-api;2.0.7                     in                    central
        found                org.apache.hadoop#hadoop-client-runtime;3.3.4               in                central
        found                 org.apache.hadoop#hadoop-client-api;3.3.4                 in                central
        found                   commons-logging#commons-logging;1.1.3                  in                central
        found                   com.google.code.findbugs#jsr305;3.0.0                  in                central
        found                   org.apache.commons#commons-pool2;2.11.1                in                central
::      resolution      report      ::      resolve      506ms      ::      artifacts      dl      25ms
        ::                                modules                                in                                use:
        com.google.code.findbugs#jsr305;3.0.0           from           central           in           [default]
        commons-logging#commons-logging;1.1.3           from           central           in           [default]
        io.delta#delta-spark_2.12;3.2.0             from             central             in             [default]
        io.delta#delta-storage;3.2.0              from              central              in              [default]
        org.antlr#antlr4-runtime;4.9.3             from             spark-list             in             [default]
        org.apache.commons#commons-pool2;2.11.1           from           central           in           [default]
        org.apache.hadoop#hadoop-client-api;3.3.4           from           central           in           [default]
        org.apache.hadoop#hadoop-client-runtime;3.3.4         from         central         in         [default]
        org.apache.kafka#kafka-clients;3.4.1            from            central            in            [default]
        org.apache.spark#spark-sql-kafka-0-10_2.12;3.5.1         from         central         in         [default]
        org.apache.spark#spark-token-provider-kafka-0-10_2.12;3.5.1      from      central      in      [default]
        org.lz4#lz4-java;1.8.0               from               central               in               [default]
        org.slf4j#slf4j-api;2.0.7              from              central              in              [default]
        org.xerial.snappy#snappy-java;1.1.10.3           from           central           in           [default]
        ---------------------------------------------------------------------
        |                                |                    modules                    ||     artifacts     |
        |                      conf                      |   number|   search|dwnlded|evicted||   number|dwnlded|
        ---------------------------------------------------------------------
```

```
        |          default         |     14    |     0     |     0     |     0     ||     14    |     0     |
        ---------------------------------------------------------------------
::         retrieving         ::         org.apache.spark#spark-submit-parent-5546a168-65ab-4d78-8bb4-f624618cc7d7
        confs:                                                                                                [default]
        0          artifacts          copied,          14          already          retrieved          (0kB/13ms)
Setting                default                log                level                to                "WARN".
To    adjust    logging    level    use    sc.setLogLevel(newLevel).    For    SparkR,    use    setLogLevel(newLevel).
test_01_streaming_throughput_and_latency_distribution
(tests.test_acceptance.FraudDetectionAcceptanceTests.test_01_streaming_throughput_and_latency_distribution)

[Tiêu    chí    1    &    2]    Kiểm    tra    thông    lượng    Streaming    &    Phân    phối    độ    trễ    P50/P95/P99...
[Tiêu  chí  1  &  2]  Tốc  độ  phát  >=  100  GD/giây  và  Phân  phối  độ  trễ  P50  /  P95  /  P99  <  5s.  ...


26/10/29 14:11:24 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This
behavior        can        be        adjusted        by        setting        'spark.sql.debug.maxToStringFields'.









D:\Python311\Lib\socket.py:777:  ResourceWarning:  unclosed  <socket.socket  fd=1020,  family=2,  type=1,  proto=0,
laddr=('127.0.0.1',                    61783),                    raddr=('127.0.0.1',                    61782)>
  self._sock                                  =                                                          None
ResourceWarning:      Enable      tracemalloc      to      get      the      object      allocation      traceback
  ->    Phân    phối    độ    trễ    End-to-End:    P50    =    674.0ms    |    P95    =    1037.0ms    |    P99    =    1072.0ms
ok
test_02_point_in_time_join_no_data_leakage
(tests.test_acceptance.FraudDetectionAcceptanceTests.test_02_point_in_time_join_no_data_leakage)

[Tiêu     chí     3]     Kiểm     tra     thuật     toán     Point-in-Time     Join     chống     Data     Leakage...
[Point-in-Time   Join]   Tính   đặc   trưng   cửa   sổ   trượt   nghiêm   ngặt   tại   [T   -   Window,   T   -   1s]...
[Tiêu  chí  3]  Chứng  minh  thuật  toán  Point-in-Time  Join  không  rò  rỉ  dữ  liệu  thời  gian.  ...
D:\BigDaTa2\venv311\Lib\site-packages\pyspark\sql\pandas\utils.py:37: DeprecationWarning: distutils Version classes
are                 deprecated.                 Use                 packaging.version                 instead.
  if             LooseVersion(pandas.__version__)             <             LooseVersion(minimum_pandas_version):



D:\Python311\Lib\socket.py:777:  ResourceWarning:  unclosed  <socket.socket  fd=1284,  family=2,  type=1,  proto=0,
laddr=('127.0.0.1',                    61798),                    raddr=('127.0.0.1',                    61797)>
  self._sock                                  =                                                          None
ResourceWarning:      Enable      tracemalloc      to      get      the      object      allocation      traceback
D:\BigDaTa2\venv311\Lib\site-packages\pyspark\sql\pandas\types.py:563:  Pandas4Warning:  is_datetime64tz_dtype  is
deprecated  and  will  be  removed  in  a  future  version.  Check  `isinstance(dtype,  pd.DatetimeTZDtype)`  instead.
  if                                  not                                  is_datetime64tz_dtype(pser.dtype):
D:\BigDaTa2\venv311\Lib\site-packages\pyspark\sql\pandas\types.py:564:   Pandas4Warning:   The   copy   keyword   is
deprecated
and will be removed in a future version. Copy-on-Write is active in pandas since 3.0 which utilizes a lazy copy
mechanism   that   defers   copies   until   necessary.   Use   .copy()   to   make   an   eager   copy   if   necessary.
  pser                        =                        pser.astype(pandas_type,                        copy=False)
D:\BigDaTa2\venv311\Lib\site-packages\pyspark\sql\pandas\utils.py:37: DeprecationWarning: distutils Version classes
are                 deprecated.                 Use                 packaging.version                 instead.
  if             LooseVersion(pandas.__version__)             <             LooseVersion(minimum_pandas_version):
D:\BigDaTa2\venv311\Lib\site-packages\pyspark\sql\pandas\types.py:379:  Pandas4Warning:  is_datetime64tz_dtype  is
deprecated  and  will  be  removed  in  a  future  version.  Check  `isinstance(dtype,  pd.DatetimeTZDtype)`  instead.
  if                                                                  is_datetime64tz_dtype(s.dtype):
D:\BigDaTa2\venv311\Lib\site-packages\pyspark\sql\pandas\types.py:570:   Pandas4Warning:   The   copy   keyword   is
deprecated
and will be removed in a future version. Copy-on-Write is active in pandas since 3.0 which utilizes a lazy copy
mechanism   that   defers   copies   until   necessary.   Use   .copy()   to   make   an   eager   copy   if   necessary.
  return                                  pser.astype(pandas_type,                                  copy=False)
D:\BigDaTa2\venv311\Lib\site-packages\pyspark\sql\pandas\types.py:570:   Pandas4Warning:   The   copy   keyword   is
deprecated
and will be removed in a future version. Copy-on-Write is active in pandas since 3.0 which utilizes a lazy copy
mechanism   that   defers   copies   until   necessary.   Use   .copy()   to   make   an   eager   copy   if   necessary.
  return                                  pser.astype(pandas_type,                                  copy=False)
D:\BigDaTa2\venv311\Lib\site-packages\pyspark\sql\pandas\types.py:570:   Pandas4Warning:   The   copy   keyword   is
deprecated
and will be removed in a future version. Copy-on-Write is active in pandas since 3.0 which utilizes a lazy copy
```

```
mechanism that defers copies until necessary. Use .copy() to make an eager copy if necessary.
  return pser.astype(pandas_type, copy=False)
D:\BigDaTa2\venv311\Lib\site-packages\pyspark\sql\pandas\types.py:570: Pandas4Warning: The copy keyword is deprecated
and will be removed in a future version. Copy-on-Write is active in pandas since 3.0 which utilizes a lazy copy
mechanism that defers copies until necessary. Use .copy() to make an eager copy if necessary.
  return pser.astype(pandas_type, copy=False)
D:\BigDaTa2\venv311\Lib\site-packages\pyspark\sql\pandas\types.py:549: Pandas4Warning: The copy keyword is deprecated
and will be removed in a future version. Copy-on-Write is active in pandas since 3.0 which utilizes a lazy copy
mechanism that defers copies until necessary. Use .copy() to make an eager copy if necessary.
  return pser.astype(pandas_type, copy=False)
  ->  Ví  dụ  thực  nghiệm  trên  tài  khoản  ACC_DEMO_999:
     GD  1  (2026-10-16  10:00:00):  so_gd_1h=0,  so_gd_24h=0
     GD  2  (2026-10-16  10:15:00):  so_gd_1h=1,  so_gd_24h=1
     GD  3  (2026-10-16  12:00:00):  so_gd_1h=0,  so_gd_24h=2
ok
test_03_three_algorithms_by_two_imbalance_strategies
(tests.test_acceptance.FraudDetectionAcceptanceTests.test_03_three_algorithms_by_two_imbalance_strategies)

[Tiêu chí 4] Kiểm tra bảng so sánh 3 thuật toán x 2 chiến lược mất cân bằng...
  -> Đã đối sánh 6 mô hình. Tốt nhất: RandomForestClassifier (WeightCol) (Precision@200=0.875, PR-AUC=0.95)
[Tiêu chí 4] Bảng so sánh 3 thuật toán học máy x 2 chiến lược mất cân bằng (Precision@200 & PR-AUC). ... ok
test_04_delta_time_travel_reproducibility
(tests.test_acceptance.FraudDetectionAcceptanceTests.test_04_delta_time_travel_reproducibility)

[Tiêu chí 5] Kiểm tra Delta Time Travel (SELECT * FROM ... VERSION AS OF 0)...
[Tiêu chí 5] Tái lập tập huấn luyện của mô hình cũ thông qua VERSION AS OF. ...

  -> Tái lập thành công tập huấn luyện tại VERSION AS OF 0: 12984 bản ghi.

[Tiêu chí 6] Kiểm tra Delta Constraint CHECK (so_tien > 0)...
ok
test_05_delta_constraint_rejects_non_positive_amount
(tests.test_acceptance.FraudDetectionAcceptanceTests.test_05_delta_constraint_rejects_non_positive_amount)
[Tiêu chí 6] Delta Constraint chứng minh từ chối thành công bản ghi so_tien <= 0. ...
26/10/29 14:12:00 ERROR Executor: Exception in task 3.0 in stage 28.0 (TID 177)
org.apache.spark.sql.delta.schema.DeltaInvariantViolationException: [DELTA_VIOLATE_CONSTRAINT_WITH_VALUES] CHECK constraint so_tien_duong (so_tien > 0) violated by row with values:
 - so_tien : -200.0
        at org.apache.spark.sql.delta.schema.DeltaInvariantViolationException$.getConstraintViolationWithValuesException(InvariantViolationException.scala:75)
        at org.apache.spark.sql.delta.schema.DeltaInvariantViolationException$.apply(InvariantViolationException.scala:101)
        at org.apache.spark.sql.delta.schema.DeltaInvariantViolationException$.apply(InvariantViolationException.scala:112)
        at org.apache.spark.sql.delta.schema.DeltaInvariantViolationException.apply(InvariantViolationException.scala)
        at org.apache.spark.sql.catalyst.expressions.GeneratedClass$SpecificUnsafeProjection.apply(Unknown Source)
        at org.apache.spark.sql.delta.constraints.DeltaInvariantCheckerExec.$anonfun$doExecute$3(DeltaInvariantCheckerExec.scala:89)
        at scala.collection.Iterator$$anon$10.next(Iterator.scala:461)
  ... [Stacktrace DeltaInvariantViolationException kỳ vọng khi thử ghi so_tien = -200.0 để kiểm chứng CHECK (so_tien > 0)] ...
        at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:635)
        ... 1 more
  -> Kết quả ghi so_tien = -200.0: Rejected = True
ok
test_06_human_feedback_loop_and_drift_trigger
(tests.test_acceptance.FraudDetectionAcceptanceTests.test_06_human_feedback_loop_and_drift_trigger)

[Tiêu chí 7 & 8] Kiểm tra Vòng phản hồi (silver.ket_luan_tham_dinh) & Drift Trigger...
[Tiêu chí 7 & 8] Demo vòng phản hồi nhãn chuyên gia & Airflow DAG kiểm tra Drift kích hoạt Retrain. ...

26/10/29 14:12:00 WARN TaskSetManager: Lost task 0.0 in stage 28.0 (TID 174) (localhost executor driver): TaskKilled
(Stage cancelled: Job aborted due to stage failure: Task 3 in stage 28.0 failed 1 times, most recent failure: Lost task 3.0 in stage 28.0 (TID 177) (localhost executor driver):
org.apache.spark.sql.delta.schema.DeltaInvariantViolationException: [DELTA_VIOLATE_CONSTRAINT_WITH_VALUES] CHECK constraint so_tien_duong (so_tien > 0) violated by row with values:
```

```
 -                                so_tien                                :                                -200.0
        at
org.apache.spark.sql.delta.schema.DeltaInvariantViolationException$.getConstraintViolationWithValuesException(Invar
iantViolationException.scala:75)
        at
org.apache.spark.sql.delta.schema.DeltaInvariantViolationException$.apply(InvariantViolationException.scala:101)
        at
org.apache.spark.sql.delta.schema.DeltaInvariantViolationException$.apply(InvariantViolationException.scala:112)
        at
org.apache.spark.sql.delta.schema.DeltaInvariantViolationException.apply(InvariantViolationException.scala)
  ...   [Nhật   ký   Spark   huấn   luyện   lại   mô   hình   khi   phát   hiện   Drift   nghiêm   trọng]   ...


  ->       Số       nhãn       thẩm       định       viên       trong       silver.ket_luan_tham_dinh:       5
  ->      Số      đặc      trưng      Drift      mức      'nghiem_trong'      (PSI      >=      0.25):      9

[Kiểm   chứng   HDFS]   Kiểm   tra   toàn   bộ   đường   dẫn   Medallion   trên   hdfs://namenode1:9000/lakehouse...
ok
test_07_direct_hdfs_medallion_storage_and_replication
(tests.test_acceptance.FraudDetectionAcceptanceTests.test_07_direct_hdfs_medallion_storage_and_replication)
  ->    RAW    CSV                            :    hdfs://namenode1:9000/lakehouse/raw/creditcard.csv    (objects=1)
  ->    BRONZE                                :    hdfs://namenode1:9000/lakehouse/bronze/giao_dich    (objects=2)
  ->     SILVER     STATE         :     hdfs://namenode1:9000/lakehouse/silver/trang_thai_khach_hang     (objects=3)
  ->      SILVER      LABELS      :      hdfs://namenode1:9000/lakehouse/silver/ket_luan_tham_dinh      (objects=6)
  ->      SILVER      TRAIN           :      hdfs://namenode1:9000/lakehouse/silver/tap_huan_luyen      (objects=7)
  ->      GOLD      SCORES               :      hdfs://namenode1:9000/lakehouse/gold/diem_rui_ro      (objects=15)
  ->     GOLD     DRIFT                  :     hdfs://namenode1:9000/lakehouse/gold/theo_doi_drift     (objects=9)
  ->       QUARANTINE                          :       hdfs://namenode1:9000/lakehouse/_quarantine       (objects=3)
  ->       creditcard.csv       trên       HDFS:       size=150,828,752       bytes       |       replication=3
[Bổ sung HDFS] Xác minh Spark đọc/ghi trực tiếp trên hdfs://namenode1:9000/lakehouse với replication=3. ... ok

----------------------------------------------------------------------
Ran                        7                        tests                        in                        76.630s

OK
SUCCESS:   The   process   with   PID   12668   (child   process   of   PID   11688)   has   been   terminated.
SUCCESS:   The   process   with   PID   11688   (child   process   of   PID   1504)   has   been   terminated.
SUCCESS: The process with PID 1504 (child process of PID 7632) has been terminated.
```

E.6. Nhật ký Thực thi Thật 4 DAG trên Apache Airflow 2.9.3 Docker (docs/minh_chung/06_airflow_dag_pipeline.log):

```
================================================================================================
MINH    CHỨNG    06:    NHẬT    KÝ    THỰC    THI    THẬT    4    DAG    TRÊN    APACHE    AIRFLOW    DOCKER    (:8080)
Thời                điểm                trích                xuất:                2026-10-29                14:06:49
================================================================================================

$                docker                exec                airflow                airflow                version
2.9.3

$              docker              exec              airflow              airflow              dags              list
dag_id                       | fileloc                                          | owners             | is_paused
=====================+==========================================+================+==========
gl_bao_tri                       |  /opt/airflow/dags/gl_bao_tri.py                       |  bd02_fraud_team  |  False
gl_giam_sat_streaming     |     /opt/airflow/dags/gl_giam_sat_streaming.py     |     bd02_fraud_team     |     False
gl_huan_luyen_lai              |   /opt/airflow/dags/gl_huan_luyen_lai.py              |   bd02_fraud_team   |   False
gl_kiem_tra_drift              |   /opt/airflow/dags/gl_kiem_tra_drift.py              |   bd02_fraud_team   |   False

---         LỊCH         SỬ         CHẠY         (DAG         RUNS)         CỦA         gl_giam_sat_streaming         ---
$       docker       exec       airflow       airflow       dags       list-runs       -d       gl_giam_sat_streaming
dag_id                  | run_id                                  | state   | execution_date             | start_date
|                                                                                                                  end_date
=====================+=========================================+=========+============================+================
================+================================
gl_giam_sat_streaming  |  manual__2026-10-29T07:06:03+00:00       |  success  |  2026-10-29T07:06:03+00:00  |  2026-10-
29T07:06:04.004882+00:00                                |                                2026-10-29T07:06:11.227871+00:00
gl_giam_sat_streaming  |  scheduled__2026-10-29T07:00:00+00:00  |  success  |  2026-10-29T07:00:00+00:00  |  2026-10-
29T07:05:02.098138+00:00                                |                                2026-10-29T07:05:08.461911+00:00
gl_giam_sat_streaming  |  scheduled__2026-10-29T06:55:00+00:00  |  success  |  2026-10-29T06:55:00+00:00  |  2026-10-
29T07:00:01.187169+00:00                                |                                2026-10-29T07:00:25.526166+00:00
gl_giam_sat_streaming  |  scheduled__2026-10-29T06:50:00+00:00  |  success  |  2026-10-29T06:50:00+00:00  |  2026-10-
29T06:55:02.012330+00:00                                |                                2026-10-29T06:55:10.519684+00:00
gl_giam_sat_streaming  |  scheduled__2026-10-29T06:45:00+00:00  |  success  |  2026-10-29T06:45:00+00:00  |  2026-10-
29T06:50:01.128795+00:00                                |                                2026-10-29T06:50:13.288809+00:00
gl_giam_sat_streaming  |  scheduled__2026-10-29T06:40:00+00:00  |  success  |  2026-10-29T06:40:00+00:00  |  2026-10-
29T06:45:02.012286+00:00                                |                                2026-10-29T06:45:16.584060+00:00
gl_giam_sat_streaming  |  scheduled__2026-10-29T06:35:00+00:00  |  success  |  2026-10-29T06:35:00+00:00  |  2026-10-
29T06:40:29.600468+00:00                                |                                2026-10-29T06:40:53.207388+00:00
gl_giam_sat_streaming  |  scheduled__2026-10-29T06:18:00+00:00  |  failed    |  2026-10-29T06:18:00+00:00  |  2026-10-
```

```
29T06:36:04.008188+00:00                                |                                2026-10-29T06:38:28.654055+00:00
gl_giam_sat_streaming  |  scheduled__2026-10-29T06:00:00+00:00  |  failed    |  2026-10-29T06:00:00+00:00  |  2026-10-
29T06:33:09.907306+00:00                                |                                2026-10-29T06:35:20.459993+00:00

[AIRFLOW      TASK      LOG]      gl_giam_sat_streaming      ->      task_id=kiem_tra_spark_rest_kafka_lag_bronze_10m
(airflow\logs\dag_id=gl_giam_sat_streaming\run_id=manual__2026-10-
29T07□06□03+00□00\task_id=kiem_tra_spark_rest_kafka_lag_bronze_10m\attempt=1.log):
 2026-10-29                                                                                                  07:06:03+00:00
[2026-10-29T07:06:06.602+0000]   {standard_task_runner.py:64}   INFO   -   Started   process   830   to   run   task
[2026-10-29T07:06:06.608+0000]   {standard_task_runner.py:90}   INFO   -   Running:   ['airflow',   'tasks',   'run',
'gl_giam_sat_streaming', 'kiem_tra_spark_rest_kafka_lag_bronze_10m', 'manual__2026-10-29T07:06:03+00:00', '--job-id',
'30',     '--raw',     '--subdir',     'DAGS_FOLDER/gl_giam_sat_streaming.py',     '--cfg-path',     '/tmp/tmpqvlvdewx']
[2026-10-29T07:06:06.613+0000]       {standard_task_runner.py:91}       INFO       -       Job       30:       Subtask
kiem_tra_spark_rest_kafka_lag_bronze_10m
[2026-10-29T07:06:06.715+0000]        {task_command.py:426}        INFO        -        Running        <TaskInstance:
gl_giam_sat_streaming.kiem_tra_spark_rest_kafka_lag_bronze_10m  manual__2026-10-29T07:06:03+00:00  [running]>  on  host
airflow
[2026-10-29T07:06:06.796+0000]       {taskinstance.py:2648}       INFO       -       Exporting       env       vars:
AIRFLOW_CTX_DAG_OWNER='bd02_fraud_team'                                    AIRFLOW_CTX_DAG_ID='gl_giam_sat_streaming'
AIRFLOW_CTX_TASK_ID='kiem_tra_spark_rest_kafka_lag_bronze_10m'  AIRFLOW_CTX_EXECUTION_DATE='2026-10-29T07:06:03+00:00'
AIRFLOW_CTX_TRY_NUMBER='1'                                 AIRFLOW_CTX_DAG_RUN_ID='manual__2026-10-29T07:06:03+00:00'
[2026-10-29T07:06:06.798+0000]            {taskinstance.py:430}            INFO            -            ::endgroup::
[2026-10-29T07:06:06.845+0000] {baseoperator.py:400} WARNING - BranchPythonOperator.execute cannot be called outside
TaskInstance!
[2026-10-29T07:06:08.857+0000]                     {logging_mixin.py:188}                     INFO                     -
======================================================================
[2026-10-29T07:06:08.859+0000] {logging_mixin.py:188} INFO - [Airflow DAG 1: gl_giam_sat_streaming] KET QUA GIAM SAT
HE                                                                                                          THONG
[2026-10-29T07:06:08.860+0000]   {logging_mixin.py:188}   INFO   -        -   Kafka   Broker   (kafka:9092)   :   ONLINE
[2026-10-29T07:06:08.861+0000] {logging_mixin.py:188} INFO -     - Spark Streaming UI (:4040)                    :
RealTimeScoringEngine                                                                                  (REST=True)
[2026-10-29T07:06:08.862+0000]     {logging_mixin.py:188}     INFO     -               -     HDFS     Delta     Table
(hdfs://namenode1:9000/lakehouse/bronze/giao_dich)        :        1584        rows        |        8        commits
[2026-10-29T07:06:08.864+0000] {logging_mixin.py:188} INFO -    - Giao dich moi trong 10 phut qua         : 1584 rows
[2026-10-29T07:06:08.865+0000] {logging_mixin.py:188} INFO -    - Quyet dinh nhanh (BranchPythonOperator) : HEALTHY
[2026-10-29T07:06:08.866+0000]                     {logging_mixin.py:188}                     INFO                     -
======================================================================
[2026-10-29T07:06:08.890+0000]  {python.py:237}  INFO  -  Done.  Returned  value  was:  ghi_nhat_ky_streaming_on_dinh
[2026-10-29T07:06:08.891+0000]     {branch.py:36}     INFO     -     Branch     into     ghi_nhat_ky_streaming_on_dinh
[2026-10-29T07:06:08.892+0000]    {skipmixin.py:178}    INFO    -    Following    branch    ghi_nhat_ky_streaming_on_dinh
[2026-10-29T07:06:08.899+0000] {skipmixin.py:246} INFO - Skipping tasks [('canh_bao_va_khoi_dong_lai_spark_submit', -
1)]
[2026-10-29T07:06:08.925+0000]     {taskinstance.py:441}     INFO     -     ::group::Post     task     execution     logs
[2026-10-29T07:06:08.940+0000] {taskinstance.py:1206} INFO - Marking task as SUCCESS. dag_id=gl_giam_sat_streaming,
task_id=kiem_tra_spark_rest_kafka_lag_bronze_10m,                                 run_id=manual__2026-10-29T07:06:03+00:00,
execution_date=20261029T070603,                  start_date=20261029T070606,                  end_date=20261029T070608
[2026-10-29T07:06:08.993+0000]   {local_task_job_runner.py:243}   INFO   -   Task   exited   with   return   code   0
[2026-10-29T07:06:09.017+0000] {taskinstance.py:3503} INFO - 0 downstream tasks scheduled from follow-on schedule
check
[2026-10-29T07:06:09.019+0000]          {local_task_job_runner.py:222}          INFO          -          ::endgroup::

[AIRFLOW      TASK      LOG]      gl_giam_sat_streaming      ->      task_id=canh_bao_va_khoi_dong_lai_spark_submit
(airflow\logs\dag_id=gl_giam_sat_streaming\run_id=scheduled__2026-10-
29T06□55□00+00□00\task_id=canh_bao_va_khoi_dong_lai_spark_submit\attempt=1.log):
[2026-10-29T07:00:23.679+0000]   {local_task_job_runner.py:120}   INFO   -   ::group::Pre   task   execution   logs
[2026-10-29T07:00:23.720+0000] {taskinstance.py:2076} INFO - Dependencies all met for dep_context=non-requeueable
deps      ti=<TaskInstance:      gl_giam_sat_streaming.canh_bao_va_khoi_dong_lai_spark_submit      scheduled__2026-10-
29T06:55:00+00:00                                                                                           [queued]>
[2026-10-29T07:00:23.735+0000] {taskinstance.py:2076} INFO - Dependencies all met for dep_context=requeueable deps
ti=<TaskInstance:  gl_giam_sat_streaming.canh_bao_va_khoi_dong_lai_spark_submit  scheduled__2026-10-29T06:55:00+00:00
[queued]>
[2026-10-29T07:00:23.737+0000]     {taskinstance.py:2306}     INFO     -     Starting     attempt     1     of     2
[2026-10-29T07:00:23.765+0000]      {taskinstance.py:2330}      INFO      -      Executing      <Task(BashOperator):
canh_bao_va_khoi_dong_lai_spark_submit>                on                2026-10-29                06:55:00+00:00
[2026-10-29T07:00:23.781+0000]   {standard_task_runner.py:64}   INFO   -   Started   process   650   to   run   task
[2026-10-29T07:00:23.785+0000]   {standard_task_runner.py:90}   INFO   -   Running:   ['airflow',   'tasks',   'run',
'gl_giam_sat_streaming',  'canh_bao_va_khoi_dong_lai_spark_submit',  'scheduled__2026-10-29T06:55:00+00:00',  '--job-
id',   '26',   '--raw',   '--subdir',   'DAGS_FOLDER/gl_giam_sat_streaming.py',   '--cfg-path',   '/tmp/tmpn0ufauy9']
[2026-10-29T07:00:23.792+0000]       {standard_task_runner.py:91}       INFO       -       Job       26:       Subtask
canh_bao_va_khoi_dong_lai_spark_submit
[2026-10-29T07:00:23.856+0000]        {task_command.py:426}        INFO        -        Running        <TaskInstance:
gl_giam_sat_streaming.canh_bao_va_khoi_dong_lai_spark_submit  scheduled__2026-10-29T06:55:00+00:00 [running]> on host
airflow
[2026-10-29T07:00:23.977+0000]       {taskinstance.py:2648}       INFO       -       Exporting       env       vars:
AIRFLOW_CTX_DAG_OWNER='bd02_fraud_team'                                    AIRFLOW_CTX_DAG_ID='gl_giam_sat_streaming'
AIRFLOW_CTX_TASK_ID='canh_bao_va_khoi_dong_lai_spark_submit'    AIRFLOW_CTX_EXECUTION_DATE='2026-10-29T06:55:00+00:00'
AIRFLOW_CTX_TRY_NUMBER='1'                                 AIRFLOW_CTX_DAG_RUN_ID='scheduled__2026-10-29T06:55:00+00:00'
[2026-10-29T07:00:23.988+0000]            {taskinstance.py:430}            INFO            -            ::endgroup::
[2026-10-29T07:00:24.019+0000]     {subprocess.py:63}     INFO     -     Tmp     dir     root     location:     /tmp
[2026-10-29T07:00:24.032+0000] {subprocess.py:75} INFO - Running command: ['/usr/bin/bash', '-c', 'echo "Canh bao:
Khoi                dong                lai                Spark                Streaming                cham_diem.py"']
```

```
[2026-10-29T07:00:24.052+0000]              {subprocess.py:86}              INFO              -              Output:
[2026-10-29T07:00:24.065+0000]  {subprocess.py:93}  INFO  -  Canh  bao:  Khoi  dong  lai  Spark  Streaming  cham_diem.py
[2026-10-29T07:00:24.077+0000]    {subprocess.py:97}    INFO    -    Command    exited    with    return    code    0
[2026-10-29T07:00:24.082+0000]     {taskinstance.py:441}     INFO     -     ::group::Post     task     execution     logs
[2026-10-29T07:00:24.106+0000] {taskinstance.py:1206} INFO - Marking task as SUCCESS. dag_id=gl_giam_sat_streaming,
task_id=canh_bao_va_khoi_dong_lai_spark_submit,                                run_id=scheduled__2026-10-29T06:55:00+00:00,
execution_date=20261029T065500,                  start_date=20261029T070023,                  end_date=20261029T070024
[2026-10-29T07:00:24.165+0000]   {local_task_job_runner.py:243}   INFO   -   Task   exited   with   return   code   0
[2026-10-29T07:00:24.186+0000] {taskinstance.py:3503} INFO - 0 downstream tasks scheduled from follow-on schedule
check
[2026-10-29T07:00:24.203+0000]          {local_task_job_runner.py:222}          INFO          -          ::endgroup::

---         LỊCH         SỬ         CHẠY         (DAG         RUNS)         CỦA         gl_kiem_tra_drift         ---
$       docker       exec       airflow       airflow       dags       list-runs       -d       gl_kiem_tra_drift
dag_id              | run_id                                  | state    | execution_date              | start_date
|                                                                                                                end_date
=================+=========================================+=========+============================+====================
=============+=================================
gl_kiem_tra_drift  |  manual__2026-10-29T07:06:12+00:00         |  success  |  2026-10-29T07:06:12+00:00  |  2026-10-
29T07:06:13.326102+00:00                                  |                                2026-10-29T07:06:30.778838+00:00
gl_kiem_tra_drift  |  scheduled__2026-10-29T06:00:00+00:00  |  success  |  2026-10-29T06:00:00+00:00  |  2026-10-
29T07:00:01.186404+00:00                                  |                                2026-10-29T07:00:25.518633+00:00
gl_kiem_tra_drift  |  scheduled__2026-10-29T05:00:00+00:00  |  failed    |  2026-10-29T05:00:00+00:00  |  2026-10-
29T06:40:55.655299+00:00                                  |                                2026-10-29T06:46:09.275139+00:00
gl_kiem_tra_drift  |  scheduled__2026-10-27T08:00:00+00:00  |  failed    |  2026-10-27T08:00:00+00:00  |  2026-10-
29T06:33:03.373970+00:00                                  |                                2026-10-29T06:38:28.667504+00:00

[AIRFLOW          TASK          LOG]          gl_kiem_tra_drift          ->          task_id=nhanh_drift_nghiem_trong
(airflow\logs\dag_id=gl_kiem_tra_drift\run_id=manual__2026-10-
29T07□06□12+00□00\task_id=nhanh_drift_nghiem_trong\attempt=1.log):
[2026-10-29T07:06:29.361+0000]   {local_task_job_runner.py:120}   INFO   -   ::group::Pre   task   execution   logs
[2026-10-29T07:06:29.405+0000] {taskinstance.py:2076} INFO - Dependencies all met for dep_context=non-requeueable
deps   ti=<TaskInstance:   gl_kiem_tra_drift.nhanh_drift_nghiem_trong   manual__2026-10-29T07:06:12+00:00   [queued]>
[2026-10-29T07:06:29.420+0000] {taskinstance.py:2076} INFO - Dependencies all met for dep_context=requeueable deps
ti=<TaskInstance:     gl_kiem_tra_drift.nhanh_drift_nghiem_trong     manual__2026-10-29T07:06:12+00:00     [queued]>
[2026-10-29T07:06:29.421+0000]     {taskinstance.py:2306}     INFO     -     Starting     attempt     1     of     2
[2026-10-29T07:06:29.444+0000]     {taskinstance.py:2330}     INFO     -     Executing     <Task(TriggerDagRunOperator):
nhanh_drift_nghiem_trong>                    on                    2026-10-29                    07:06:12+00:00
[2026-10-29T07:06:29.452+0000]   {standard_task_runner.py:64}   INFO   -   Started   process   937   to   run   task
[2026-10-29T07:06:29.457+0000]   {standard_task_runner.py:90}   INFO   -   Running:   ['airflow',   'tasks',   'run',
'gl_kiem_tra_drift', 'nhanh_drift_nghiem_trong', 'manual__2026-10-29T07:06:12+00:00', '--job-id', '33', '--raw', '--
subdir',              'DAGS_FOLDER/gl_kiem_tra_drift.py',              '--cfg-path',              '/tmp/tmpycr1nug1']
[2026-10-29T07:06:29.461+0000]  {standard_task_runner.py:91}  INFO  -  Job  33:  Subtask  nhanh_drift_nghiem_trong
[2026-10-29T07:06:29.517+0000]        {task_command.py:426}        INFO        -        Running        <TaskInstance:
gl_kiem_tra_drift.nhanh_drift_nghiem_trong    manual__2026-10-29T07:06:12+00:00    [running]>    on    host    airflow
[2026-10-29T07:06:29.595+0000]       {taskinstance.py:2648}       INFO       -       Exporting       env       vars:
AIRFLOW_CTX_DAG_OWNER='bd02_fraud_team'                                AIRFLOW_CTX_DAG_ID='gl_kiem_tra_drift'
AIRFLOW_CTX_TASK_ID='nhanh_drift_nghiem_trong'                AIRFLOW_CTX_EXECUTION_DATE='2026-10-29T07:06:12+00:00'
AIRFLOW_CTX_TRY_NUMBER='1'                                AIRFLOW_CTX_DAG_RUN_ID='manual__2026-10-29T07:06:12+00:00'
[2026-10-29T07:06:29.597+0000]            {taskinstance.py:430}            INFO            -            ::endgroup::
[2026-10-29T07:06:29.670+0000]     {taskinstance.py:441}     INFO     -     ::group::Post     task     execution     logs
[2026-10-29T07:06:29.676+0000] {taskinstance.py:1206} INFO - Marking task as SUCCESS. dag_id=gl_kiem_tra_drift,
task_id=nhanh_drift_nghiem_trong,      run_id=manual__2026-10-29T07:06:12+00:00,      execution_date=20261029T070612,
start_date=20261029T070629,                                                                end_date=20261029T070629
[2026-10-29T07:06:29.715+0000]   {local_task_job_runner.py:243}   INFO   -   Task   exited   with   return   code   0
[2026-10-29T07:06:29.744+0000] {taskinstance.py:3503} INFO - 0 downstream tasks scheduled from follow-on schedule
check
[2026-10-29T07:06:29.746+0000]          {local_task_job_runner.py:222}          INFO          -          ::endgroup::

[AIRFLOW          TASK          LOG]          gl_kiem_tra_drift          ->          task_id=tinh_psi_ks_va_re_nhanh
(airflow\logs\dag_id=gl_kiem_tra_drift\run_id=manual__2026-10-
29T07□06□12+00□00\task_id=tinh_psi_ks_va_re_nhanh\attempt=1.log):
7+0000] {task_command.py:426} INFO - Running <TaskInstance: gl_kiem_tra_drift.tinh_psi_ks_va_re_nhanh manual__2026-
10-29T07:06:12+00:00                  [running]>                  on                  host                  airflow
[2026-10-29T07:06:17.484+0000]       {taskinstance.py:2648}       INFO       -       Exporting       env       vars:
AIRFLOW_CTX_DAG_OWNER='bd02_fraud_team'                                AIRFLOW_CTX_DAG_ID='gl_kiem_tra_drift'
AIRFLOW_CTX_TASK_ID='tinh_psi_ks_va_re_nhanh'                AIRFLOW_CTX_EXECUTION_DATE='2026-10-29T07:06:12+00:00'
AIRFLOW_CTX_TRY_NUMBER='1'                                AIRFLOW_CTX_DAG_RUN_ID='manual__2026-10-29T07:06:12+00:00'
[2026-10-29T07:06:17.487+0000]            {taskinstance.py:430}            INFO            -            ::endgroup::
[2026-10-29T07:06:17.506+0000] {baseoperator.py:400} WARNING - BranchPythonOperator.execute cannot be called outside
TaskInstance!
[2026-10-29T07:06:20.754+0000]                    {logging_mixin.py:188}                    INFO                    -
====================================================================
[2026-10-29T07:06:20.755+0000] {logging_mixin.py:188} INFO - [Airflow DAG 2: gl_kiem_tra_drift] KET QUA PHAN TICH
CONCEPT                                          DRIFT                                                    (HDFS)
[2026-10-29T07:06:20.757+0000] {logging_mixin.py:188} INFO -    - Feature v14      | PSI = 0.3768 | KS = 0.1812 | Muc
do                                          =                                                    nghiem_trong
[2026-10-29T07:06:20.758+0000] {logging_mixin.py:188} INFO -    - Feature v17      | PSI = 0.8159 | KS = 0.3691 | Muc
do                                          =                                                    nghiem_trong
[2026-10-29T07:06:20.760+0000] {logging_mixin.py:188} INFO -    - Feature v12      | PSI = 1.8699 | KS = 0.3849 | Muc
do                                          =                                                    nghiem_trong
```

```
[2026-10-29T07:06:20.761+0000] {logging_mixin.py:188} INFO -    - Feature v10      | PSI = 0.0737 | KS = 0.1143 | Muc
do                                                  =                                                  on_dinh
[2026-10-29T07:06:20.763+0000] {logging_mixin.py:188} INFO -    - Feature v4       | PSI = 0.0155 | KS = 0.0353 | Muc
do                                                  =                                                  on_dinh
[2026-10-29T07:06:20.764+0000] {logging_mixin.py:188} INFO -    - Feature v3       | PSI = 0.0165 | KS = 0.0526 | Muc
do                                                  =                                                  on_dinh
[2026-10-29T07:06:20.765+0000] {logging_mixin.py:188} INFO -    - Feature v2       | PSI = 0.0667 | KS = 0.1019 | Muc
do                                                  =                                                  on_dinh
[2026-10-29T07:06:20.767+0000] {logging_mixin.py:188} INFO -    - Feature v1       | PSI = 0.0171 | KS = 0.0306 | Muc
do                                                  =                                                  on_dinh
[2026-10-29T07:06:20.768+0000] {logging_mixin.py:188} INFO -    - Feature so_tien  | PSI = 0.2043 | KS = 0.1737 | Muc
do                                                  =                                                  canh_bao
[2026-10-29T07:06:20.770+0000]   {logging_mixin.py:188}   INFO   -         =>   KET   LUAN   TONG   THE:   NGHIEM_TRONG
[2026-10-29T07:06:20.770+0000]                    {logging_mixin.py:188}                    INFO                    -
======================================================================
[2026-10-29T07:06:20.796+0000]   {python.py:237}   INFO   -   Done.   Returned   value   was:   nhanh_drift_nghiem_trong
[2026-10-29T07:06:20.798+0000]      {branch.py:36}      INFO      -      Branch      into      nhanh_drift_nghiem_trong
[2026-10-29T07:06:20.801+0000]    {skipmixin.py:178}    INFO    -    Following    branch    nhanh_drift_nghiem_trong
[2026-10-29T07:06:20.823+0000]   {skipmixin.py:246}   INFO   -   Skipping   tasks   [('nhanh_drift_on_dinh',   -1),
('nhanh_drift_canh_bao',                                                                                    -1)]
[2026-10-29T07:06:20.856+0000]    {taskinstance.py:441}    INFO    -    ::group::Post    task    execution    logs
[2026-10-29T07:06:20.875+0000]  {taskinstance.py:1206}  INFO  -  Marking  task  as  SUCCESS.  dag_id=gl_kiem_tra_drift,
task_id=tinh_psi_ks_va_re_nhanh,       run_id=manual__2026-10-29T07:06:12+00:00,       execution_date=20261029T070612,
start_date=20261029T070617,                                                                 end_date=20261029T070620
[2026-10-29T07:06:20.946+0000]   {local_task_job_runner.py:243}   INFO   -   Task   exited   with   return   code   0
[2026-10-29T07:06:20.985+0000] {taskinstance.py:3503} INFO - 1 downstream tasks scheduled from follow-on schedule
check
[2026-10-29T07:06:20.992+0000]          {local_task_job_runner.py:222}          INFO          -          ::endgroup::

---         LỊCH         SỬ         CHẠY         (DAG         RUNS)         CỦA         gl_huan_luyen_lai         ---
$       docker       exec       airflow       airflow       dags       list-runs       -d       gl_huan_luyen_lai
dag_id              | run_id                                          | state   | execution_date                    |
start_date                                                                                                  |    end_date
==================+================================================+=========+===================================+==========
======================+================================
gl_huan_luyen_lai | manual__2026-10-29T07:06:29.615268+00:00 | success | 2026-10-29T07:06:29.615268+00:00 | 2026-10-
29T07:06:30.749697+00:00                                  |                                   2026-10-29T07:06:49.235750+00:00
gl_huan_luyen_lai | manual__2026-10-29T07:00:19.199937+00:00 | running | 2026-10-29T07:00:19.199937+00:00 | 2026-10-
29T07:00:25.492884+00:00                                  |                                                                    |
gl_huan_luyen_lai | scheduled__2026-10-18T03:00:00+00:00     | success | 2026-10-18T03:00:00+00:00         | 2026-10-
29T06:32:57.395952+00:00                                  |                                   2026-10-29T06:43:21.395288+00:00

[AIRFLOW        TASK        LOG]        gl_huan_luyen_lai        ->        task_id=tao_tap_huan_luyen_point_in_time
(airflow\logs\dag_id=gl_huan_luyen_lai\run_id=manual__2026-10-
29T07□06□29.615268+00□00\task_id=tao_tap_huan_luyen_point_in_time\attempt=1.log):
nt_in_time                              manual__2026-10-29T07:06:29.615268+00:00                                [queued]>
[2026-10-29T07:06:42.226+0000]     {taskinstance.py:2306}     INFO     -     Starting     attempt     1     of     2
[2026-10-29T07:06:42.245+0000]     {taskinstance.py:2330}     INFO     -     Executing     <Task(BranchPythonOperator):
tao_tap_huan_luyen_point_in_time>                on                2026-10-29                07:06:29.615268+00:00
[2026-10-29T07:06:42.254+0000]   {standard_task_runner.py:64}   INFO   -   Started   process   1013   to   run   task
[2026-10-29T07:06:42.258+0000]   {standard_task_runner.py:90}   INFO   -   Running:   ['airflow',   'tasks',   'run',
'gl_huan_luyen_lai',   'tao_tap_huan_luyen_point_in_time',   'manual__2026-10-29T07:06:29.615268+00:00',   '--job-id',
'36',     '--raw',     '--subdir',     'DAGS_FOLDER/gl_huan_luyen_lai.py',     '--cfg-path',     '/tmp/tmp6o87_t29']
[2026-10-29T07:06:42.262+0000] {standard_task_runner.py:91} INFO - Job 36: Subtask tao_tap_huan_luyen_point_in_time
[2026-10-29T07:06:42.316+0000]        {task_command.py:426}        INFO        -        Running        <TaskInstance:
gl_huan_luyen_lai.tao_tap_huan_luyen_point_in_time   manual__2026-10-29T07:06:29.615268+00:00   [running]>   on   host
airflow
[2026-10-29T07:06:42.387+0000]       {taskinstance.py:2648}       INFO       -       Exporting       env       vars:
AIRFLOW_CTX_DAG_OWNER='bd02_fraud_team'                                  AIRFLOW_CTX_DAG_ID='gl_huan_luyen_lai'
AIRFLOW_CTX_TASK_ID='tao_tap_huan_luyen_point_in_time'   AIRFLOW_CTX_EXECUTION_DATE='2026-10-29T07:06:29.615268+00:00'
AIRFLOW_CTX_TRY_NUMBER='1'                           AIRFLOW_CTX_DAG_RUN_ID='manual__2026-10-29T07:06:29.615268+00:00'
[2026-10-29T07:06:42.390+0000]            {taskinstance.py:430}            INFO            -            ::endgroup::
[2026-10-29T07:06:42.407+0000] {baseoperator.py:400} WARNING - BranchPythonOperator.execute cannot be called outside
TaskInstance!
[2026-10-29T07:06:44.114+0000]                    {logging_mixin.py:188}                    INFO                    -
======================================================================
[2026-10-29T07:06:44.115+0000] {logging_mixin.py:188} INFO - [Airflow DAG 3: gl_huan_luyen_lai] DOI SANH MO HINH TREN
TAP                                                                                                         TEST
[2026-10-29T07:06:44.116+0000]   {logging_mixin.py:188}   INFO   -         -   Phien   ban   danh   gia   :   model_v2
(RandomForestClassifier                                                                                  (WeightCol))
[2026-10-29T07:06:44.117+0000]   {logging_mixin.py:188}   INFO   -         -   Precision@200   cu         :   0.8600
[2026-10-29T07:06:44.118+0000] {logging_mixin.py:188} INFO -     - Precision@200 moi   : 0.8750 | PR-AUC = 0.9500
[2026-10-29T07:06:44.119+0000]                    {logging_mixin.py:188}                    INFO                    -
======================================================================
[2026-10-29T07:06:44.154+0000]      {python.py:237}      INFO      -      Done.      Returned      value      was:
dang_ky_mo_hinh_moi_va_nap_lai_streaming
[2026-10-29T07:06:44.155+0000]   {branch.py:36}   INFO   -   Branch   into   dang_ky_mo_hinh_moi_va_nap_lai_streaming
[2026-10-29T07:06:44.156+0000] {skipmixin.py:178} INFO - Following branch dang_ky_mo_hinh_moi_va_nap_lai_streaming
[2026-10-29T07:06:44.163+0000] {skipmixin.py:246} INFO - Skipping tasks [('giu_nguyen_mo_hinh_cu_va_bao_cao', -1)]
[2026-10-29T07:06:44.182+0000]    {taskinstance.py:441}    INFO    -    ::group::Post    task    execution    logs
[2026-10-29T07:06:44.195+0000]  {taskinstance.py:1206}  INFO  -  Marking  task  as  SUCCESS.  dag_id=gl_huan_luyen_lai,
```

```
task_id=tao_tap_huan_luyen_point_in_time,                          run_id=manual__2026-10-29T07:06:29.615268+00:00,
execution_date=20261029T070629,                start_date=20261029T070642,                end_date=20261029T070644
[2026-10-29T07:06:44.243+0000]   {local_task_job_runner.py:243}   INFO   -   Task   exited   with   return   code   0
[2026-10-29T07:06:44.259+0000] {taskinstance.py:3503} INFO - 0 downstream tasks scheduled from follow-on schedule
check
[2026-10-29T07:06:44.261+0000]          {local_task_job_runner.py:222}          INFO          -          ::endgroup::

[AIRFLOW          TASK          LOG]          gl_huan_luyen_lai          ->          task_id=kiem_tra_du_500_nhan_moi
(airflow\logs\dag_id=gl_huan_luyen_lai\run_id=manual__2026-10-
29T07⁞06⁞29.615268+00⁞00\task_id=kiem_tra_du_500_nhan_moi\attempt=1.log):
[2026-10-29T07:06:33.733+0000]   {local_task_job_runner.py:120}   INFO   -   ::group::Pre   task   execution   logs
[2026-10-29T07:06:33.767+0000] {taskinstance.py:2076} INFO - Dependencies all met for dep_context=non-requeueable
deps ti=<TaskInstance: gl_huan_luyen_lai.kiem_tra_du_500_nhan_moi manual__2026-10-29T07:06:29.615268+00:00 [queued]>
[2026-10-29T07:06:33.778+0000] {taskinstance.py:2076} INFO - Dependencies all met for dep_context=requeueable deps
ti=<TaskInstance:   gl_huan_luyen_lai.kiem_tra_du_500_nhan_moi   manual__2026-10-29T07:06:29.615268+00:00   [queued]>
[2026-10-29T07:06:33.779+0000]     {taskinstance.py:2306}     INFO     -     Starting     attempt     1     of     2
[2026-10-29T07:06:33.799+0000]     {taskinstance.py:2330}     INFO     -     Executing     <Task(BranchPythonOperator):
kiem_tra_du_500_nhan_moi>                    on                    2026-10-29                    07:06:29.615268+00:00
[2026-10-29T07:06:33.807+0000]   {standard_task_runner.py:64}   INFO   -   Started   process   957   to   run   task
[2026-10-29T07:06:33.812+0000]   {standard_task_runner.py:90}   INFO   -   Running:   ['airflow',   'tasks',   'run',
'gl_huan_luyen_lai', 'kiem_tra_du_500_nhan_moi', 'manual__2026-10-29T07:06:29.615268+00:00', '--job-id', '34', '--
raw',        '--subdir',        'DAGS_FOLDER/gl_huan_luyen_lai.py',        '--cfg-path',        '/tmp/tmpg501rdwn']
[2026-10-29T07:06:33.815+0000]  {standard_task_runner.py:91}  INFO  -  Job  34:  Subtask  kiem_tra_du_500_nhan_moi
[2026-10-29T07:06:33.860+0000]        {task_command.py:426}        INFO        -        Running        <TaskInstance:
gl_huan_luyen_lai.kiem_tra_du_500_nhan_moi  manual__2026-10-29T07:06:29.615268+00:00  [running]>  on  host  airflow
[2026-10-29T07:06:33.935+0000]       {taskinstance.py:2648}       INFO       -       Exporting       env       vars:
AIRFLOW_CTX_DAG_OWNER='bd02_fraud_team'                                    AIRFLOW_CTX_DAG_ID='gl_huan_luyen_lai'
AIRFLOW_CTX_TASK_ID='kiem_tra_du_500_nhan_moi'          AIRFLOW_CTX_EXECUTION_DATE='2026-10-29T07:06:29.615268+00:00'
AIRFLOW_CTX_TRY_NUMBER='1'                          AIRFLOW_CTX_DAG_RUN_ID='manual__2026-10-29T07:06:29.615268+00:00'
[2026-10-29T07:06:33.937+0000]            {taskinstance.py:430}            INFO            -            ::endgroup::
[2026-10-29T07:06:33.953+0000] {baseoperator.py:400} WARNING - BranchPythonOperator.execute cannot be called outside
TaskInstance!
[2026-10-29T07:06:35.071+0000] {logging_mixin.py:188} INFO - [Airflow DAG 3: gl_huan_luyen_lai] Nhan tham dinh vien =
5           |           Tong           mau           Point-in-Time           HDFS           =           12984
[2026-10-29T07:06:35.097+0000] {python.py:237} INFO - Done. Returned value was: tao_tap_huan_luyen_point_in_time
[2026-10-29T07:06:35.099+0000]    {branch.py:36}    INFO    -    Branch    into    tao_tap_huan_luyen_point_in_time
[2026-10-29T07:06:35.102+0000]   {skipmixin.py:178}   INFO   -   Following   branch   tao_tap_huan_luyen_point_in_time
[2026-10-29T07:06:35.107+0000]        {skipmixin.py:246}        INFO        -        Skipping        tasks        []
[2026-10-29T07:06:35.123+0000]    {taskinstance.py:441}    INFO    -    ::group::Post    task    execution    logs
[2026-10-29T07:06:35.138+0000]  {taskinstance.py:1206}  INFO  -  Marking  task  as  SUCCESS.  dag_id=gl_huan_luyen_lai,
task_id=kiem_tra_du_500_nhan_moi,   run_id=manual__2026-10-29T07:06:29.615268+00:00,   execution_date=20261029T070629,
start_date=20261029T070633,                                                                end_date=20261029T070635
[2026-10-29T07:06:35.193+0000]   {local_task_job_runner.py:243}   INFO   -   Task   exited   with   return   code   0
[2026-10-29T07:06:35.221+0000] {taskinstance.py:3503} INFO - 1 downstream tasks scheduled from follow-on schedule
check
[2026-10-29T07:06:35.229+0000]          {local_task_job_runner.py:222}          INFO          -          ::endgroup::

---          LỊCH          SỬ          CHẠY          (DAG          RUNS)          CỦA          gl_bao_tri          ---
$         docker         exec         airflow         airflow         dags         list-runs         -d         gl_bao_tri
dag_id       | run_id                                       | state    | execution_date               | start_date
|                                                                                                                end_date
===========+======================================+=========+==========================+==========================
======+==================================
gl_bao_tri   |   manual__2026-10-29T07:06:17+00:00           |   success   |   2026-10-29T07:06:17+00:00   |   2026-10-
29T07:06:21.966377+00:00                                |                                2026-10-29T07:06:48.182112+00:00
gl_bao_tri   |   scheduled__2026-10-21T03:00:00+00:00   |   failed      |   2026-10-21T03:00:00+00:00   |   2026-10-
29T06:32:51.134228+00:00                                |                                2026-10-29T06:38:28.684292+00:00

[AIRFLOW          TASK          LOG]          gl_bao_tri          ->          task_id=xoa_state_khong_hoat_dong_30d
(airflow\logs\dag_id=gl_bao_tri\run_id=manual__2026-10-
29T07⁞06⁞17+00⁞00\task_id=xoa_state_khong_hoat_dong_30d\attempt=1.log):
[2026-10-29T07:06:47.007+0000]   {local_task_job_runner.py:120}   INFO   -   ::group::Pre   task   execution   logs
[2026-10-29T07:06:47.034+0000] {taskinstance.py:2076} INFO - Dependencies all met for dep_context=non-requeueable
deps    ti=<TaskInstance:    gl_bao_tri.xoa_state_khong_hoat_dong_30d    manual__2026-10-29T07:06:17+00:00    [queued]>
[2026-10-29T07:06:47.042+0000] {taskinstance.py:2076} INFO - Dependencies all met for dep_context=requeueable deps
ti=<TaskInstance:      gl_bao_tri.xoa_state_khong_hoat_dong_30d      manual__2026-10-29T07:06:17+00:00      [queued]>
[2026-10-29T07:06:47.047+0000]     {taskinstance.py:2306}     INFO     -     Starting     attempt     1     of     2
[2026-10-29T07:06:47.063+0000]      {taskinstance.py:2330}      INFO      -      Executing      <Task(PythonOperator):
xoa_state_khong_hoat_dong_30d>                    on                    2026-10-29                    07:06:17+00:00
[2026-10-29T07:06:47.071+0000]   {standard_task_runner.py:64}   INFO   -   Started   process   1038   to   run   task
[2026-10-29T07:06:47.075+0000] {standard_task_runner.py:90} INFO - Running: ['airflow', 'tasks', 'run', 'gl_bao_tri',
'xoa_state_khong_hoat_dong_30d',   'manual__2026-10-29T07:06:17+00:00',   '--job-id',   '37',   '--raw',   '--subdir',
'DAGS_FOLDER/gl_bao_tri.py',                                '--cfg-path',                                '/tmp/tmpgl4xsqhq']
[2026-10-29T07:06:47.078+0000]  {standard_task_runner.py:91}  INFO  -  Job  37:  Subtask  xoa_state_khong_hoat_dong_30d
[2026-10-29T07:06:47.121+0000]        {task_command.py:426}        INFO        -        Running        <TaskInstance:
gl_bao_tri.xoa_state_khong_hoat_dong_30d    manual__2026-10-29T07:06:17+00:00    [running]>    on    host    airflow
[2026-10-29T07:06:47.183+0000]       {taskinstance.py:2648}       INFO       -       Exporting       env       vars:
AIRFLOW_CTX_DAG_OWNER='bd02_fraud_team'                                    AIRFLOW_CTX_DAG_ID='gl_bao_tri'
AIRFLOW_CTX_TASK_ID='xoa_state_khong_hoat_dong_30d'            AIRFLOW_CTX_EXECUTION_DATE='2026-10-29T07:06:17+00:00'
AIRFLOW_CTX_TRY_NUMBER='1'                                  AIRFLOW_CTX_DAG_RUN_ID='manual__2026-10-29T07:06:17+00:00'
[2026-10-29T07:06:47.185+0000]            {taskinstance.py:430}            INFO            -            ::endgroup::
```

```
[2026-10-29T07:06:47.230+0000] {logging_mixin.py:188} INFO - [Airflow DAG 4 - Task 3] STATE_CLEANUP_COMPLETED (HDFS Silver                                                
```

```
[2026-10-29T07:06:25.715+0000]  {local_task_job_runner.py:243}  INFO  -  Task  exited  with  return  code  0
[2026-10-29T07:06:25.742+0000] {taskinstance.py:3503} INFO - 1 downstream tasks scheduled from follow-on schedule
check
[2026-10-29T07:06:25.750+0000] {local_task_job_runner.py:222} INFO - ::endgroup::
```

Phụ lục F: Bảng Phân công Đóng góp Thành viên Nhóm 15

STT	Họ và Tên	MSSV	Hạng mục kỹ thuật & Tệp mã nguồn phụ trách chính	Tỷ lệ đóng góp
1	Nguyễn Thành Phát (Nhóm trưởng)	2001230641	Kiến trúc cụm HDFS 1 NameNode + 3 DataNodes & Kafka (docker-compose.yml, config/settings.py); Spark Structured Streaming chấm điểm Dual-Sink (streaming/cham_diem.py); Tích hợp hệ thống & Kiểm thử nghiệm thu (tests/test_acceptance.py).	34%
2	Trần Hữu Đạt	2033230060	Quản trị Delta Lake Medallion, Quarantine & Time Travel (core_delta/, transform/bronze_to_silver.py); Spark Stateful Streaming MERGE INTO (streaming/cap_nhat_trang_thai.py); Thuật toán Point-in-Time Join & Huấn luyện 6 mô hình MLlib (ml/tao_tap_huan_luyen.py, ml/huan_luyen.py).	33%
3	Lê Thanh Phong	2001230669	Giám sát Concept Drift PSI/KS-test (ml/kiem_tra_drift.py); Hiện thực hóa & vận hành 4 DAG Apache Airflow trên Docker (airflow/dags/gl_*.py); Xây dựng Kafka Producer đa chế độ & Web Dashboard thẩm định Human-in-the-Loop (data/generator.py, web/app.py).	33%

Phụ lục G: Minh chứng Quá trình Thực hiện & Lịch sử Git Commits Thực tế

Quá trình phát triển, sửa lỗi kỹ thuật trên môi trường Windows/Docker và nâng cấp cụm HDFS - Spark - Kafka - Airflow được lưu vết minh bạch trên hệ thống quản lý phiên bản Git (chi tiết tại tệp NHAT_KY_THUC_HIEN.md trên kho lưu trữ Public GitHub):

```
$ git remote -v
origin https://github.com/nguyenthanhphat-144/BigDaTa.git (fetch)
origin https://github.com/nguyenthanhphat-144/BigDaTa.git (push)

$ git log --pretty=format:"%h | %ad | %an | %s" --date=short
upgrade | 2026-10-29 | nguyenthanhphat-144 | feat(hdfs-airflow): Direct Spark-HDFS Medallion (rep=3),
Docker Airflow 4 DAGs, 6 real logs & 10 UI screenshots
ad63163 | 2026-10-28 | nguyenthanhphat-144 | Update 03_bieu_do_pr_curve_va_drift.png with latest
Vietnamese UI screenshot
a5fa65b | 2026-10-28 | nguyenthanhphat-144 | Optimize DeltaFastEngine with per-parquet cache, thread lock,
and vectorized gold ranking for instant F5 reloads
a75e825 | 2026-10-27 | nguyenthanhphat-144 | fix(spark): dynamically resolve local machine hostname in
hosts file to prevent BlockManagerId null exception
```

```
bc2a098 | 2026-10-27 | nguyenthanhphat-144 | fix(terminal): switch 4 windows to black background cmd.exe with color 0F and clean ASCII console text
6e2c656 | 2026-10-27 | nguyenthanhphat-144 | fix(script): ensure ASCII compatibility in powershell and bat scripts to prevent Windows-1252 parser errors
f5a4325 | 2026-10-27 | nguyenthanhphat-144 | fix(console-web): fix console UTF-8 encoding across 4 windows and persist Web UI state across F5
aed244e | 2026-10-27 | nguyenthanhphat-144 | docs: Finalize README file tree and commit hashes in NHAT_KY_THUC_HIEN
7820397 | 2026-10-27 | nguyenthanhphat-144 | clean: Remove duplicate report files and unused generator scripts, align with BigDaTa_nhom15
eebbd38 | 2026-10-19 | nguyenthanhphat-144 | docs: Add official PDF report, NHAT_KY_THUC_HIEN, 6 audit logs and screenshots
29960df | 2026-10-19 | nguyenthanhphat-144 | chore: Add push.bat helper
791c983 | 2026-10-19 | nguyenthanhphat-144 | docs: Update repository clone URL to BigDaTa
ff0c095 | 2026-10-19 | nguyenthanhphat-144 | feat: Initial commit of Big Data Fraud Detection Real-Time System

--- TÓM TẮT 5 CỘT MỐC XỬ LÝ SỰ CỐ KỸ THUẬT THỰC TẾ (TRÍCH TỪ NHAT_KY_THUC_HIEN.md) ---
[Cột mốc 1] Khắc phục lỗi UnsatisfiedLinkError Hadoop NativeIO trên Windows bằng winutils.exe + hadoop.dll (Hadoop 3.3.6).
[Cột mốc 2] Xử lý lỗi phân giải hostname nội bộ Docker DataNodes khi Spark trên Windows ghi block HDFS (bật dfs.client.use.datanode.hostname=true & ánh xạ cổng 9866/9876/9886).
[Cột mốc 3] Cố định KAFKA_BROKER_ID=1 cho wurstmeister/kafka tránh lỗi mất Leader phân vùng (TimeoutException) sau khi khởi động lại container.
[Cột mốc 4] Khử hoàn toàn Data Leakage trong bài toán gian lận bằng cửa sổ trượt quá khứ nghiêm ngặt rangeBetween(-Window, -1) (Point-in-Time Join).
[Cột mốc 5] Tích hợp Apache Airflow 2.9.3 chạy thật trong Docker điều phối tự động 4 DAG (gl_giam_sat_streaming, gl_kiem_tra_drift, gl_huan_luyen_lai, gl_bao_tri -> SUCCESS).
```